



School of AI & Data for Business & Society

Paris Region · Nice Côte d'Azur

Grande Ecole Programme PGE5 – AI & Data Science

FINAL YEAR PROJECT REPORT (PFE)

Synthetic Data Generation for Robust Informal Wolof Processing

A Controlled Evaluation through Text Normalization

Student:	Ibrahima THIOYE
Academic Supervisor:	Antoun YAACOUB (aivancity)
Project Setting:	Independent academic project
Academic Year:	2025–2026

Defence date: 24th of September 2026

Acknowledgements

First and foremost, I thank **Allah** the Most High for granting me the strength, patience, and perseverance needed to complete this project.

I am deeply grateful to my parents, **Abdoulaye THIOYE and Diariétou DI-AGNE** for their constant support, sacrifices, encouragement, financing and confidence throughout my studies. Their presence has been essential at every stage of this journey.

I sincerely thank my academic supervisor, **Profesor Antoun YAACOUB**, for the guidance, availability, constructive feedback, and trust provided throughout the project. I also thank the aivancity academic and administrative teams for their teaching, support, and assistance during the programme.

Finally, I thank my friends for their encouragement, discussions, patience, and moral support throughout the five years of this program.

Abstract

Most Wolof natural language processing resources contain standardized text, while users often write with informal orthography, phonetic spellings, omitted spaces, repeated characters, names, and French–Wolof code-switching. This mismatch can hinder normalization and downstream systems. Because no large parallel corpus links informal forms to standardized equivalents, I examine whether synthetic formal-to-informal data can help a model process authentic comments.

A gold-standard benchmark of 201 manually normalized YouTube comments from five videos was constructed. Video-disjoint training, development, and test partitions contain 142, 30, and 29 sentences, while the training partition includes 1,315 occurrence-level linguistic annotations. Five strategies were compared: deterministic spelling rules, bilingual alignment, distributional code-mixing, an external dataset generated with a large language model, and a proposed training-only linguistic engine using error categories, part-of-speech information, language identification, and entity protection. Every strategy supplied the same 3,330 formal targets.

Usefulness was measured through a shared character-level edit Transformer with 2.15 million parameters. Each synthetic condition received 20 pretraining epochs followed by 20 fine-tuning epochs on the same authentic training set. Development data selected checkpoints, which were evaluated once on the frozen test set using character error rate (CER), word error rate (WER), character n-gram F-score (chrF), exact match, and edit-based correction measures.

Gold-only training reproduced the unchanged-input baseline at 0.220 CER. Every synthetic condition reduced CER; the external Oolel data performed best at 0.191, followed by deterministic rules at 0.195. However, all synthetic conditions worsened WER, and correction F1 remained below 0.031. Synthetic pretraining therefore transfers useful character correspondences, but the limited authentic data and lightweight evaluator do not yet support reliable contextual or word-level normalization. The benchmark nevertheless provides a reproducible basis for stronger multilingual evaluators and future semantic-retrieval experiments.

Keywords: *Wolof, text normalization, synthetic data, low-resource NLP, code-switching, Transformer.*

Résumé

La plupart des ressources informatiques en wolof reposent sur des textes standardisés, alors que les usages réels comportent des écritures phonétiques, des omissions d'espaces, des répétitions et de l'alternance codique français–wolof. Ce décalage nuit à la normalisation et aux applications. Faute de grand corpus parallèle, ce projet évalue si des exemples synthétiques générés du formel vers l'informel peuvent aider à traiter des commentaires authentiques.

Un corpus de 201 commentaires YouTube provenant de cinq vidéos a été normalisé manuellement. Les partitions d'entraînement, de développement et de test, séparées par vidéo, contiennent 142, 30 et 29 phrases. Les données d'entraînement comprennent 1 315 annotations linguistiques. Cinq stratégies sont comparées : règles orthographiques, alignement bilingue, mélange distributionnel de langues, corpus externe produit par un grand modèle de langage et moteur linguistique appris sur les annotations d'entraînement. Toutes utilisent 3 330 cibles formelles.

Un Transformer d'édition au niveau caractère, comportant 2,15 millions de paramètres, mesure l'utilité des corpus. Chaque condition synthétique reçoit 20 époques de préentraînement, puis 20 époques d'affinage sur les données authentiques. Les points de contrôle sélectionnés sur le développement sont évalués une fois sur le test gelé. Les mesures comprennent le taux d'erreur caractère (CER), le taux d'erreur mot (WER), le score F sur les n-grammes de caractères (chrF), la correspondance exacte et la correction d'erreurs.

L'entraînement limité aux données authentiques reproduit l'entrée inchangée, avec un CER de 0,220. Toutes les conditions synthétiques réduisent ce taux. Oolel obtient 0,191, devant les règles déterministes à 0,195. Toutefois, toutes dégradent le WER et leur F1 de correction reste inférieur à 0,031. Le préentraînement synthétique transmet donc des correspondances utiles au niveau caractère, sans permettre encore une normalisation contextuelle fiable. Le benchmark fournit néanmoins une base reproductible pour des évaluateurs multilingues et de futures expériences de recherche sémantique.

Mots-clés : *wolof, normalisation de texte, données synthétiques, traitement automatique des langues peu dotées, alternance codique, Transformer.*

Contents

Acknowledgements	I
Abstract	II
Résumé	III
List of Figures	IX
List of Tables	IX
List of Abbreviations and Acronyms	XI
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement and Objectives	3
1.2.1 Problem Statement	4
1.2.2 Research Hypotheses	5
1.2.3 SMART Objectives	5
1.3 Report Structure	7
2 State of the Art	8
2.1 Theoretical Foundations	8
2.1.1 Low-Resource NLP and Domain Mismatch	8
2.1.2 Non-Standard Text and the Meaning of Noise	9
2.1.3 Text Normalization and Spelling Correction	10
2.1.4 Text Style Transfer as a Related Formulation	11
2.1.5 Code-Switching and Language Ambiguity	11
2.1.6 Synthetic Data Generation	12
2.1.7 Unsupervised Lexical-Variant Mining	13
2.1.8 Sequence-to-Sequence Models and Transformers	14
2.1.9 Synthetic Pretraining Followed by Gold Fine-Tuning	15
2.1.10 Lexical Alignment for Controlled Code-Switching	16
2.1.11 Evaluation of Normalization and Synthetic Data	16
2.2 Related Work	18
2.2.1 Wolof Linguistic Processing and Normalization	18
2.2.2 Rule-Based Synthetic Error Generation	19

2.2.3	Unsupervised Normalization and Variant Mining	19
2.2.4	Synthetic Code-Mixing	20
2.2.5	LLM-Generated Wolof Synthetic Data	21
2.2.6	Unsupervised Text Style Transfer	22
2.2.7	Comparative Synthesis	22
2.3	Positioning of This Work	24
2.3.1	Identified Research Gap	24
2.3.2	Baseline Families	24
2.3.3	Controlled Comparison Philosophy	25
2.3.4	Supervision and Data Leakage	26
2.3.5	Why Full Unsupervised Style Transfer Is Not the Core Method . . .	26
2.3.6	Relationship to Semantic Retrieval	27
2.3.7	Summary of the Positioning	28
3	Project Context and Scope	29
3.1	Academic Setting and Responsibility	29
3.2	Scope of the Project	29
3.2.1	Initial Objective and Evolution	29
3.2.2	Project Tasks and Personal Contributions	30
3.2.3	Project Schedule	31
3.2.4	Expected Deliverables	32
3.2.5	Completed Contributions at the Time of Writing	33
3.3	Data and Constraints	33
3.3.1	Data Inventory	33
3.3.2	Data Quality Constraints	36
3.3.3	Personal Data, Ethics, and Legal Considerations	37
3.3.4	Technical and Infrastructure Constraints	38
3.3.5	Budget and Time Constraints	40
3.3.6	Scientific Validity Constraints	40
4	Methodology	42
4.1	Overall Experimental Architecture	43
4.2	Data Construction	45
4.2.1	Formal Resources	45
4.2.2	YouTube Collection	45
4.2.3	Cleaning and Heuristic Language Filtering	46
4.2.4	Local LLM Candidate Filtering	46
4.2.5	Sentence-Level Gold Annotation	47
4.2.6	Gold-Training Linguistic Annotation	47
4.2.7	Video-Disjoint Gold Split and Corrections	48

4.3	Synthetic Generation Conditions	48
4.3.1	M0: Identity Reference	48
4.3.2	M1: Beqi-Inspired Deterministic Rules	49
4.3.3	M2: Eflomal Alignment Plus Shared Rules	49
4.3.4	M3: Distributional Code-Mixing Plus Rules	50
4.3.5	M4: External Oolel LLM-Generated Data	50
4.3.6	M5: Excluded Unsupervised Lexical-Mining Prototype	51
4.3.7	M6: Gold-Train-Informed Linguistic Engine	51
4.3.8	Common-Target Matching and Quality-Gate Policy	52
4.4	Downstream Normalization Model	53
4.4.1	Evaluator Selection	53
4.4.2	TNT-Inspired Character Edit Transformer	54
4.4.3	Two-Stage Training Protocol	54
4.4.4	Checkpointing, Resumption, and Selective Reruns	56
4.5	Training Environment	57
4.6	Evaluation Methodology	57
4.6.1	Intrinsic Synthetic-Corpus Analysis	57
4.6.2	Automatic Normalization Metrics	58
4.6.3	Development Selection and Frozen Test Comparison	59
4.6.4	External Oolel-Corrector Comparison	59
4.6.5	Final Evaluation State	60
5	Results and Analysis	61
5.1	Authentic Gold-Training Transformations	61
5.1.1	Annotation Coverage and Language Composition	61
5.1.2	Error Inventory	62
5.1.3	Part-of-Speech Differences	63
5.2	Intrinsic Comparison of Training Corpora	63
5.2.1	Perturbation Strength	64
5.2.2	Transformation Coverage and Mismatch	65
5.2.3	Edit-Operation Supervision	66
5.3	Development-Set Selection	66
5.4	Frozen Test Results	67
5.4.1	Automatic Metrics	67
5.4.2	Test-Sample Uncertainty	69
5.5	Qualitative Prediction Analysis	70
5.6	Interpretation with Respect to the Research Question	71
5.6.1	What Can Be Concluded	71
5.6.2	Gold-Informed Generation and Fairness	72
5.6.3	Data Availability and Model Complexity	72

5.7	Threats to Validity	73
5.7.1	Internal Validity	73
5.7.2	Construct Validity	73
5.7.3	External Validity	73
5.8	Chapter Summary	74
6	Conclusion and Future Work	75
6.1	Answer to the Research Question	75
6.1.1	Answers to the Operational Research Questions	75
6.2	Summary of Contributions and Objectives	76
6.3	Limitations	78
6.3.1	Gold Data and Sampling	78
6.3.2	Synthetic Data Quality	79
6.3.3	Evaluator and Metrics	79
6.3.4	Experimental and Computational Constraints	79
6.4	Future Directions	80
6.4.1	Expand and Strengthen the Gold Benchmark	80
6.4.2	Improve Synthetic Generators Without Using Held-Out Data	80
6.4.3	Use Stronger and Multiple Evaluators	81
6.4.4	Deployment, Ethics, and Sustainability	81
6.5	Final Perspective	82
	Bibliography	83
A	Implementation and Reproducibility Reference	86
A.1	Source-Code Map	86
A.2	Fair Common-Target Selection	87
A.3	Frozen-Test Guard	89
A.4	Shared Generation Metrics	90
A.5	Reproduction Sequence	92
B	Data, Methods, and Annotation Reference	93
B.1	Experimental Condition Legend	93
B.2	Core Dataset Inventory	94
B.3	Gold Split Contract	95
B.4	Sentence-Annotation Schema	95
B.5	Linguistic-Annotation Schema and Inventories	96
B.6	Transformation and Error Labels	97
B.7	Supplementary Results	98
B.7.1	Complete Gold-Training Error Inventory	99
B.7.2	Gold Change Rate by POS	99
B.7.3	Matched-Corpus Transformation Mechanisms	100

B.7.4	Matching and Leakage Audit	101
B.7.5	Runtime and Checkpoint Diagnostics	101
C	Technical Environment	102
C.1	Recorded Final-Benchmark Environment	102
C.2	Environment Separation	102
C.3	Declared Python Dependencies	103
C.4	Operational Entry Points	104
C.5	Artifact Layout	104
D	Declaration of Generative Artificial-Intelligence Use	106
D.1	Tools and Purposes	106
D.2	Extent and Limits of Assistance	107

List of Figures

4.1	Experimental architecture	44
5.1	Gold-training transformation labels	63
5.2	Intrinsic corpus distance	64
5.3	Frozen test metrics	68

List of Tables

1.1	Illustration of the domain gap between formal Wolof resources and informal user-generated text.	1
1.2	SMART objectives of the project.	6
2.1	Comparison of approaches relevant to informal Wolof normalization. . .	23
3.1	Functional schedule of the project.	32
3.2	Principal data resources and artifacts.	33
4.1	Implementation state of the experimental methodology.	43
4.2	Fixed TNT-inspired edit-Transformer benchmark configuration.	56
5.1	Principal transformation labels in the 1,315 gold-training occurrences. .	62
5.2	Intrinsic source-to-target distance of gold train and matched synthetic corpora.	64
5.3	Character-operation distribution used to train the TNT-inspired evaluator.	66
5.4	Development results used for checkpoint selection (30 sentences).	66
5.5	Frozen normalization results on the held-out gold test set (29 sentences).	67
5.6	Paired bootstrap uncertainty for test CER (5,000 resamples).	69
5.7	Representative frozen-test predictions from the TNT-inspired evaluator.	70
6.1	Final assessment of the project objectives.	78

A.1	Principal implementation units and responsibilities.	86
B.1	Meaning and final role of each project method identifier.	93
B.2	Core data artifacts at the methodology snapshot.	94
B.3	Locked gold partition used by the final experiment.	95
B.4	Fields in the append-only sentence-level gold annotation file.	96
B.5	Principal fields in the gold-training linguistic annotation file.	96
B.6	Error-label definitions displayed by the linguistic annotation interface. .	97
B.7	Complete multi-label error inventory for 1,315 gold-training occurrences.	99
B.8	Occurrence-level gold-training changes by POS.	99
B.9	Recorded transformation mechanisms in each matched 3,330-row corpus.	100
B.10	Dataset matching and held-out overlap audit.	101
B.11	Recorded training runtime and selected epoch by condition.	101
C.1	Principal environment recorded for the final benchmark.	102
C.2	Execution environments used by project components.	103
D.1	Declared uses of generative artificial intelligence.	106

List of Abbreviations and Acronyms

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BF16	Brain Floating Point, 16-bit format
CER	Character Error Rate
chrF	Character n-gram F-score
CMDR	Code-Mixing from Distributed Representations
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DL	Deep Learning
ECT	Equivalence Constraint Theory
F1	Harmonic mean of precision and recall
FN	False Negative
FP	False Positive
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
IBT	Iterative Back-Translation
JSON	JavaScript Object Notation
JSONL	JSON Lines
LID	Language Identification
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LSTM	Long Short-Term Memory network
mBART	Multilingual Bidirectional and Auto-Regressive Transformer
ML	Machine Learning
mT5	Multilingual Text-to-Text Transfer Transformer
NER	Named-Entity Recognition
NFC	Unicode Normalization Form C
NLLB	No Language Left Behind
NLP	Natural Language Processing
OOV	Out of Vocabulary

Abbreviation	Meaning
POS	Part of Speech
PFE	Projet de Fin d'Études (Final Year Project)
PGE	Programme Grande École
RAG	Retrieval-Augmented Generation
RAM	Random-Access Memory
RQ	Research Question
SHA-256	Secure Hash Algorithm with a 256-bit digest
TF32	TensorFloat-32
TNT	Text Normalization based pre-training of Transformers
TP	True Positive
UNMT	Unsupervised Neural Machine Translation
URL	Uniform Resource Locator
UTF-8	Unicode Transformation Format, 8-bit encoding
WER	Word Error Rate
WSL	Windows Subsystem for Linux
XLM-R	Cross-lingual Language Model–RoBERTa

Chapter 1

Introduction

1.1 Background and Motivation

Natural Language Processing (Natural Language Processing (NLP)) has progressed rapidly through large pretrained models, multilingual representations, and sequence-to-sequence architectures. These advances, however, remain unevenly distributed across languages and domains. A model may represent the meaning of a standardized expression while failing on an equivalent expression whose spelling and language mixture differ from those seen during training. This domain mismatch is especially consequential for low-resource languages, because their available corpora are limited and often concentrated in a small number of formal sources.

Wolof illustrates this problem. It is widely used in Senegal and by communities in neighboring countries and the diaspora, yet many resources available for computational processing consist of dictionaries, translations, news-like text, or other relatively standardized material. Wolof on social media is much more variable: it includes phonetic spellings, omitted diacritics, character substitutions, abbreviations, expressive repetitions, non-standard word boundaries, and French–Wolof code-switching. These forms are not simply meaningless noise; they can reflect pronunciation, typing constraints, community conventions, and multilingual practices [1].

This situation creates a domain gap between the language represented in conventional training resources and the language encountered in real-world usage. A simplified representation of this gap is shown in Table 1.1.

Table 1.1 – Illustration of the domain gap between formal Wolof resources and informal user-generated text.

Training distribution	Real-use distribution
Formal and standardized Wolof	Informal and code-switched Wolof
Dictionary, news, and translation text	Social-media and user-written text
Relatively consistent spelling	Variable and phonetic spelling
Mostly controlled linguistic forms	French insertions and online conventions

The consequences extend beyond spelling correction. In semantic retrieval, an

informal query may fail to retrieve a relevant formal passage despite expressing the same intent. An Large Language Model (LLM) may misinterpret an informal prompt whose spelling variants were poorly represented during pretraining. A machine-translation system may also translate standardized Wolof more reliably than an equivalent phonetic or code-switched sentence.

These issues are compounded by tokenization and language identification. Das and Gambäck [2] show that code-mixing, borrowing, and phonetic typing complicate language identification in social-media text. For Wolof specifically, Dione [3] demonstrates the importance of language-specific tokenization and normalization rules, particularly for phenomena such as clitics, multiword expressions, and word boundaries. Informal social-media language can amplify these difficulties.

I began this project with the aim of making formal Wolof resources more accessible to people who write informally. I carried out the work as an independent academic PFE under academic supervision, without a host company. I defined the project, constructed the data, implemented and ran the experiments, and interpreted the results. A normalization layer that maps informal or code-switched Wolof to a standardized representation could make both sides easier to compare and, in the longer term, support retrieval, LLM querying, and machine translation.

A direct supervised solution would require a large parallel corpus in which each informal Wolof sentence is paired with a meaning-equivalent standardized sentence. Such data are scarce. Constructing a sufficiently large dataset manually would require fluent annotators, a consistent normalization policy, extensive quality control, and substantial annotation time.

The lack of naturally paired data made that broader objective difficult to evaluate at scale. I therefore focused first on the underlying data question:

How can useful formal–informal Wolof training pairs be created when naturally paired data are scarce?

Synthetic data generation provides a possible solution. Standardized Wolof sentences can be transformed into plausible informal variants, producing parallel examples without manually rewriting every source sentence. A model can then be trained in the reverse direction, from synthetic informal input to its known standardized source. The central scientific issue is not merely whether synthetic sentences can be produced, but whether the patterns they contain transfer to authentic user-written Wolof.

This principle has already shown promise for Wolof spelling correction. Mbaye and Diallo [4] formulate Wolof spelling correction as a noisy-to-correct sequence-transduction problem and use synthetic data to compensate for the limited availability of naturally paired errors. Their work demonstrates the potential of synthetic augmentation for Wolof. However, it does not fully address the problem investigated in this project:

generating synthetic data that reproduces the spelling, lexical, spacing, and French–Wolof code-switching phenomena observed in a specific real-world social-media domain.

The final controlled comparison retains five strategies for generating synthetic informal Wolof:

- a deterministic rule baseline adapted from Wolof spelling transformations described in Beqi;
- an Eflomal word-alignment method for controlled French substitutions;
- a code-mixing method based on distributional representations (CMDR);
- an external LLM-generated dataset produced with Wolof-oriented Oolel resources [5];
- a proposed linguistic engine that estimates transformations and their distributions only from the manually annotated gold training partition.

An unsupervised lexical-mining method based on unpaired YouTube comments was implemented during the project but removed from the final controlled benchmark. Its accepted mapping inventory and coverage were too limited to justify treating it as a comparable principal condition. It is retained as an exploratory result and discussed as a design decision rather than used to support the final comparative claim.

A central methodological distinction in this work is made between **synthetic plausibility** and **synthetic usefulness**. A generated sentence may appear convincingly informal while inadvertently changing the meaning of its source sentence. Conversely, a synthetic sentence that does not perfectly imitate a naturally occurring social-media comment may still provide a useful training example if it teaches a normalization model an important correspondence.

For this reason, the principal evaluation is downstream rather than purely intrinsic. Each retained dataset is used to pretrain the same TNT-inspired character edit Transformer, after which every model is fine-tuned on the same gold training split. The resulting systems are evaluated on the same manually normalized YouTube comments. The downstream model is therefore an *evaluation instrument*: the objective is to compare what each synthetic dataset teaches under fixed conditions, not to claim that this small Transformer is the best possible Wolof normalizer. A gold-only model and an unchanged-input identity condition provide reference points. Intrinsic statistics derived from the annotation workflow complement the downstream results by showing which phenomena each generator reproduces.

1.2 Problem Statement and Objectives

1.2.1 Problem Statement

The operational problem can be summarized as follows:

Wolof NLP resources and models are predominantly exposed to formal or standardized text, whereas real users frequently produce informal and French–Wolof code-switched text. Large paired formal–informal corpora are scarce, making supervised adaptation difficult. This study asks whether synthetic formal-to-informal data can reduce that distribution gap and improve the processing of authentic informal Wolof.

This problem leads to the following central research question, framed as a comparison rather than an assumption that one generator is superior:

To what extent do different synthetic formal-to-informal Wolof generation strategies improve the normalization of authentic informal and French–Wolof code-switched YouTube comments under a fixed downstream architecture and limited gold supervision?

This central question is decomposed into five operational research questions:

- RQ1:** Does synthetic pretraining improve normalization on authentic informal Wolof compared with gold-only training and an unchanged-input identity baseline?
- RQ2:** Under a matched source pool, model initialization, training budget, and gold fine-tuning protocol, how do rule-based, alignment-based, distributional code-mixing, external LLM-generated, and gold-informed synthetic data compare?
- RQ3:** Does the proposed gold-train-informed linguistic engine provide greater downstream utility than the retained generic and external generation strategies without using development or test annotations?
- RQ4:** How closely do the edit frequency, error categories, code-switching behavior, and unchanged-output rate of each synthetic dataset correspond to the phenomena observed by the annotator in authentic comments?
- RQ5:** Which error categories and sentence characteristics are improved, left unchanged, or degraded by each training condition, and what do these failures reveal about the limitations of the synthetic data and the evaluator?

1.2.2 Research Hypotheses

Before running the benchmark, I stated the following hypotheses:

- H1.** At least one synthetic-pretraining condition was expected to improve normalization relative to gold-only training under the same architecture and gold fine-tuning budget.
- H2.** The five retained generators were expected to differ in downstream utility because they produce different edit distributions, lexical coverage, and code-switching behavior, even when trained on the same formal targets.
- H3.** The gold-train-informed generator was expected to reproduce a broader or better-calibrated subset of authentic error categories than deterministic generic rules, while its train-only construction avoided direct leakage from development and test examples.
- H4.** Apparent synthetic naturalness or transformation volume was not expected to predict downstream normalization performance on its own.
- H5.** Any gains were expected to vary across error categories and sentence lengths, which made annotator statistics and qualitative error analysis important complements to aggregate scores.

1.2.3 SMART Objectives

I translated the research questions into operational deliverables using the SMART principle: Specific, Measurable, Achievable, Relevant, and Time-bound.

Table 1.2 states each objective together with its measurable acceptance criterion.

Table 1.2 – SMART objectives of the project.

ID	Objective	Measurement / acceptance criterion
O1	Build a documented corpus of authentic informal Wolof candidates from public YouTube comments.	Preserve data provenance and make collection, cleaning, filtering, and encoding procedures reproducible.
O2	Construct a manually normalized gold benchmark using a consistent normalization policy.	Obtain at least 200 validated sentence pairs from at least five videos and create controlled train, development, and test partitions.
O3	Implement comparable synthetic data generation methods.	Produce traceable M1–M4 and M6 datasets covering deterministic rules, alignment, distributional code-mixing, external LLM generation, and the proposed gold-informed engine.
O4	Control data leakage and experimental confounding factors.	Exclude held-out annotations from generation and use the same 3,330 formal targets, initialization, architecture, optimization budget, gold fine-tuning data, and seed for every retained synthetic condition.
O5	Train and compare downstream normalization models.	Compare gold-only training with five synthetic-pretraining-plus-gold conditions using one TNT-inspired character edit Transformer and a development-only checkpoint-selection protocol.
O6	Evaluate synthetic data from complementary perspectives.	Report CER, WER, chrF, exact match, edit-based correction scores, overcorrection, annotator-derived error distributions, and qualitative prediction analysis.
O7	Ensure reproducibility of the experimental pipeline and final report.	Maintain versioned code, configurations, data lineage, experiment outputs, and sufficient documentation to reproduce the principal experiments.

The completed work produced the following contributions:

1. a documented corpus of authentic informal and code-switched Wolof comments;
2. an annotation workflow and leakage-aware gold benchmark for Wolof normalization;
3. five traceable synthetic datasets constructed from the same retained formal-sentence pool;
4. a controlled downstream comparison of rule-based, alignment-based, distributional, external LLM-generated, and gold-informed generation strategies;
5. an auditable gold-training linguistic generator retaining transformation provenance and error categories;
6. empirical evidence concerning whether, when, and for which error categories

- synthetic data improves normalization of authentic Wolof text;
7. an analysis of an excluded unsupervised YouTube lexical-mining prototype and the evidence motivating its removal from the final benchmark;
 8. a methodological foundation that can later be extended to semantic retrieval, LLM-input robustness, and machine translation.

1.3 Report Structure

The remainder of this report is organised as follows:

- **Chapter 2: State of the Art** introduces the theoretical foundations of the project, including domain shift, non-standard text, lexical normalization, code-switching, synthetic data generation, sequence-to-sequence learning, multilingual Transformers, word alignment, and evaluation. It also reviews relevant Wolof-specific and multilingual approaches and positions the proposed work relative to existing research.
- **Chapter 3: Project Context and Scope** presents the independent academic setting, explains my role and contributions, and describes the data, ethical, technical, and scientific constraints affecting the project.
- **Chapter 4: Methodology and Technical Approach** describes the complete experimental pipeline, including YouTube-comment collection and preprocessing, annotation, formal-data preparation, synthetic generation strategies, data splitting, downstream normalization training, and evaluation procedures.
- **Chapter 5: Results and Analysis** reports corpus statistics, annotator-derived error distributions, synthetic-data characteristics, downstream normalization results, qualitative errors, and relevant diagnostic experiments.
- **Chapter 6: Conclusion and Outlook** answers the research questions, compares the achieved results with the initial objectives, discusses limitations and threats to validity, and outlines possible extensions toward semantic retrieval, LLM robustness, machine translation, and additional Wolof data collection.

The appendices provide supplementary implementation details, configurations, representative code extracts, annotation material, additional results, and environment information required for reproducibility.

Chapter 2

State of the Art

This chapter reviews the concepts and previous work required to position synthetic data generation for informal Wolof normalization. The relevant literature spans low-resource natural language processing, domain mismatch, lexical normalization, code-switching, synthetic augmentation, sequence-to-sequence modelling, word alignment, and unsupervised text transformation. Some of these approaches motivate retained benchmark conditions, whereas others explain alternatives investigated and ultimately excluded from the final comparison.

The objective is not only to identify existing methods, but also to determine which assumptions are appropriate for the setting considered in this project. In particular, Wolof presents a combination of limited annotated resources, substantial variation in informal writing, and frequent interaction with French. These characteristics make it necessary to distinguish techniques that are theoretically applicable from those that can realistically be evaluated under controlled experimental conditions.

2.1 Theoretical Foundations

2.1.1 Low-Resource NLP and Domain Mismatch

The notion of a *low-resource language* is task-dependent. A language may possess dictionaries, monolingual corpora, or translation resources while still lacking the annotations required for a specific NLP task. For the present work, the main limitation is therefore not simply the total quantity of Wolof text available online. The more relevant shortage is the scarcity of representative pairs linking authentic informal Wolof to an equivalent standardized form.

This distinction is particularly important for supervised normalization. A large collection of formal Wolof sentences cannot directly teach a model how users abbreviate words, omit diacritics, merge tokens, introduce phonetic spellings, or combine Wolof with French. Similarly, a large collection of unannotated social-media comments provides examples of informal language but does not directly provide the corresponding normalized targets.

The problem can be formulated as the domain mismatch in Equation (2.1). Source-domain samples contain primarily formal or standardized Wolof, whereas target-domain inputs may contain informal spelling, unconventional spacing, abbreviations, expressive forms, and French–Wolof code-switching.

$$(x, y) \sim P_s(x, y), \quad (x, y) \sim P_t(x, y), \quad P_s(x) \neq P_t(x). \quad (2.1)$$

Here, x denotes an input sentence, y its target output, and P_s and P_t the source- and target-domain joint distributions. Because the marginal input distributions differ, patterns learned on the source domain may not generalize reliably to the target domain. The underlying semantic content may remain similar while its surface realization differs considerably.

For downstream semantic systems, this mismatch can manifest itself through different tokenization patterns, reduced lexical overlap, poorly represented subwords, unreliable semantic representations, or incorrect generations.

Three broad strategies can be used to address such a mismatch:

1. **input normalization**, where informal target-domain text is transformed toward the formal distribution;
2. **model adaptation**, where a model is explicitly exposed to target-domain variation during training;
3. **representation invariance**, where formal and informal variants are encouraged to receive similar internal representations.

2.1.2 Non-Standard Text and the Meaning of Noise

Social-media text is often described as “noisy” because it may contain unconventional spelling, abbreviations, repeated characters, emojis, unusual punctuation, and multiple languages. The term deserves some caution, however.

Al Sharou et al. [1] emphasize that non-standard forms are not necessarily meaningless corruptions. A repeated character can express emphasis, a spelling variant can reflect pronunciation, an omitted diacritic may result from keyboard constraints, and a word from another language may represent intentional code-switching.

This distinction matters for normalization because the goal is not to transform every non-standard feature indiscriminately. The normalizer targets phenomena defined as normalization errors while preserving the intended meaning of the original utterance.

Two opposite failure modes can therefore occur:

- **under-normalization**, where forms that prevent effective matching with standardized resources remain unchanged;

- **over-normalization**, where legitimate information such as named entities, emphasis, intentional French spans, or valid lexical variation is unnecessarily modified.

For this reason, normalization requires an explicit target policy. In the present project, informal Wolof orthography and selected social-media phenomena are normalized while intentional French content is preserved whenever possible. The task is consequently better described as *normalization of code-switched text* than as translation toward monolingual Wolof.

Translating every French segment would fundamentally change the problem. It would require bilingual semantic decisions and would combine lexical normalization with machine translation, making the source of model errors more difficult to isolate.

2.1.3 Text Normalization and Spelling Correction

Lexical normalization consists of mapping a non-standard word or sequence to an appropriate standardized representation. Classical spell-checking methods typically rely on edit distance, dictionaries, language models, phonetic similarity, or noisy-channel formulations.

Such techniques can be effective when the observed form is produced by a small typographical error. Informal social-media language, however, is more heterogeneous. A variant may differ substantially from the formal form because of phonetic writing, abbreviation, word merging, character repetition, or community-specific conventions.

Normalization can therefore be formulated as the sequence-transduction problem in Equation (2.2):

$$\hat{y} = \arg \max_y P(y \mid x), \quad (2.2)$$

where x denotes the informal input sequence, y a candidate standardized representation, and $\hat{y}$ the highest-probability prediction.

Unlike isolated spell checking, this formulation allows a model to learn transformations at multiple levels. A single prediction can potentially involve character substitution, token splitting or merging, lexical replacement, and context-dependent correction.

For Wolof, Mbaye and Diallo’s Beqi system follows this general perspective by generating synthetic noisy–correct pairs and training sequence-to-sequence spelling-correction models [4]. Their results establish synthetic augmentation as a relevant approach for Wolof when naturally occurring error-correction pairs are scarce.

The present problem is broader than isolated spelling correction. Authentic social-media comments may combine orthographic variation with token-boundary changes, lexicalized informal forms, and intentional code-switching. Consequently, the transformation being learned is closer to sentence-level normalization than to conventional dictionary-based spelling correction.

2.1.4 Text Style Transfer as a Related Formulation

Formal-to-informal generation can also be interpreted through the perspective of text style transfer. Text style transfer seeks to modify a particular attribute of a sentence while preserving its underlying content. Jin et al. [6] review neural methods for this problem and highlight the difficulty of jointly evaluating style strength, fluency, and content preservation.

Under this interpretation, standardized Wolof represents one surface style and informal or code-switched Wolof another. A synthetic generator attempts to transform $y_{\text{formal}} \rightarrow \tilde{x}_{\text{informal}}$ while preserving the semantic information contained in y_{formal} .

The analogy is useful because it emphasizes a central requirement of synthetic generation: a plausible informal sentence is not useful if the transformation changes the intended meaning.

However, the analogy has limits. Many transformations relevant to Wolof normalization are relatively local orthographic operations rather than unrestricted stylistic rewrites. In addition, introducing French elements may require translation and grammatical constraints that are absent from conventional monolingual style-transfer settings.

Style transfer is therefore used in this work as a conceptual framework rather than as an assumption that a complex neural style-transfer architecture is required.

2.1.5 Code-Switching and Language Ambiguity

Code-switching refers to the use of linguistic elements from more than one language within the same discourse, sentence, or constituent. In multilingual digital communication, it frequently interacts with borrowing, phonetic spelling, abbreviations, and other informal phenomena.

Das and Gambäck [2] show that language identification becomes particularly difficult in social-media text because document-level language labels are insufficient. Individual tokens may belong to different languages, and short or borrowed words can be linguistically ambiguous.

This issue is important for French–Wolof normalization. Not every French-looking token warrants transformation, and an out-of-vocabulary token is not automatically an incorrect Wolof word.

A candidate token may instead correspond to:

- an intentional French word;
- a borrowed expression;
- a named entity;
- an acronym;

- a URL;
- a number;
- a valid Wolof form absent from the available lexicon;
- or an actual non-standard Wolof variant.

An automatic generator therefore needs to determine both *how* a token can be transformed and whether any transformation is appropriate.

Linguistic approaches to code-switching, including constraints on permissible switching positions, can theoretically guide generation. Their practical application, however, generally requires reliable syntactic analyses and language-specific resources. Such resources are comparatively limited for Wolof–French informal text.

This motivates the use of more conservative lexical and alignment-based methods as controlled baselines, combined with explicit rejection criteria rather than forcing code-switching into every generated sentence.

2.1.6 Synthetic Data Generation

Synthetic data generation provides a way to construct training pairs when naturally aligned data are unavailable.

Let y denote a formal Wolof sentence. Equation (2.3) defines a synthetic generator g that produces an informal variant

$$\tilde{x} = g(y; \theta, z), \tag{2.3}$$

where θ represents the generator parameters, rules, mappings, or learned distributions, and z represents stochastic choices.

The pair $(\tilde{x}, y)$ can then be used to train a model in the opposite direction, $\tilde{x}_{\text{informal}} \longrightarrow y_{\text{formal}}$.

This strategy is attractive in low-resource settings because a relatively large formal corpus can be converted into a parallel normalization corpus without manually correcting every sentence.

Synthetic generation provides several practical advantages:

- formal monolingual data can be reused at scale;
- every synthetic source has a known clean target;
- transformations can be made reproducible;
- specific error categories can deliberately be represented;

- the quantity of generated training data can be controlled.

Nevertheless, synthetic data introduce their own distribution gap: the generated distribution $P_g(x)$ may itself differ from the true target-domain distribution $P_t(x)$, so $P_g(x) \neq P_t(x)$.

The usefulness of synthetic data therefore depends not simply on the number of generated examples but on the extent to which the transformations teach patterns that generalize to authentic inputs.

Potential weaknesses include unrealistic corruption, excessive regularity, semantic changes, incorrect modification of protected tokens, and models learning artifacts specific to the generator.

Dekker and van der Goot [7] demonstrate that carefully constructed synthetic errors can approach the usefulness of manually annotated English lexical-normalization data. Their results also show the relevance of unsupervised alternatives, supporting the comparison of both explicit generation and corpus-driven methods.

This distinction between **synthetic plausibility** and **synthetic usefulness** is central to this project. A generated sentence can appear realistic while providing a poor training signal. Conversely, a transformation that appears somewhat artificial can still teach a useful normalization correspondence.

2.1.7 Unsupervised Lexical-Variant Mining

Instead of manually defining transformations, lexical variants can be discovered directly from a large unannotated target-domain corpus.

Gouws et al. [8] investigate the unsupervised extraction of lexical variants from noisy text. Their approach illustrates how relations between informal and standard forms can be discovered without manually paired examples.

Li and Liu [9] similarly combine surface similarity with semantic information and discriminative reranking to improve unsupervised text normalization.

Roy et al. [10] propose a language-independent unsupervised normalization method and evaluate not only normalization accuracy but also its impact on downstream information retrieval and stance detection.

These approaches are relevant to this project because the collected YouTube comments constitute a large source of authentic but unpaired target-domain text. Such comments can provide evidence about spelling variants and lexical patterns that generic hand-written rules may fail to represent.

A possible relation can conceptually be expressed by the weighted score in Equation (2.4).

$$s(x, w) = \lambda_1 s_{\text{char}}(x, w) + \lambda_2 s_{\text{context}}(x, w) + \lambda_3 s_{\text{freq}}(x, w), \quad (2.4)$$

where an observed informal candidate x is compared with a possible standard form w using character similarity, contextual evidence, and frequency information. The coefficients λ_1 , λ_2 , and λ_3 weight those three signals.

However, applying this idea to Wolof raises several difficulties. A low-frequency token may not possess reliable contextual representations. Formal lexicons may be incomplete. French words and named entities may appear out of vocabulary despite requiring no correction.

For this reason, unsupervised lexical mining in a low-resource multilingual setting benefits from prioritizing high-confidence mappings rather than maximizing coverage at all costs. This creates a practical tension: conservative filtering improves precision but may leave too few changed examples for a fair downstream comparison. The prototype developed here encountered exactly this limitation, so I retained it as exploratory work rather than as a principal benchmark condition.

2.1.8 Sequence-to-Sequence Models and Transformers

Sequence-to-sequence models estimate a conditional probability distribution between an input sequence and an output sequence [11]. For normalization, the objective is to estimate $P(y_1, \dots, y_m \mid x_1, \dots, x_n)$, where $x_1, \dots, x_n$ are the input symbols and $y_1, \dots, y_m$ are the output symbols.

Earlier neural sequence-transduction systems frequently relied on recurrent architectures such as Long Short-Term Memory networks (LSTMs). Transformers replaced recurrent computation with self-attention, allowing each sequence position to directly model relationships with other positions [12].

The scaled dot-product attention mechanism is defined in Equation (2.5).

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}, \quad (2.5)$$

where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ represent the query, key, and value matrices, and d_k is the key dimension.

Contextual modelling is relevant to normalization because the correct standardized form of a token can depend on surrounding words rather than on character similarity alone.

Pretrained text-to-text models can reduce the amount of task-specific supervision required. T5 casts multiple NLP tasks into a unified text-to-text framework [13]; mT5 extends this formulation to multilingual pretraining [14]; and mBART applies multilingual denoising pretraining to sequence-to-sequence transfer [15]. These architectures were credible candidates for the downstream evaluator because they offer contextual representations learned before task-specific fine-tuning.

Multilingual pretraining nevertheless introduces two difficulties for the present

comparison. First, it does not guarantee strong or balanced Wolof coverage. Second, large pretrained models substantially increase training time when several synthetic conditions are rerun under an identical protocol. Preliminary experiments with mT5, including parameter-efficient fine-tuning, were computationally expensive relative to the small gold benchmark and produced unstable generations. These experiments remain useful feasibility evidence, but mT5 was not retained as the final common evaluator.

Tokenization presents an additional design choice. Informal spellings create surface forms that may be rare or absent from a fixed word or subword vocabulary. Character-level modelling avoids out-of-vocabulary informal tokens and represents diacritic deletion, repeated letters, substitutions, and word-boundary changes directly. Mbaye and Diallo [4] likewise show the relevance of segmentation choices for Wolof spelling correction. Character sequences are longer, however, and conventional autoregressive decoding can accumulate errors. A small character model is therefore useful as a controlled evaluator without being a state-of-the-art normalizer.

The final benchmark therefore uses a small TNT-inspired bidirectional Transformer that predicts source-anchored character-edit operations and replacement segments. This non-autoregressive choice makes repeated controlled training feasible, avoids free-decoding loops, and gives every synthetic method the same vocabulary, initialization, capacity, and optimization budget. Its purpose is comparative sensitivity: the model acts as a common probe of the training signal supplied by each dataset. The architecture choice does not imply that multilingual pretraining is generally inferior; earlier autoregressive collapse and copy-aware identity behavior are retained as diagnostic architecture experiments rather than benchmark results.

2.1.9 Synthetic Pretraining Followed by Gold Fine-Tuning

A useful training strategy in low-resource settings is to first expose a model to a comparatively large synthetic dataset and then fine-tune it on a smaller amount of authentic manually annotated data.

If $\mathcal{D}_{synth}$ denotes a synthetic dataset and $\mathcal{D}_{gold}$ a manually normalized dataset, Equation (2.6) represents the two-stage training sequence.

$$\theta_0 \xrightarrow{\mathcal{D}_{synth}} \theta_{synth} \xrightarrow{\mathcal{D}_{gold}} \theta_{final}. \quad (2.6)$$

Here, θ_0 , θ_{synth} , and θ_{final} denote the initial, synthetic-pretrained, and gold-fine-tuned parameter states. The synthetic stage can expose the model to a larger variety of transformations, while the second stage adapts the model toward the authentic target distribution.

Jawahar et al. [16] provide a relevant example in English-to-Hinglish generation. Their work explores multilingual text-to-text Transformers and reports benefits from a

training curriculum involving synthetic code-mixed data followed by gold data.

Although Hinglish and French–Wolof are linguistically different settings, the experimental principle is transferable: the value of synthetic data can be assessed by whether they improve a model after the same authentic fine-tuning stage.

2.1.10 Lexical Alignment for Controlled Code-Switching

Parallel French–Wolof corpora can provide another source of transformations by identifying lexical translation correspondences.

Statistical word alignment estimates associations between tokens in parallel sentences. Classical IBM alignment models provide a probabilistic foundation for estimating translation and alignment parameters [17].

More recent tools such as Eflomal perform efficient Bayesian word alignment using Markov Chain Monte Carlo methods [18].

An alignment model can therefore provide candidate correspondences such as $w_{\text{Wolof}} \leftrightarrow w_{\text{French}}$, which can subsequently be used to produce controlled lexical code-switches.

This approach is attractive because it uses observed bilingual correspondences instead of manually constructing every translation dictionary entry. It is also substantially simpler than training a complete machine-translation system when the immediate objective is only to obtain lexical candidates.

However, alignment probabilities do not guarantee that replacing a word in a new sentence produces a grammatically or semantically appropriate code-switch. Frequent function words can have diffuse alignments, and lexical translations are often context-dependent.

Word alignment is therefore best treated as a candidate-generation mechanism rather than as ground truth.

Distributional representations offer a related but distinct source of bilingual candidates. Character n-gram word embeddings can associate lexically or semantically related forms even when a direct alignment probability is weak. A code-mixing system may combine this similarity with sentence-level co-occurrence and positional compatibility when selecting a replacement. The advantage is broader candidate coverage; the risk is that distributional neighbours may be topically related without being valid translations or grammatical substitutes. This distinction motivates keeping probabilistic alignment and CMDR-style distributional selection as separate benchmark conditions.

2.1.11 Evaluation of Normalization and Synthetic Data

No single metric captures all aspects of text normalization.

Character Error Rate (**CER**) measures character-level editing and is particularly sensitive to spelling corrections. If S , D , and I denote substitutions, deletions, and insertions relative to a reference containing N characters, then $CER = (S + D + I)/N$.

Word Error Rate (**WER**) applies the analogous token-level formulation $WER = (S_w + D_w + I_w)/N_w$, where the subscript w marks word-level counts.

WER therefore captures changes in token identity and word boundaries more directly than CER.

Exact-match accuracy provides a stricter sentence-level measure: over N evaluated sentences, $EM = N^{-1} \sum_{i=1}^N \mathbb{I}[\hat{y}_i = y_i]$, where $\hat{y}_i$ and y_i are the predicted and reference sentences.

Character n-gram metrics such as chrF provide a complementary similarity measure that is less brittle than exact sentence equality.

Token-level normalization precision, recall, and F1 can additionally distinguish successful corrections from missed or unnecessary modifications. This is especially useful for detecting **overcorrection**: changes applied to elements that were already correct.

Automatic metrics nevertheless have limitations. A benchmark with a single normalized reference can penalize another linguistically acceptable form. Conversely, a high string-similarity score can conceal an important semantic substitution.

The evaluation consequently operates at several levels:

- automatic downstream normalization metrics;
- error-category analysis;
- intrinsic synthetic-data statistics;
- and qualitative inspection of representative generated examples and model predictions.

Intrinsic synthetic evaluation can consider the number of edits, changed-token rates, duplicate rates, mapping coverage, code-switch frequency, rejected candidates, and correspondence with error distributions observed in authentic training data. In the present project, the linguistic annotation of gold-training token occurrences supplies the reference distributions for this comparison.

Manual annotation and qualitative review complement these statistics by considering meaning preservation, plausible error types, corruption severity, and visible generation artifacts. Because not every synthetic method received a complete blinded human rating, I use these observations as diagnostic evidence rather than as a second quantitative ranking.

Most importantly, comparisons between synthetic generators require controlled downstream conditions. If methods are evaluated with different architectures, training volumes, authentic fine-tuning sets, or optimization budgets, differences in final performance cannot reliably be attributed to synthetic-data quality.

2.2 Related Work

2.2.1 Wolof Linguistic Processing and Normalization

Wolof-specific work provides essential evidence that generic multilingual preprocessing is insufficient for all linguistic phenomena encountered in the language.

Dione [3] proposes finite-state tokenization for a deep Wolof Lexical Functional Grammar. The work incorporates language-specific rules for phenomena including clitics, multiword expressions, normalization, and token boundaries.

Its importance for the present project lies in demonstrating that apparently simple preprocessing decisions can depend on Wolof-specific linguistic structure. A generic whitespace tokenizer or language-independent spell-checker may therefore be insufficient.

The finite-state approach is interpretable and linguistically grounded. Its limitation with respect to the present project is its intended domain: it supports formal linguistic processing rather than modelling the open-ended variation encountered in contemporary social-media comments.

The closest direct precedent to the present work is Beqi by Mbaye and Diallo [4]. Beqi addresses the absence of a sufficiently large naturally occurring noisy-correct Wolof corpus by artificially introducing errors into standard text and training neural spelling-correction models.

The study is particularly important for two reasons. First, it provides direct evidence that synthetic augmentation is feasible for Wolof. Second, it formulates Wolof spelling correction as sequence transduction rather than as isolated dictionary lookup.

The present project extends this line of research toward a different target distribution. Its evaluation concerns authentic YouTube writing containing not only spelling variation but also spacing phenomena, lexicalized informal variants, and French–Wolof code-switching.

It also compares multiple synthetic-generation strategies under a shared downstream evaluation rather than studying a single synthetic-noise process.

These differences do not imply that a broader approach is inherently superior. Beqi benefits from a more constrained task definition, whereas social-media normalization introduces additional ambiguity and makes evaluation more difficult. Its results instead motivate the inclusion of a transparent synthetic spelling baseline before introducing target-domain or neural generation methods.

2.2.2 Rule-Based Synthetic Error Generation

Rule-based generation is one of the most direct approaches to synthetic normalization data.

Common transformations can include:

- deletion or substitution of characters;
- removal of diacritics;
- vowel or consonant variation;
- repeated characters;
- token merging or splitting;
- punctuation modification;
- and selected lexical substitutions.

The main strength of rule-based augmentation is transparency. Every generated change can be associated with an explicit operation and probability. Outputs can be reproduced using a fixed random seed, and unsafe categories can be protected.

This property is valuable for scientific comparison because it allows a generated example to be audited.

The principal weakness is that the rules themselves determine the synthetic distribution. Transformations designed from linguistic intuition may overrepresent easily described error patterns while underrepresenting lexicalized or community-specific forms.

Uniformly sampling edits can also produce combinations or error densities that differ substantially from authentic social-media writing.

Dekker and van der Goot [7] show that custom synthetic error generation can provide strong results for English lexical normalization and can approach models trained with manually annotated examples.

Their study also demonstrates why the origin of a synthetic rule set matters. A generator informed by annotated training data represents a different experimental condition from one constructed without supervision. Both may be valid, but they answer different questions and are not interchangeable.

2.2.3 Unsupervised Normalization and Variant Mining

A complementary research direction attempts to infer normalization relations from unannotated text.

Gouws et al. [8] mine lexical variants from noisy corpora without requiring manually aligned normalization pairs. Their work demonstrates that target-domain corpora can provide direct evidence about naturally occurring lexical variation.

Li and Liu [9] combine unsupervised candidate generation with discriminative reranking, incorporating both surface and semantic information.

Roy et al. [10] propose a language-independent unsupervised normalization algorithm and evaluate its impact not only intrinsically but also through downstream information retrieval and stance-detection tasks.

These approaches are particularly relevant to the original motivation of this project. The purpose of normalization is ultimately not merely to produce a visually standardized sentence, but to make informal language easier for downstream systems to process.

Collectively, the studies suggest that a large collection of unpaired YouTube comments can be used as more than an annotation source. Frequencies and contexts observed in the corpus can potentially reveal target-domain variants missing from hand-written transformation rules.

However, transferring these methods to Wolof is not straightforward. An out-of-vocabulary item is not equivalent to an error. It can instead be a French expression, a proper name, a valid Wolof word absent from the reference vocabulary, or a rare context-specific form.

An unsupervised Wolof variant-mining system therefore depends heavily on candidate confidence and precision. I implemented this idea, but conservative filtering accepted only a narrow mapping inventory and modified a small fraction of the formal corpus. Retaining it as a full benchmark condition would have compared a mostly unchanged dataset with generators designed to introduce much broader variation. I therefore document it as an informative negative result rather than as one of the five final generators.

2.2.4 Synthetic Code-Mixing

Synthetic code-mixing has been approached using bilingual dictionaries, embeddings, statistical alignment, linguistic constraints, machine translation, and neural generation.

The work of Das and Gambäck [2] illustrates a fundamental difficulty: before generating or normalizing mixed-language text, the system may itself need to determine the language identity of individual tokens. This decision is not always unambiguous.

Jawahar et al. [16] study synthetic English–Hinglish text generation using multi-lingual text-to-text Transformers and compare synthetic strategies involving bilingual information and code-mixing constraints.

Of particular relevance to this project is their use of synthetic data before fine-tuning on authentic examples. This supports evaluating synthetic generation not only

according to the visual properties of generated sentences but according to its effect on a downstream model.

However, conclusions drawn from English–Hinglish cannot simply be transferred to French–Wolof. The linguistic structures, available corpora, switching practices, and pretrained-model coverage differ. The transferable contribution is therefore primarily methodological rather than numerical.

More complex neural approaches have also been investigated.

Li and Vu [19] use cycle-consistent adversarial networks to generate code-switched text and evaluate the generated data through language modelling and speech-recognition tasks.

Chandu and Black [20] similarly investigate code-switching through the perspective of style variation and adversarial generation.

Such approaches can potentially model distributions that are more flexible than deterministic lexical replacement. Their disadvantages are increased training complexity, dependence on additional modelling assumptions, and reduced control over semantic preservation.

For the present low-resource setting, these characteristics make them useful related work but less appropriate as primary baselines before simpler and more auditable approaches have been established.

2.2.5 LLM-Generated Wolof Synthetic Data

Large language models provide another mechanism for transforming formal sentences into non-standard variants. Rather than applying a predefined sequence of local rules, the model receives a complete sentence and generates another complete sequence conditionally.

This approach can potentially capture contextual interactions that are difficult to encode manually.

The *Wolof-Non-Standard-Orthography* dataset released by Soynade Research contains standard/non-standard Wolof pairs generated for normalization-oriented use [21]. According to the resource description considered in this project, the synthetic transformations include phenomena such as phonetic spelling, missing diacritics, character substitution, token merging, and French code-switching.

This dataset is particularly relevant because its target language and normalization direction closely match the current project.

Compared with local deterministic rules, full-sequence LLM generation offers potentially richer and more contextual transformations. It also presents important limitations.

First, the exact transformation that produced an output is less transparent. Second, generation may accidentally alter content instead of merely altering its form. Third,

the apparent naturalness of an output does not guarantee that it provides a useful normalization example.

The published resource remains an external baseline because its original prompting, decoding parameters, seed, and detailed generation process are not controlled by this project. A source-corpus confound was nevertheless reduced empirically: all 3,430 unique formal Oolel targets were found in the larger project corpus, allowing the benchmark to select the same 3,330 formal targets for Oolel and every local generator. Differences in downstream performance can therefore be related more directly to the informal synthetic realizations, while the external origin and lower transformation transparency remain explicit limitations.

2.2.6 Unsupervised Text Style Transfer

Unsupervised text style transfer seeks to transform sentences between different domains or attributes without requiring direct sentence-level parallel pairs.

Jin et al. [6] survey neural text-style-transfer methods and emphasize three recurrent evaluation dimensions: preservation of content, strength of the desired transformation, and fluency.

Zhang et al. [22] formulate style transfer as unsupervised machine translation, using pseudo-parallel generation and iterative training mechanisms.

He et al. [23] provide a probabilistic formulation of unsupervised text style transfer in which latent parallel sequences connect non-parallel corpora.

These approaches are conceptually attractive for the present task because formal Wolof text and informal YouTube comments form two largely unpaired domains.

However, the assumptions required for full unsupervised style transfer are demanding in this setting. Defining a reliable informal-style objective is difficult when the target corpus contains heterogeneous phenomena and multiple languages. Cycle consistency also does not by itself guarantee semantic preservation.

For these reasons, the project investigated unsupervised learning in a narrower and more auditable lexical-mining prototype rather than adopting unrestricted sentence-level style transfer. The lexical prototype was subsequently excluded from the final benchmark because its high-confidence coverage was insufficient. Full neural style transfer remains relevant as a possible future extension, but it is not required to answer the final comparative research question.

2.2.7 Comparative Synthesis

Table 2.1 summarizes the main research strands and their relevance to this project.

Table 2.1 – Comparison of approaches relevant to informal Wolof normalization.

Approach	Resources	Strengths	Limitations for this project	Relevance
Wolof finite-state processing [3]	Linguistic rules and lexicons	Interpretable and language-specific treatment of tokenization and normalization	Designed for structured linguistic processing rather than open-ended social-media variation	Linguistic and preprocessing reference
Beqi synthetic correction [4]	Formal Wolof and synthetic error generation	Direct evidence that synthetic data can support Wolof spelling correction	More constrained spelling setting and limited modelling of real code-switching	Closest Wolof task precedent
Custom synthetic errors [7]	Formal text and transformation rules	Transparent, controllable, reproducible, and potentially effective	Synthetic distribution may differ from authentic variation	Generic rules baseline
Unsupervised variant mining [8], [9], [10]	Large unpaired noisy corpus	Exploits authentic target-domain evidence without paired labels	Ambiguous OOV items, names, French words, and unreliable rare contexts	Exploratory prototype; excluded for insufficient coverage
Word alignment [17], [18]	Parallel French–Wolof text	Provides bilingual lexical correspondences without requiring a full translation system	Lexical replacements may be contextually or grammatically inappropriate	Controlled code-switch baseline
Multilingual code-mixing [16]	Multilingual representations and synthetic/gold data	Supports contextual generation and synthetic-to-gold training curricula	Evidence originates from a different language pair	Training-strategy precedent
Oolel synthetic data [21]	Wolof-oriented LLM-generated dataset	Language-specific and potentially context-sensitive transformations	Lower interpretability, possible semantic drift, and unavailable original generation settings	External LLM baseline
Adversarial code-switch generation [19], [20]	Monolingual and code-switched corpora	Can model broader distributional properties than fixed lexical rules	More complex training and weaker direct control over meaning preservation	Related work and possible future extension
Unsupervised style transfer [22], [23]	Non-parallel domain corpora	Does not require direct sentence pairs	Difficult evaluation and demanding assumptions in heterogeneous multilingual text	Conceptual reference rather than core method
Multilingual pretrained seq2seq [14], [15]	Pretrained multilingual model and task-specific pairs	Contextual sequence correction and transfer learning	Wolof coverage, tokenizer suitability, and repeated-training cost remain limitations	Investigated but not retained as the final evaluator
TNT-inspired character edit Transformer [12], [24]	Matched synthetic pairs and a shared character vocabulary	Directly represents sparse orthographic operations and permits fast parallel reconstruction	Trained from scratch; limited lexical knowledge and source-anchored local edits	Final common downstream evaluator

The comparison reveals complementary advantages rather than a single complete solution. Wolof-specific work establishes the feasibility of language-aware normalization and synthetic correction. Generic normalization literature contributes rule-based and unsupervised techniques. Code-switching research provides bilingual and distributional

generation strategies. Pretrained multilingual models offer transfer learning, whereas a TNT-inspired character edit Transformer provides a computationally tractable controlled evaluator for repeated experiments.

However, no approach reviewed here simultaneously addresses all of the characteristics targeted in this project: Wolof-specific normalization, authentic social-media variation, French–Wolof code-switching, controlled synthetic-data comparison, strict separation between training and held-out information, and evaluation on the same real target-domain benchmark.

2.3 Positioning of This Work

2.3.1 Identified Research Gap

The literature review highlights three principal gaps relevant to this project.

First, existing Wolof correction research demonstrates that synthetic data can compensate for limited noisy–correct training pairs, but the studied corruption processes do not fully represent the heterogeneous variation encountered in authentic French–Wolof social-media text.

Second, unsupervised normalization research demonstrates how unpaired noisy corpora can reveal lexical variants, but the applicability of these methods to Wolof remains uncertain. The multilingual nature of the collected comments increases ambiguity because French words, names, valid rare forms, and actual normalization errors may all appear outside a formal Wolof vocabulary. The limited coverage of the project’s conservative prototype illustrates the gap between theoretical applicability and a sufficiently strong benchmark condition.

Third, research on synthetic code-mixing and style transfer often evaluates generation quality, language modelling, translation, or speech-recognition performance in better-resourced language combinations. It does not directly answer which synthetic-data construction strategy provides the strongest normalization signal for authentic informal Wolof under identical downstream conditions.

The contribution of this work is therefore not to claim the invention of synthetic noise, word alignment, lexical mining, or sequence-to-sequence normalization individually.

Instead, the project investigates their **controlled and domain-informed application to a low-resource Wolof normalization setting**.

2.3.2 Baseline Families

The experimental positioning distinguishes several generation families with different information requirements.

Generic rule-based generation. This represents the most controlled baseline. It uses explicit transformations applied to formal Wolof without deriving transformations from the collected YouTube comments or manually normalized benchmark.

Its purpose is to determine how much can be achieved through linguistic intuition and generic synthetic augmentation alone.

Alignment-based code-switching. This family introduces bilingual lexical evidence derived from French–Wolof parallel data. It tests whether explicit lexical correspondences improve synthetic representation of code-switching compared with purely monolingual noise rules.

External LLM-generated data. The published Oolel-related synthetic resource represents a different generation paradigm in which an LLM produces complete non-standard sequences rather than composing explicitly recorded local transformations.

It remains an external baseline because its original generation configuration is not reproduced locally. However, its 3,430 unique formal targets all occur in the larger project corpus. Matching 3,330 of these targets across every retained method controls the clean sentence content while preserving the distinction between external and project-controlled generation.

Exploratory unsupervised target-domain mining. This approach exploits authentic unpaired YouTube comments to discover lexical patterns without using manually normalized sentence pairs.

It was implemented to test whether access to the real target distribution could add useful information without direct supervision. The retained high-confidence mappings changed too few formal sentences for a balanced comparison, so the method is reported as an excluded prototype rather than a final baseline.

Gold-informed linguistic generation. The most supervised generator considered in the project learns transformations and distributions only from the training portion of the manually normalized benchmark.

Its purpose is to determine whether a limited amount of carefully isolated authentic supervision can make synthetic generation more representative and useful than generic rules, alignment-derived code-switching, distributional mixing, or external LLM-generated data.

2.3.3 Controlled Comparison Philosophy

I compared every synthetic-generation method through the same downstream task so that differences could be traced mainly to the training data.

Let $G_1, G_2, \dots, G_k$ represent the retained generation methods, where each generator produces a synthetic dataset $\mathcal{D}_{\text{synth}}^{(i)}$.

Across the controlled conditions, I kept the architecture, tokenizer, training budget,

gold data, and evaluation set constant. Only the synthetic initialization dataset changed.

This design separates two questions that are often conflated:

1. Does a generated sentence appear linguistically plausible?
2. Does training on the generated sentence make a normalization model more effective on authentic held-out text?

The second question constitutes the primary evidence in this project.

Intrinsic synthetic statistics, annotator-derived distributions, and qualitative inspection remain useful because they can explain why one dataset performs better or worse, but they do not replace downstream evaluation.

2.3.4 Supervision and Data Leakage

The distinction between legitimate supervised development and data leakage is particularly important when authentic annotations are used to inform synthetic generation.

Using patterns from a training set does not itself constitute leakage. Training data are intended to provide information from which model parameters, mappings, or transformation distributions can be learned.

Leakage occurs when information reserved for final evaluation influences those choices.

Conceptually, let the manually annotated benchmark be divided into mutually exclusive subsets $\mathcal{D}_{gold} = \mathcal{D}_{train} \cup \mathcal{D}_{dev} \cup \mathcal{D}_{test}$. Mappings learned from $\mathcal{D}_{train}$ can then legitimately inform a supervised synthetic generator.

I did not design transformations, rules, prompts, or thresholds from examples in $\mathcal{D}_{test}$.

A domain-aware project introduces an additional concern. Comments originating from the same YouTube video may share topics, vocabulary, expressions, or participants. Separating individual comments randomly may therefore underestimate the difficulty of generalization.

For this reason, the experimental design described later in Chapter 4 uses source-aware separation so that target evaluation more closely reflects transfer to unseen contexts.

2.3.5 Why Full Unsupervised Style Transfer Is Not the Core Method

The literature demonstrates that adversarial learning, back-translation, and cycle-consistency can support unsupervised text transformation [19], [22], [23].

Nevertheless, selecting a method for a PFE requires consideration not only of theoretical capability but also of experimental interpretability.

A full style-transfer system would introduce several additional variables:

- an informal-style model or discriminator;
- additional neural training objectives;
- adversarial or iterative optimization;
- stronger assumptions regarding the equivalence of the formal and informal domains;
- and additional evaluation requirements for semantic preservation.

Introducing these elements before establishing simpler baselines would make it more difficult to determine whether improvements originate from synthetic-data quality or from additional model complexity.

The project tested the main insight of unsupervised adaptation—learning from an unpaired authentic target-domain corpus—through traceable lexical mining. Because that prototype did not achieve sufficient coverage for the balanced benchmark, it is retained as exploratory evidence. Full neural style transfer remains relevant as a possible future extension.

2.3.6 Relationship to Semantic Retrieval

Although normalization is the principal experimental task, the original motivation of the project concerns robust semantic access to formal Wolof resources.

Suppose an informal query is represented by q_i , its normalized form by q_n , and a formal document by d . The underlying hypothesis is that normalization may yield $\text{sim}(q_n, d) > \text{sim}(q_i, d)$ for at least some queries affected by strong orthographic or lexical variation, where sim denotes the retrieval system’s similarity function.

Roy et al. [10] provide evidence in another noisy-text setting that normalization can improve downstream information retrieval.

This motivates retrieval as a possible secondary validation of the normalization approach. It is not used as the primary benchmark because a complete retrieval or Retrieval-Augmented Generation pipeline would introduce additional variables unrelated to the quality of the normalization model itself.

Evaluating normalization first therefore provides a more controlled answer to the central research question: whether synthetic formal–informal Wolof data can improve processing of authentic non-standard text.

2.3.7 Summary of the Positioning

The state of the art suggests that no single existing strategy completely addresses the requirements of the present setting.

Rule-based generation offers control but may poorly reproduce the real distribution. Word alignment offers bilingual correspondences but not contextual guarantees. Distributional code-mixing adds broader lexical evidence while remaining dependent on the quality of bilingual representations. LLM generation can produce richer outputs but reduces transparency and may alter meaning. Gold-informed generation can better reproduce observed error categories but is constrained by a very small authentic training sample. Unsupervised lexical mining provides authentic target-domain evidence but, in this project, did not achieve sufficient high-confidence coverage. Neither pretrained nor compact sequence-to-sequence models solve the lack of representative parallel data by themselves.

The resulting research question is therefore fundamentally comparative:

To what extent do different synthetic formal-to-informal Wolof generation strategies improve normalization of authentic informal and French–Wolof code-switched YouTube comments when the downstream architecture, clean source sentences, training budget, gold supervision, and evaluation data are held constant?

The following chapters operationalize this comparison by defining the data, generation pipelines, training protocol, and evaluation methodology under controlled experimental conditions.

Chapter 3

Project Context and Scope

3.1 Academic Setting and Responsibility

I conducted this PFE as an independent academic project under academic supervision at aivancity. Since there was no host company, the work involved no industrial department, proprietary company dataset, or industry supervisor.

I acted as project lead: I refined the research question, selected and read the relevant literature, defined the annotation policy, manually annotated the gold data, chose the generation methods and evaluator architectures included in the final benchmark, ran the experiments, inspected their outputs, and interpreted the results. Appendix D describes the generative-AI assistance used for implementation and writing. The experimental decisions and conclusions remain my responsibility.

3.2 Scope of the Project

3.2.1 Initial Objective and Evolution

The initial project was motivated by semantic retrieval. The intended application was to query a corpus containing new Wolof data even when user queries were written informally. Early investigation showed that the deeper obstacle was not limited to one retrieval algorithm: the available formal resources and real informal inputs followed different distributions. The same issue affected LLM prompts and machine-translation inputs.

I therefore shifted the project toward synthetic data generation. Instead of optimizing a retrieval pipeline without adequate training and evaluation pairs, I first addressed the data scarcity that affects several downstream systems. The benchmark transforms formal Wolof resources into synthetic informal variants and measures each method through normalization of real comments. Retrieval remains the motivating application, but adding a retrieval or Retrieval-Augmented Generation (RAG) case study would introduce representation, indexing, and answer-generation variables beyond the scope of the controlled comparison.

This change refined rather than abandoned the initial objective: the focus moved from one application to a reusable formal–informal bridge. The revised objective is:

Design and compare synthetic formal-to-informal Wolof generation strategies by measuring the downstream normalization signal they provide for authentic informal and French–Wolof code-switched YouTube comments under a fixed architecture and limited gold supervision.

3.2.2 Project Tasks and Personal Contributions

My work covered the following areas.

Literature and resource review. I collected and compared work on Wolof normalization, lexical normalization, code-switching, synthetic error generation, unsupervised mining, sequence-to-sequence models, and style transfer. I also inventoried the available Wolof dictionaries, translation data, and pretrained-model options.

Real-data collection and cleaning. I collected public YouTube comments with scripts that enumerate video links and download comments. I then used a cleaning notebook to remove or normalize URLs, usernames, timestamps, emojis, extra whitespace, and unusably short rows while preserving source-video provenance.

Candidate filtering. Because the cleaned corpus remained too large for manual review, I used a local Gemma model to select annotation candidates. The pipeline classifies bounded batches, validates response identifiers, retries failures, stores append-only checkpoints, and resumes interrupted work. It merges checkpoints by stable source index and retains video provenance for positive rows.

Formal-resource and vocabulary preparation. I consolidated French–Wolof translation and dictionary data and extracted a formal Wolof vocabulary for candidate lookup, spelling comparison, and annotation assistance. I then combined Wolof and French lexical resources, including POS information where available, into a seed lookup for linguistic annotation.

Synthetic generation prototypes and external baseline selection. I implemented or imported five synthetic conditions: Beqi-inspired deterministic transformations (M1), Eflomal alignment followed by the shared spelling transformations (M2), CMDR-style distributional code-mixing followed by the shared spelling transformations (M3), the published Oolel-generated non-standard Wolof data (M4), and the proposed gold-train-informed linguistic engine (M6). I also developed a Windows-native IBM Model 1 alignment experiment. The unsupervised YouTube lexical-mining prototype (M5) was completed but excluded from the final benchmark because only 114 mappings passed its conservative filters and 15,956 of 17,777 generated rows remained unchanged.

Gold annotation infrastructure. I built a Flask application for keep, discard, uncertainty, sentence correction, and token-level correspondence decisions. It preserves

append-only history and stable source identifiers. I froze annotation after accepting 201 comments from five videos. I then used a second Flask application to annotate all 1,315 token occurrences in the gold training partition with normalized forms, POS, language, entity, protection, and error information for M6 and the error analysis. Of these occurrences, 1,297 are reviewed and 18 remain explicitly uncertain.

Benchmark integrity. I implemented a video-disjoint split tool with minimum-data checks and manifest hashes. The locked split contains 142 training, 30 development, and 29 test pairs. I sampled every retained generator over the same 3,330 formal targets, and automated audits found no exact overlap with development or test inputs in the selected synthetic data. Workflow tests and UTF-8 audits protect the critical processing steps and Wolof characters.

Controlled downstream evaluation. I implemented a fixed character edit Transformer inspired by TNT as the common normalization probe. Every retained synthetic condition uses twenty epochs of synthetic pretraining followed by twenty uninterrupted fine-tuning epochs on the same Gold training set. Generated development Character Error Rate (CER) selects checkpoints at five-epoch intervals. Gold-only and M1–M4/M6 completed development selection, after which I froze the checkpoint manifests and evaluated every condition once on the locked 29-sentence test set. The identity condition requires no training and was scored directly.

3.2.3 Project Schedule

Table 3.1 presents the project in chronological functional phases. Exact dates were not retained for every early exploratory phase, so the table reports their verified order rather than inventing retrospective calendar precision; the dated gold-completion interval is preserved from the project record.

Table 3.1 – Functional schedule of the project.

Sequence	Phase	Principal outputs	Status
Phase 1	Problem framing and literature review	Initial retrieval problem, literature corpus, synthetic-data reformulation	Completed / continuing review
Phase 2	Collection and cleaning	Raw and cleaned YouTube corpora; scraping and cleaning workflow	Completed
Phase 3	Formal resources and first generators	Vocabulary, rules-only data, CMDR and Eflomal prototypes	Completed as prototypes
Phase 4	Filtering and reverse normalization	LLM checkpoints, filtered candidates, reverse pipeline	Completed as working pipeline / prototype
Phase 5	Annotation and reliability improvements	Flask tool, gold files, diversified video order, tests, split tooling	Completed
Phase 6 (8–24 August 2026)	Gold benchmark completion	Confirmed normalization policy, 201 kept pairs across 5 videos, locked splits	Completed
Phase 7	Synthetic generation and linguistic annotation	M1–M4 and M6 exports; matched source pool; M5 exploratory result	Implemented; reviews incomplete
Phase 8	Modeling and evaluation	Matched TNT-inspired edit-Transformer training, development selection, locked-test metrics	Completed for all final conditions
Phase 9	Report and handover	Final report, reproducibility package, presentation	In progress / remaining

3.2.4 Expected Deliverables

The project deliverables are:

- a documented raw and cleaned YouTube comment corpus;
- candidate-classification checkpoints and filtered data with provenance;
- a formal Wolof vocabulary and documented translation resources;
- five retained synthetic formal–informal datasets over a matched source pool;
- a validated external Oolel LLM-generated baseline with pinned revision and license;
- a documented protocol for a future external-system evaluation of Oolel-Corrector;
- lexical alignment artifacts and portability comparison;
- an annotation application and gold annotation files;
- a locked video-disjoint benchmark split;
- a documented exploratory unsupervised YouTube lexical-mining prototype and its exclusion analysis;
- the proposed standalone gold-informed linguistic generator;

- controlled downstream model checkpoints and predictions;
- metric tables, qualitative synthetic inspection, error analysis, and feasible ablations;
- code, tests, environment documentation, data lineage, and the final report.

3.2.5 Completed Contributions at the Time of Writing

At the current snapshot, the project has produced an end-to-end data-preparation and annotation path. The strongest completed contributions are the collection and cleaning of a large real-world corpus, robust resumable filtering, stable provenance across filtered data, formal vocabulary construction, five retained synthetic-data conditions, a reverse-normalization assistant, two Flask annotation workflows, a Windows-native IBM Model 1 alternative, and a hash-locked video-disjoint benchmark with automated integrity tests.

Annotation is complete at 201 kept gold pairs across five videos, the corrected video-disjoint split is locked, and all 1,315 gold-training token occurrences have linguistic labels. M6 has been independently exported as 17,777 pairs without using M1 or M5 outputs, although its manifest remains provisional because the planned 100-row qualitative review is incomplete. The TNT-inspired workflow completed all six trained conditions, development-only selection, frozen-test evaluation, uncertainty analysis, and prediction diagnostics. Chapter 5 reports that synthetic pretraining improves character-level reconstruction but remains weak on word-level and exact-edit criteria.

This distinction remains important: pipeline completion is an engineering contribution, while the failed-evaluator finding limits the scientific claim that can be made from the final comparison.

3.3 Data and Constraints

3.3.1 Data Inventory

Table 3.2 lists the principal resources available at the time of writing.

Table 3.2 – Principal data resources and artifacts.

Resource	Volume	Format	Origin and role	Status
Raw YouTube comments	396,559 records	JSON Lines	Public comments collected from selected videos; source corpus	Completed snapshot

Continued on next page

Table 3.2 – continued from previous page

Resource	Volume	Format	Origin and role	Status
Cleaned YouTube comments	242,174 rows from 193 videos	CSV	Cleaned unpaired target-domain corpus	Completed snapshot
LLM-classified comments	7,250 rows from 5 videos	JSON checkpoints and CSV	Candidate filtering for annotation	Expansion recommended
Positive filtered candidates	5,498 rows from 5 videos	CSV	Informal/code-switched annotation queue	Current
Annotation decisions	477 history rows	CSV	Append-only decision and correction history	Frozen; latest decisions reconciled
Kept gold pairs	201 pairs from 5 videos	CSV	Formal–informal benchmark pairs	Locked as 142 train, 30 development, and 29 test pairs
Token corrections	918 rows; 614 distinct informal tokens	CSV	Annotation memory and correction correspondences	Used only after train-only filtering
Gold-train linguistic annotations	1,315 occurrences	CSV	POS, language, entity, protection, and error information	1,297 reviewed; 18 uncertain
Main French–Wolof parallel corpus	17,777 sentence pairs	Parquet	Formal source data and alignment	Available

Continued on next page

Table 3.2 – continued from previous page

Resource	Volume	Format	Origin and role	Status
Additional bilingual entries	7,971 rows	CSV	Vocabulary expansion	Available
Dictionary translations	9,626 entries	JSON	Vocabulary and generation	Available
M1 Beqi-inspired rules	17,777 pairs; 17,017 changed	CSV	Deterministic orthographic synthetic baseline	Exported with manifest
M2 Eflomal plus rules	17,777 pairs; 17,180 changed	CSV	Alignment-based French substitution plus M1 rules	Exported with manifest
M3 CMDR plus rules	17,777 pairs; 17,102 changed	CSV	Distributional code-mixing plus M1 rules	Exported with manifest
Oolel non-standard orthography dataset	3,438 raw / 3,433 accepted pairs	Hugging Face / Parquet	External LLM-generated baseline with wo, English, and non-standard fields	Pinned CC BY-SA 4.0 snapshot; five duplicate pairs rejected
M5 YouTube lexical mining	17,777 pairs; 1,821 changed	CSV	Unsupervised target-domain prototype	Exploratory; excluded from benchmark
M6 linguistic engine	17,777 pairs; 14,215 changed	CSV	Learned only from gold-training linguistic annotations	Exported; qualitative review incomplete

Continued on next page

Table 3.2 – continued from previous page

Resource	Volume	Format	Origin and role	Status
Formal Wolof vocabulary	32,178 unique entries	Text	Candidate and spelling lookup	Available
Eflomal lexicon	1,989 Wolof entries	Pickle	Wolof-to-French alignment baseline	Available artifact
IBM Model 1 trial lexicon	1,672 entries; 2,775 links from 2,000 pairs	Pickle	Windows-native alignment alternative	Trial artifact

The corpus has two distinct roles. Formal French–Wolof and dictionary resources provide clean source sentences and lexical candidates. YouTube comments represent the target distribution and supply unpaired evidence, annotation candidates, and the final benchmark. These roles remain separated during evaluation. Although the retained local exports contain 17,777 rows and Oolel contains 3,433 accepted pairs, the controlled training sampler found 3,430 unique formal targets shared by all five retained methods and selected the same 3,330 targets for each condition using seed 2026. Thus, both training quantity and clean sentence content are controlled.

3.3.2 Data Quality Constraints

The YouTube corpus is large but weakly structured. Comments can be duplicated, context-dependent, extremely short, mostly French, composed of names or emojis, or unrelated to the target task. Video topic and community can create strong local vocabulary patterns.

The final annotated gold subset contains 201 kept pairs across five videos. Randomly splitting these comments would overestimate generalization because nearly identical expressions from the same context could appear in training and test data.

The following mitigations were implemented:

- preserve stable row identifiers, source-video URLs, and derived video identifiers throughout the pipeline;

- diversify classification and annotation in round-robin video order;
- remove or detect exact and near duplicates;
- require at least 200 kept pairs across at least 5 videos before splitting;
- create video-disjoint train, development, and test partitions;
- lock row identifiers and hashes in a split manifest;
- exclude test annotations and test-video comments from method development.

Annotation quality is another constraint. Informal Wolof may permit several acceptable formalizations, and the intended meaning may be uncertain without conversational context. French spans can be preserved or translated, but mixing both policies would make the target inconsistent.

The adopted task policy normalizes Wolof while preserving intentional French code-switches. The annotation interface records corrections and uncertainty rather than silently forcing questionable examples into the benchmark. A second-speaker review would strengthen reliability but was not available as a completed artifact at the reporting snapshot; this is therefore treated as a limitation rather than implied as completed validation.

The formal vocabulary is also incomplete by definition. A token absent from the vocabulary is not necessarily an error. It can be a valid inflection, French word, name, borrowed expression, or rare Wolof form. The exploratory unsupervised miner therefore treated vocabulary membership as candidate evidence rather than ground truth. Its low high-confidence coverage was one reason for excluding M5 from the final comparison.

3.3.3 Personal Data, Ethics, and Legal Considerations

The raw source consists of publicly visible user-generated comments. Public visibility does not by itself remove privacy and data-protection responsibilities. Usernames, profile information, URLs, names mentioned in text, and combinations of content and video provenance may permit direct or indirect identification.

The cleaning stage removes usernames and several obvious identifiers from the modeling text, but the raw collection and source URLs remain more sensitive.

The project applies purpose limitation, data minimization, integrity, and confidentiality principles consistent with Article 5 of the General Data Protection Regulation [25]. Collection is limited to the academic objective described in this report; model inputs exclude usernames and profile information, and the project does not attempt to identify or profile comment authors.

In practical terms:

- retain only fields necessary for the documented research purpose and reproducible video-disjoint splitting;
- do not attempt to identify or profile comment authors;
- keep raw provenance separate from modeling text and avoid publishing usernames or unnecessary source metadata;
- use anonymized or paraphrased examples in the report when a verbatim comment could be searchable;
- separate internal provenance from public model inputs;
- exclude raw identifiable comments from the public source-code repository and final report archive;
- retain only derived examples and manifests whose redistribution is compatible with their source terms;
- verify platform terms and dataset licenses before redistributing raw comments or external corpora.

This report is not a legal determination. The project is independent and uses no proprietary company data, so company dissemination authorisation and a company confidentiality notice are not applicable. Any institutional retention or redistribution requirement communicated by aivancity remains authoritative over the operational policy described here.

No company sponsored or commercially directed the research, and no commercial conflict of interest has been identified. External datasets, models, and software are presented as research resources rather than endorsements.

Ethical quality also concerns language representation. The classifier and generator may reproduce biases toward particular videos, speakers, topics, or French–Wolof practices. Labeling every non-standard form as an error can stigmatize legitimate community usage.

The report therefore describes the output as a task-specific normalized form, not as a judgment that one speaker’s language is invalid. In the qualitative analysis, I distinguish harmful synthetic artifacts from legitimate variation and document uncertainty instead of treating one normalization as linguistically absolute.

3.3.4 Technical and Infrastructure Constraints

The main workflow is Windows-native and uses Python notebooks and scripts. The annotation interface and IBM Model 1 alternative run on Windows.

Eflomal requires a compiled Linux/WSL environment to rebuild, although the existing `artifacts/lexicon.pkl` can be consumed on Windows. This creates a portability constraint: reproducibility depends either on documenting the WSL build or using the separately validated IBM Model 1 artifact.

LLM-assisted filtering uses a local Ollama model, Gemma 3 12B. Local execution avoids sending comments to a third-party hosted API, but it requires sufficient memory and compute and may produce nondeterministic or malformed classifications.

The implemented workflow mitigates this through bounded batches, identifier validation, retries, single-row fallback, checkpointing, and resumability. The LLM output is used only to prioritize manual annotation; it is not treated as a gold label.

Preliminary downstream experiments used mT5-small, full fine-tuning, and later LoRA adapters. On the available laptop, these alternatives were too slow for repeated controlled training and produced unstable development generations. A copy-aware compact architecture was also tested but showed excessive identity copying. These diagnostics informed the final choice without being included as synthetic-method results.

The retained evaluator is a 2,148,676-parameter character-level Transformer encoder trained from scratch. It uses three encoder layers, a hidden dimension of 192, four attention heads, a 120-symbol shared vocabulary, and parallel operation/segment prediction. The final comparison uses a shared initialization, maximum sequence length 256, a fixed seed, identical sampling and optimization budgets, and deterministic reconstruction. The experiment runs locally on an NVIDIA GeForce RTX 5060 Laptop GPU with approximately 8 GB of video memory, using Python 3.11, PyTorch, and Hugging Face Transformers. Exact package versions, runtime, batch sizes, precision settings, and checkpoint paths are recorded in run manifests and are reported in Chapter 4.

The benchmark controller provides visible progress bars and step logs, records live status and metrics, saves resumable checkpoints, preserves previous attempts, and can rerun one method without retraining unchanged conditions. Parallel edit reconstruction removes the earlier multi-hour autoregressive evaluation bottleneck; the complete suite nevertheless remains a material local-compute workload because five conditions each include 20 synthetic and 20 Gold epochs.

Other technical constraints include CSV encoding and Wolof-specific characters, notebook reproducibility, duplicated artifacts, and path differences across environments.

Central path configuration, `.editorconfig`, UTF-8 auditing, workflow tests, and data-lineage documentation mitigate these risks. The final benchmark is driven by reusable Python modules and a controller notebook, so the notebook exposes progress and configuration without containing the only copy of the experimental logic.

3.3.5 Budget and Time Constraints

The dominant cost is human time rather than paid data. Manual annotation requires Wolof competence and careful sentence-level judgment. Mapping and generated-sentence inspection are also necessary because automatic metrics cannot guarantee meaning preservation.

I used local LLM filtering to reduce the candidate pool and reused prior token corrections to accelerate annotation. Model training also ran on local hardware rather than paid cloud infrastructure. This avoided direct API and cloud-compute charges, but shifted the cost to elapsed time, electricity, hardware availability, and the practical limit of one complete training seed.

The initial blueprint included unsupervised machine translation, iterative back-translation, adversarial CycleGAN training, multilingual Transformers, and two-stage fine-tuning as one pipeline.

Implementing all components would exceed the available time and obscure attribution. The revised scope prioritizes a small number of auditable generators and a controlled downstream comparison. Cycle-consistent and fully unsupervised style-transfer models remain related work or future extensions.

3.3.6 Scientific Validity Constraints

The final benchmark remains small. The selected minimum of 200 kept pairs enables an operational split, but it does not provide strong statistical power, especially after division by error category.

The final 201 pairs produce an allocation of 142 training, 30 development, and 29 test examples. The split is video-disjoint: three videos occur in training, one in development, and one in test. This reduces direct within-video leakage but also confounds topic, speaker community, and video identity with the partitions. I therefore interpret small differences cautiously and report paired bootstrap uncertainty for the frozen test predictions.

Synthetic methods also differ in edit difficulty and vocabulary. The final design controls the clean source pool, number of examples, model initialization, architecture, seed, gold fine-tuning data, epoch budget, and development-only checkpoint rule; the synthetic realizations are the main varying factor. Because the available compute budget allowed only seed 2026, random-initialization variance remains unknown, and I avoid conclusions that imply more precision than the experiment supports.

Standalone M6 learns from the 142-example gold training partition and its token-level linguistic annotations, but it consumes neither M1 rules nor M5 mappings. Development and test annotations are excluded from generator construction. M5 is reported only

as an exploratory negative result and is not one of the trained final conditions. M6’s use of gold training evidence is therefore explicit supervised method design, not hidden access to held-out answers.

The locked test set remained closed until every retained development checkpoint was frozen. Gold-only and M1–M4/M6 now have immutable selection records, and the one-time test evaluation is complete. The reported test results were not used to adjust a generator, training schedule, architecture, or checkpoint.

The M2, M3, M4, and M6 qualitative review sheets were incomplete when model training began. The runner recorded an explicit provisional-quality override instead of marking the datasets as manually validated. This allowed the long benchmark to proceed, but it limits claims about naturalness and meaning preservation; I retain that limitation throughout the interpretation.

Finally, normalization accuracy is evidence for the formal–informal bridge, but it does not automatically prove gains for every downstream task.

Retrieval, LLM querying, and translation have their own objectives and failure modes. Since I did not run focused experiments for those tasks, I discuss them only as future work rather than as demonstrated outcomes.

Chapter 4

Methodology

This chapter describes how synthetic formal–informal Wolof data are constructed and compared. The evaluated downstream task is sentence normalization: given an authentic informal or French–Wolof code-switched comment x , a model produces a task-specific normalized form $\hat{y}$. Synthetic generation operates in the opposite direction. A method g_m transforms a formal sentence y into an artificial informal source $\tilde{x}_m = g_m(y)$, producing the training pair $(\tilde{x}_m, y)$.

The model is an evaluation instrument rather than the primary contribution. If two conditions use the same formal targets, architecture, initialization, optimization budget, gold fine-tuning data, and checkpoint rule, their difference after fine-tuning provides evidence about the training signal supplied by their synthetic sources. This design cannot prove that the best condition is also optimal for retrieval or machine translation, but it gives a controlled answer to the normalization question posed in Chapter 1.

Table 4.1 gives the final implementation state. It separates completed computational evidence from optional human-review or external-comparison work so that the reported benchmark is not confused with exploratory artifacts.

Table 4.1 – Implementation state of the experimental methodology.

Component	State	Evidence or remaining work
YouTube collection and cleaning	Implemented	396,559 raw comments and 242,174 cleaned rows from 193 videos.
Local LLM filtering	Implemented subset	7,250 classified rows from five videos; 5,498 retained candidates.
Gold annotation	Frozen and corrected	201 kept pairs; 142/30/29 video-disjoint train/development/test split.
Linguistic annotation	Completed with uncertainty	1,315 gold-training occurrences: 1,297 reviewed and 18 uncertain.
M1–M4 and M6	Exported and trained	Manifests, provenance, matched samples, checkpoints, and predictions exist; optional 100-row review sheets remain incomplete.
M5 lexical mining	Exploratory, excluded	Only 114 mappings accepted and 1,821 of 17,777 outputs changed.
Compact benchmark	Completed	Six checkpoints selected on development data and evaluated once on the locked 29-sentence test set.
Report analysis	Completed	Intrinsic corpus profiles, prediction diagnostics, and 5,000-sample paired CER bootstrap analysis exported reproducibly.
Oolel-Corrector	Not evaluated	This optional external-system comparison remains outside the controlled generator benchmark.

4.1 Overall Experimental Architecture

The workflow contains two data streams. The first consists of formal French–Wolof parallel sentences and lexical resources. These provide clean Wolof targets and bilingual evidence from which synthetic informal inputs can be generated. The second consists of authentic but initially unpaired YouTube comments. They define the target domain, supply annotation candidates, and provide the real development and test inputs. Only the gold training partition can inform M6; the development and test targets do not enter any generator.

Figure 4.1 summarizes the resulting architecture. The five synthetic branches remain independent until they enter the common training protocol. M0 and gold-only act as controls rather than synthetic generators.

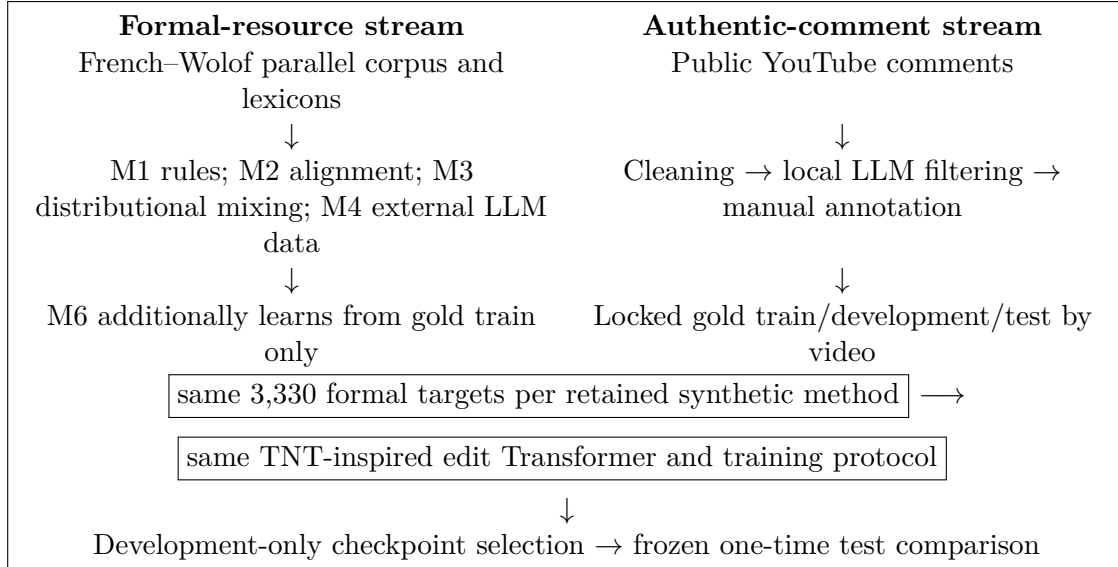


Figure 4.1 – End-to-end controlled architecture. M5 is retained as an exploratory artifact but does not enter the final training branches. Source: project implementation.

The principal comparison contains seven conditions:

1. **M0 identity**: return the input unchanged, without training;
2. **gold-only**: train the evaluator only on the 142 authentic training pairs;
3. **M1+gold**: deterministic rule pretraining followed by gold fine-tuning;
4. **M2+gold**: Eflomal-based code-mixing and rule pretraining followed by gold fine-tuning;
5. **M3+gold**: CMDR-style distributional code-mixing and rule pretraining followed by gold fine-tuning;
6. **M4+gold**: external Oolel synthetic pretraining followed by gold fine-tuning; and
7. **M6+gold**: gold-train-informed linguistic generation followed by gold fine-tuning.

Synthetic-only checkpoints are necessary intermediate states, but they are not principal reported conditions. The research question asks whether synthetic pretraining improves the final normalizer after identical limited gold supervision. I did not execute Oolel-Corrector in the final benchmark because its architecture and original training history differ from those of the controlled evaluator; any later evaluation belongs in a separate external comparison.

4.2 Data Construction

4.2.1 Formal Resources

The main formal resource contains 17,777 French–Wolof sentence pairs in Parquet format. The Wolof side supplies the normalized targets used by the local generators, while the French side supports bilingual alignment and distributional candidate selection. Additional resources contain 7,971 bilingual entries and 9,626 dictionary translations. A vocabulary-building script extracts Unicode word forms from the formal Wolof fields, case-folds and deduplicates them, rejects digit-only items, and writes a 32,178-entry UTF-8 vocabulary.

The vocabulary is used as candidate evidence, not as a complete definition of correct Wolof. A missing form can be an unseen inflection, a name, a French borrowing, or a legitimate rare word. This qualification is especially important for annotation assistance and M5 mining.

All registered paths are centralized in the project configuration module. CSV, JSON, JSON Lines, Pickle, and Parquet artifacts are read according to their type. UTF-8 and Unicode normalization are used throughout to preserve Wolof characters such as $\tilde{n}$, η , and $\tilde{e}$. Dataset and artifact manifests store SHA-256 hashes so a later run can detect silent input changes.

4.2.2 YouTube Collection

The authentic corpus was collected from public comments associated with selected YouTube playlists. The platform’s official data interface and resource model are documented by Google [26]; the implemented ingestion workflow first enumerates video links with `yt-dlp` in flat-playlist mode and removes duplicate links. It then uses the `youtube-comment-downloader` package to retrieve the available comments for each video. Each raw record retains the text and source-video URL. A processed-video ledger makes collection resumable and avoids re-downloading videos completed by an earlier execution.

The resulting research snapshot contains 396,559 raw records. It is not a continuously synchronized crawl: deletions, new comments, moderation, and changes in platform behavior mean a later extraction may differ. The source URL is retained internally to support video-disjoint splitting, but author profiling is outside the purpose of the project and identifiable fields are not needed by the normalizer.

4.2.3 Cleaning and Heuristic Language Filtering

The cleaning notebook removes URLs, username mentions, timestamp-like strings, broad emoji ranges, repeated exclamation and question marks, and redundant whitespace. It lowercases modeling text, removes empty rows, and rejects comments containing two tokens or fewer. The output retains the original text, cleaned text, source-video URL, and an estimated French ratio.

For language filtering, alphabetic material is extracted from each token and passed to `langdetect`. A row is marked unusable when more than half of its tokens contain no usable alphabetic material. Rows whose estimated French-token ratio is at least 0.5 are also removed. These operations reduce the snapshot to 242,174 cleaned comments from 193 videos.

The thresholds are heuristic. Word-level language detection is fragile for short strings, spelling variants, borrowings, and words shared across French and Wolof. The filter may therefore discard relevant code-switched comments or retain French material. It is a corpus-reduction step rather than a gold language label, and the final benchmark relies on manual normalization rather than this ratio.

4.2.4 Local LLM Candidate Filtering

Manual reading of all cleaned comments was infeasible. A local Gemma 3 12B model served as a candidate selector through Ollama. Its prompt requests a strict JSON mapping from stable integer identifiers to Boolean decisions. A positive decision marks the row as plausibly informal Wolof or Wolof–French, so the interface presents it for annotation; it does not automatically become gold data.

The classifier uses temperature zero, batches of 50, a 300-second request timeout, up to three retries, and a two-second delay between retries. Returned identifiers and Boolean types are validated. A failed batch is retried row by row, append-only checkpoints are written regularly, and resumed executions skip previously classified identifiers. Pending comments are interleaved across videos rather than exhausting one video at a time. Checkpoint labels are joined back to the source corpus by stable index with one-to-one validation.

The final annotation campaign used 7,250 classified comments from five videos, of which 5,498 entered the positive candidate queue. Because the local model only prioritizes candidates, its unknown precision and recall affect annotation efficiency and corpus coverage but do not directly create benchmark targets.

4.2.5 Sentence-Level Gold Annotation

The first Flask application presents candidate comments in deterministic round-robin video order. Four decisions are available: `keep`, `discard_false_positive`, `discard_uninteresting`, and `skip_uncertain`. For a kept row, the annotator validates or rewrites the complete formal sentence and records token correspondences. The stored fields include the stable source index, original comment, automatic suggestion, manual normalized target, decision status, source-video URL, and serialized token corrections.

The adopted policy normalizes non-standard Wolof while preserving intentional French spans. It therefore defines code-switched normalization rather than translation into monolingual Wolof. Names, entities, numbers, and meaningful French material remain unchanged when their only issue is absence from the formal Wolof vocabulary.

Annotation is append-only. Revisiting a comment creates a later decision rather than overwriting history, and downstream preparation keeps the last decision for each source index. This explains why the raw annotation file contains 477 history rows while the final benchmark contains 201 unique kept comments. The workflow recorded 918 correction rows involving 614 distinct informal forms. Earlier Streamlit records whose automatic suggestion had been copied into the manual sentence were reconstructed from their reviewed token corrections; later sentence corrections were propagated to the locked files through audited, versioned revisions.

4.2.6 Gold-Training Linguistic Annotation

A second Flask application enriches only the 142 gold-training sentences. It represents each reviewed occurrence using the informal unit, normalized unit, lemma, universal part-of-speech category, source and target language, named-entity category, protection decision, one or more error labels, an automatically derived character-edit script, a lexical-reuse flag, provenance, and review status.

The error-label inventory includes identity, capitalization, diacritic change, spacing split or merge, abbreviation, repetition, deletion, insertion, substitution, phonetic spelling, French spelling, lexical replacement, punctuation, multiple, other, and uncertain. Labels are multi-valued because one surface form can involve several phenomena. Code proposes conservative labels from character differences and spacing, but the annotator remains authoritative for semantic categories.

Lexical lookup reduces repetitive manual work. A reviewed Wolof dictionary seed can prefill Wolof language, candidate POS, non-entity status, and non-protection. French lexical entries prefill French language and protection. Senegalese surnames and manually selected person entities are assigned proper-noun status and protected. Ambiguous forms with several POS values or several language interpretations remain available for occurrence-level review rather than being locked to the first lookup row. Identical

confirmed properties can be reused for later occurrences while retaining new occurrence identifiers and timestamps.

All 1,315 gold-training occurrences have a saved linguistic record: 1,297 are marked reviewed and 18 remain uncertain. M6 filters this table again before learning, so uncertain, French, protected, or otherwise ineligible occurrences do not silently enter its transformation inventory.

4.2.7 Video-Disjoint Gold Split and Corrections

The split utility first retains the latest non-empty **keep** decision per source index. It refuses to create a final benchmark below 200 kept pairs or five represented videos. Complete video groups are assigned with seed 2026 using a greedy approximation to 70/15/15 proportions. This produces 142 training examples from three videos, 30 development examples from one video, and 29 test examples from one video.

The manifest records the source hash, split hashes, ratios, timestamp, seed, and video membership. The utility refuses to overwrite existing locked files. When ten later annotation errors were corrected, the prior files were archived and a revision manifest linked the corrected split to the preceding manifest. Consistency scripts verified that reconstructed token corrections and current formal sentences agreed and that linguistic occurrence identifiers still corresponded to the corrected gold-training rows.

No exact source or pair duplicates occur across gold partitions, and the selected synthetic sets contain no exact development or test input/pair overlap. Keeping videos disjoint reduces direct contextual leakage, but the small number of videos means topic and video identity are also confounded with the split. The test targets remain inaccessible to generation, architecture changes, training, and checkpoint selection.

4.3 Synthetic Generation Conditions

Every exported method uses the common normalization schema **source_id**, **method**, **informal_wolof**, and **formal_wolof**. Generation begins from **formal_wolof**; training reverses the pair so the model receives **informal_wolof** and predicts the unchanged formal target. Richer columns record source rows, seeds, changed flags, edit counts, error categories, applied rules, mappings, scores, and other method-specific provenance.

4.3.1 M0: Identity Reference

M0 sets the synthetic source equal to the formal sentence and was exported as a pipeline/schema check. It is not a plausible error generator and is not trained in the final comparison. Instead, the identity baseline is evaluated directly on real comments

by setting $\hat{y} = x$. This measures how difficult the benchmark is before normalization and detects whether a learned system performs worse than doing nothing.

4.3.2 M1: Beqi-Inspired Deterministic Rules

M1 is a controlled rules-only baseline inspired by Wolof transformations discussed in Beqi [4]; it is not presented as an exact reproduction of the Beqi training pipeline. An ordered inventory of 27 regular-expression operations lowercases text, modifies hyphenation, reduces selected repeated vowels, and maps forms such as $\tilde{n} \rightarrow gn$, $\eta \rightarrow ng$, $\tilde{e} \rightarrow eu$, $u \rightarrow ou$, and $x/q \rightarrow kh$. Other contextual rules affect *c*, *j*, *th*, final *e*, final *n*, and doubled *g*. Temporary sentinels prevent the output of one rule from being unintentionally transformed again by a later rule.

The export masks multilingual named-entity spans using Davlan’s multilingual XLM-RoBERTa large NER checkpoint, as well as uppercase acronyms and numeric expressions. The exact model identifier, `Davlan/xlm-roberta-large-ner-hrl`, is stored in the generation manifest. Protected substrings are restored after transformation. Each row records which rules fired and how often. The complete export contains 17,777 pairs, of which 17,017 differ from the formal source and 760 are unchanged. Because every matching rule fires deterministically, M1 measures the utility of transparent broad coverage, not a human-calibrated error distribution.

4.3.3 M2: Eflomal Alignment Plus Shared Rules

M2 adds bilingual substitutions before applying the exact M1 spelling component. Eflomal [18] is trained on the 17,777 French–Wolof pairs under Linux/WSL in batches of 5,000. Forward alignment links are aggregated into conditional lexical probabilities. I retained only links that occurred at least three times and accounted for at least 0.10 of the aligned mass for their Wolof token. The saved lexicon contains 1,989 Wolof entries and can be consumed on Windows without rebuilding the aligner.

For each Wolof sentence, I restricted replacement candidates to words found in its paired French sentence. Candidates are ordered by Eflomal probability with deterministic position and lexical tie-breaking. The generator attempts to replace approximately 20% of eligible Wolof word positions, with at least one replacement when a candidate exists, and prevents reuse of a source position or French target within the row. Inserted French strings are masked before the M1 transformations so their spelling is not corrupted. The final export did not run the optional named-entity model, although acronym and number masking remained available through the shared structural protection.

M2 contains 17,777 pairs: 17,180 changed rows, 13,661 rows with at least one alignment edit, 49,063 alignment substitutions, and 287,274 recorded spelling-rule

operations. Alignment probability does not guarantee contextual grammaticality, so these activity counts are not treated as quality scores.

4.3.4 M3: Distributional Code-Mixing Plus Rules

M3 explores whether shared distributional representations produce a different bilingual signal. It is described as CMDR-style because it adapts the general distributional code-mixing idea rather than reproducing a published system exactly. Wolof and French n-grams up to length three from each parallel row train a shared Word2Vec space with 150 dimensions, context window 10, minimum count 2, 20 epochs, one worker, and fixed training seeds. Generation considers candidates up to bigram length. A stable CRC32 hash removes Python-process hash variation. A separate co-occurrence table provides bilingual alignment evidence.

For a Wolof candidate w and French candidate f , Equation (4.1) combines embedding similarity, bilingual evidence, relative position, and an n-gram-size penalty:

$$S_{M3}(w, f) = \text{sim}(w, f) [1 + a(w, f)] \exp\left(-\frac{(p_w - p_f)^2}{2\sigma^2}\right) q_n, \quad (4.1)$$

where $\text{sim}(w, f)$ is cosine similarity, $a(w, f)$ is the alignment/co-occurrence score, p_w and p_f are normalized positions, $\sigma = 0.35$, and q_n penalizes longer n-grams. Candidates require embedding similarity of at least 0.70 and alignment evidence of at least 0.01. French candidates are restricted to the paired French sentence. At most two non-overlapping substitutions are selected; target phrases are not reused. The inserted French spans are protected while M1 spelling rules process the remaining text.

The deterministic M3 export contains 17,777 pairs, including 17,102 changed rows. There are 6,176 rows with a distributional substitution and 9,548 such substitutions in total; the remaining changes come from the shared spelling stage. Saved embeddings, alignment tables, component scores, hashes, and row-level edits make the method auditable. Its central risk is semantic substitutability: distributionally related words are not necessarily translations that can replace one another in context.

4.3.5 M4: External Oolel LLM-Generated Data

M4 imports `soynade-research/Wolof-Non-Standard-Orthography` at pinned revision `ce17c80cff6626a91dc4e38b43de12e005bc240a` [21]. The resource is distributed under CC BY-SA 4.0 and provides standard Wolof, LLM-generated non-standard Wolof, and English information. Import code strips the documented `<NON_STANDARD>` wrapper and validates empty values, malformed wrappers, unchanged normalized pairs, and duplicates.

Of 3,438 raw rows, 3,433 pairs are accepted; five repeated pairs are rejected. Every

accepted non-standard source differs from its formal target. Although the original dataset is externally generated, its formal side is not a distinct-content confound in the final matched experiment: the 3,430 unique formal Oolel targets all occur in the larger project corpus. This overlap permits the same formal targets to be used for every method. M4 nevertheless remains an external baseline because its original prompt, seed, decoding parameters, and row-level transformation decisions are not available to this project.

4.3.6 M5: Excluded Unsupervised Lexical-Mining Prototype

M5 investigated whether unpaired YouTube comments could reveal informal variants without manual formal–informal pairs. It collects frequent out-of-vocabulary forms, excludes known French and protected terms, retrieves formal candidates through canonical Wolof rewrites and character n-gram overlap, and ranks each pair using the score in Equation (4.2), which combines character similarity, rule compatibility, contextual overlap, and corpus frequency:

$$S_{M5}(u, s) = 0.4C(u, s) + 0.3R(u, s) + 0.2X(u, s) + 0.1F(u), \quad (4.2)$$

where u is an observed comment form, s a formal candidate, and C , R , X , and F are the character-similarity, rule-compatibility, contextual-overlap, and frequency components. The final leakage-safe run excludes all comments belonging to the locked development and test videos. Candidates require frequency at least five, character similarity at least 0.50, total score at least 0.68, and a 0.03 margin over the next candidate. At most two mappings are applied per formal sentence.

The eligible corpus contains 239,795 comments after held-out-video exclusion. Of 19,828 proposed candidates, only 114 pass the conservative filters. The resulting 17,777-row export changes 1,821 sentences and leaves 15,956 unchanged. This provides useful negative evidence about the precision–coverage trade-off, but the condition is too close to identity for a balanced comparison with the retained generators. It is therefore documented but not trained in the final benchmark.

4.3.7 M6: Gold-Train-Informed Linguistic Engine

M6 is the proposed method. It learns only from the locked gold training partition and its linguistic occurrence annotations. The learner first keeps the latest record per occurrence and requires **reviewed** status, Wolof source and target languages, **protected=no**, and non-empty formal and informal forms. This yields 865 eligible occurrences, of which 588 are changed.

The method learns three complementary components. First, a POS-conditioned

profile estimates change rates and error-type frequencies. Second, reviewed lexical mappings—including spacing-merge phrases—are retained when their count is at least one and their dominant informal realization accounts for at least 0.50 of observations for that formal unit. Third, character-edit templates are extracted from insert, delete, and replace operations. A template records POS, error type, source and destination strings, left and right character context, and prefix/internal/suffix position. Only diacritic, deletion, insertion, substitution, and repetition templates are considered generalizable; a template needs at least two examples and dominance of at least 0.60.

The final learned profile contains 171 lexical mappings and 105 contextual transformation templates. Spacing merges are special: their normalized units may be multiword phrases and therefore have unknown POS, but they remain reusable exact phrase mappings. French terms, named entities, ambiguous lookup entries, acronyms, numbers, and suspicious sentence-medial title-case tokens are protected before generation.

For each formal source, a stable seed derived from the source identifier samples an edit budget from the empirical number of changed occurrences per gold-training sentence. The selected configuration removes an artificial maximum cap while retaining that empirical distribution. Candidate edits are weighted by confidence, evidence, and the POS error profile; overlapping edits are forbidden. Every accepted operation records its position, source, category, evidence count, confidence, and linguistic metadata.

M6 deliberately imports neither M1 rules nor M5 mappings. Its 17,777-row standalone export contains 14,215 changed and 3,562 unchanged sentences, with 14,809 lexical-mapping edits and 22,119 generalized transformation edits. The manifest verifies that only 142 gold-training source identifiers were available to learning and that no development or test annotations were consumed. Its 100-row qualitative review was incomplete when training was authorized, so the export remains explicitly provisional.

4.3.8 Common-Target Matching and Quality-Gate Policy

Raw export size is not the downstream training budget. For each retained method, the loader normalizes whitespace and Unicode, rejects empty text and duplicate source identifiers, removes duplicate pairs, and retains one deterministic informal realization per canonical formal target. Canonical targets use NFC normalization, whitespace collapse, and case-folding.

The intersection across M1, M2, M3, M4, and M6 contains 3,430 unique formal targets. A deterministic sample of 3,330 targets is drawn with seed 2026. Each method then receives exactly one source for each selected target, sorted identically, and the M4 formal surface is used as the shared target representation. Consequently, data quantity and clean sentence content are held constant while the informal realization varies. The audit found zero exact input and pair overlap with both development and test. Between

six and ten sources per method and six common targets exceed 255 characters; these are recorded because the 256-position model may truncate them.

M2, M3, M4, and M6 have seeded 100-row review sheets, but these were not completed before the long-running benchmark began. The runner was therefore invoked with an explicit provisional-quality override. This bypass affects only readiness validation: it does not change the generated pairs, matched sample, or training configuration. The override is stored in run manifests and limits claims about naturalness and semantic preservation.

4.4 Downstream Normalization Model

4.4.1 Evaluator Selection

Initial experiments considered mT5-small, motivated by multilingual pretraining and the synthetic-to-gold curriculum used in related code-mixing work. Full fine-tuning of approximately 300 million parameters was impractically slow on the available laptop. Rank-8 LoRA greatly reduced the number of trainable parameters, but repeated generation and data processing still made the complete multi-method suite expensive, and early development predictions were unstable. mBART and NLLB were considered conceptually; NLLB is primarily a translation model, while changing pretrained architectures between synthetic conditions would weaken attribution.

A sentence-level adaptation of the Transformer’s earlier word-correction project was therefore tested. Sanity overfitting showed that a compact character model could learn the transduction, although long sequences remained harder. A conventional autoregressive version nevertheless accumulated decoding errors and produced repetitive outputs. An experimental pointer-like copy mechanism reduced some errors but increasingly favored returning the input unchanged. These pilots showed that the evaluator needed an explicit representation of sparse edits without a sequential decoder whose early mistake could shift every subsequent character.

The final benchmark consequently adopts a non-autoregressive edit-prediction Transformer inspired by TNT [24]. TNT jointly predicts manipulation operations and recovery labels so that normalization is part of representation learning. The present implementation is an adaptation rather than an exact reproduction: it operates over Wolof characters, learns directly from paired informal–formal sentences, and reconstructs normalized output rather than using the objective primarily for content-moderation pretraining. The architecture remains deliberately small and identically configured for every condition.

4.4.2 TNT-Inspired Character Edit Transformer

The retained evaluator is a bidirectional Transformer encoder trained from scratch. A shared character vocabulary is built only from gold training and the matched synthetic training frames; development and test text do not define the vocabulary. Each Unicode character is isolated as one token, and the vocabulary contains 120 symbols including padding, beginning, end, and unknown markers.

Character and learned positional embeddings are summed before three encoder layers with hidden size 192, four attention heads, feed-forward size 768, GELU activations, and dropout 0.1. Two prediction heads operate on each encoded source position. The operation head predicts one of four actions: KEEP, DELETE, SUBSTITUTE, or EXPAND. The segment head predicts up to 32 output characters anchored to the same source position. KEEP copies the input character, DELETE emits nothing, SUBSTITUTE emits one replacement character, and EXPAND emits a multi-character segment. The terminal input position can anchor characters inserted after the final ordinary source character. This permits local spelling repair and spacing changes while preserving deterministic source order. Explicit transposition, one of the operations in the original TNT paper, is not modelled as a separate class; it can only be approximated through a local replacement segment.

Training labels are constructed through a deterministic Levenshtein alignment between every informal source and formal target. Let o_i be the operation attached to source position i , s_{ij} the j th character in its target segment, and p_θ and q_θ the operation and character distributions. The joint objective is given in Equation (4.3).

$$\mathcal{L}(\theta) = - \sum_i \log p_\theta(o_i | x) - \sum_{i: o_i \in \{\text{substitute}, \text{expand}\}} \sum_{j=1}^{32} \log q_\theta(s_{ij} | x, i), \quad (4.3)$$

with equal weights for the two components and label smoothing 0.1. Segment padding is also supervised so generation knows where each local segment ends. At inference, all operations and segments are predicted in one encoder pass and assembled in source order. There is no teacher forcing, beam search, autoregressive feedback loop, or learned copy gate. The model contains 2,148,676 trainable parameters, caps source and reconstructed target sequences at 256 positions, and generated all 30 development sentences in approximately 0.27 seconds in an isolated timing check on the recorded GPU.

4.4.3 Two-Stage Training Protocol

One immutable vocabulary and random initialization is created with seed 2026 and reused by every condition. The gold-only model begins from this initialization and trains for 20 epochs on the 142 gold-training pairs. Each synthetic method begins from

the same initialization, trains for 20 epochs on all 3,330 matched pairs, and then starts a new optimizer stage from the selected synthetic model weights for 20 uninterrupted epochs on the same 142 gold pairs. No synthetic development partition is removed from the matched training corpus. Instead, the same 30-sentence gold development split is used at predefined intervals to monitor both stages. Results are reported only after the gold stage.

Algorithm 1 formalizes the procedure. All scheduled epochs run; there is no early stopping. Saving the lowest-CER epoch is checkpoint selection, not an early termination rule.

Algorithm 1: Controlled synthetic-pretraining and gold-fine-tuning protocol.

Input: matched synthetic sets $\{\mathcal{S}_m\}$, gold train $\mathcal{G}_{tr}$, gold development $\mathcal{G}_{dev}$, common initialization θ_0

Output: one frozen development-selected checkpoint per trained condition

- 1 Train θ_{gold} from θ_0 on $\mathcal{G}_{tr}$ for 20 epochs;
 - 2 Generate $\mathcal{G}_{dev}$ after epochs 5, 10, 15, and 20 and retain the minimum-CER gold checkpoint;
 - 3 **foreach** *retained method* $m \in \{M1, M2, M3, M4, M6\}$ **do**
 - 4 Train θ_m^s from θ_0 on $\mathcal{S}_m$ for 20 epochs;
 - 5 Retain its minimum-CER development checkpoint;
 - 6 Reset the optimizer and train from θ_m^s on $\mathcal{G}_{tr}$ for 20 epochs;
 - 7 Generate $\mathcal{G}_{dev}$ after epochs 5, 10, 15, and 20 and retain the minimum-CER final checkpoint;
 - 8 **end**
 - 9 Freeze all selected checkpoint paths before test evaluation;
-

Generated development CER is used because operation/segment loss does not directly express the quality of the reconstructed sentence. The 30-row development set fits in one evaluation batch, but evaluating it during both synthetic pretraining and gold fine-tuning entails repeated reuse of a small sample. Restricting generated selection to four scheduled epochs per stage reduces but does not remove development-overfitting risk. In particular, the Gold-dev split influences which synthetic checkpoint proceeds to fine-tuning even though no Gold-dev row is used for gradient updates. The test set remains sealed to provide the final independent comparison.

Table 4.2 gives the frozen configuration.

Table 4.2 – Fixed TNT-inspired edit-Transformer benchmark configuration.

Parameter	Value
Architecture	Character encoder-only edit Transformer; 2,148,676 parameters
Vocabulary / maximum length	120 symbols / 256 positions for source and target
Layers and dimensions	3 encoder layers, $d_{model} = 192$, 4 heads, feed-forward size 768
Prediction heads	4 edit operations plus 32 character slots per source position
Regularization	Dropout 0.1; weight decay 0.01; label smoothing 0.1
Optimizer / schedule	Fused AdamW; constant learning rate with 5% warm-up
Synthetic stage	3,330 rows; 20 epochs; learning rate 3×10^{-4} ; batch 64
Gold stage	142 rows; 20 epochs; learning rate 1×10^{-4} ; batch 32
Evaluation	Batch 64; deterministic reconstruction; generated CER every 5 epochs
Checkpoint rule	Minimum development CER; all epochs run; one retained Trainer checkpoint
Precision / performance	Float32 parameters; TF32 enabled; FP16 and BF16 disabled
Reproducibility	Model and data seed 2026; shared initialization; deterministic sampling

CUDA’s strict deterministic-kernel mode is disabled for throughput, so the fixed seed does not guarantee bitwise identity on every device. Only one full training seed was feasible for the final benchmark. This is a computational limitation, not evidence that run-to-run variance is negligible.

4.4.4 Checkpointing, Resumption, and Selective Reruns

Each run is stored under a benchmark, condition, seed, regime, and monotonically numbered attempt directory. Before training, the controller fingerprints the dataset, configuration, shared initialization, and parent synthetic checkpoint. An unchanged completed run is reused; an interrupted attempt resumes from its latest Hugging Face checkpoint; changed inputs or an explicit rerun create a new attempt without overwriting earlier evidence.

Every attempt saves the model and tokenizer, experiment input, run manifest, training state, history CSV, development predictions, final metrics, live JSON Lines log, status JSON, and training-curve plot. Console progress remains visible through `tqdm`. Aggregate summaries and figures are regenerated from saved artifacts, so rerunning M6, for example, does not require retraining M1–M4.

The test command first requires selected checkpoints for gold and all five synthetic conditions. It writes a fingerprint over the selected manifests and refuses to replace an existing frozen evaluation with different checkpoints. This guard separates iterative

development from the single confirmatory test comparison.

4.5 Training Environment

Development and final model training run primarily on Windows. The final run manifests record Python 3.11.0, PyTorch 2.8.0 with CUDA 12.9, Transformers 5.5.4, Datasets 2.20.0, and an NVIDIA GeForce RTX 5060 Laptop GPU with 8,151 MiB reported video memory. Training uses local compute; no paid cloud platform is part of the recorded benchmark.

Eflomal compilation and alignment require Linux/WSL, but its serialized lexicon is portable and is reused by the Windows generator. The IBM Model 1 trial provides a fully Windows-native alignment alternative, although it is not substituted into M2 without a separate validation. Gemma filtering is served locally through Ollama. Other principal dependencies include pandas, a Parquet engine, Gensim, Flask, RapidFuzz, SentencePiece, Matplotlib, `yt-dlp`, `youtube-comment-downloader`, and `langdetect`.

Project data are separated from learned and experimental artifacts: `data/` contains registered corpora and annotations, `artifacts/` contains lexicons and M6 profiles, `checkpoints/` contains versioned model attempts, and `results/` contains summaries, plots, selections, and frozen-test outputs. Centralized configuration reduces path duplication, while UTF-8 audits and automated tests cover split integrity, generation isolation, leakage checks, metric behavior, checkpoint fingerprints, resumption, and selective reruns.

The saved artifacts do not reliably record the exact CPU, system RAM, energy consumption, or total elapsed duration, so I do not report numerical claims for them. Runtime and peak CUDA allocation are retained for completed modeling attempts when the library provides those values.

4.6 Evaluation Methodology

Evaluation is multi-level because a generator can look plausible yet provide a poor training signal, or look mechanically simple yet teach a useful correspondence. Intrinsic corpus evidence describes what each method generated; downstream metrics answer the central comparative question; qualitative inspection explains artifacts and semantic failures.

4.6.1 Intrinsic Synthetic-Corpus Analysis

For each export and matched sample, the analysis records total and unique pairs, unchanged and changed counts, duplicate removal, edit counts, rule or mapping cover-

age, rejection reasons, formal-source coverage, sentence lengths, and held-out overlap. Method-specific manifests add alignment substitutions for M2, embedding and alignment scores for M3, wrapper validation for M4, mapping acceptance for M5, and POS/error/template distributions for M6.

The gold-training linguistic annotations provide a reference distribution for error types, language classes, POS categories, protection decisions, and edit budgets. Similarity to this distribution is descriptive rather than a tuning target for every baseline: making M1–M4 use gold labels would erase the intended distinction between generic/external methods and M6.

Seeded 100-row review sheets were generated for M2, M3, M4, and M6. Their fields cover meaning preservation, informal or spelling plausibility, code-switch naturalness where applicable, severity, artifacts, and notes. Since these sheets were incomplete when training started, they can support later diagnosis but cannot currently provide a complete quantitative method ranking.

4.6.2 Automatic Normalization Metrics

Before scoring, predictions and references are normalized to Unicode NFC and internal whitespace is collapsed. Casing is preserved for character metrics and exact match. Let S_c , D_c , and I_c be the corpus totals of character substitutions, deletions, and insertions relative to references containing N_c characters. Character error rate is defined in Equation (4.4).

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c}. \quad (4.4)$$

CER is sensitive to diacritics and short graphemic changes. Lower is better, and values can exceed one when insertions are numerous. Word error rate applies the analogous token-level calculation in Equation (4.5) to case-folded Unicode word-and-punctuation tokens:

$$\text{WER} = \frac{S_w + D_w + I_w}{N_w}. \quad (4.5)$$

WER captures token and boundary recovery but treats a one-character misspelling as a complete token error. The two measures are therefore complementary.

The implementation also reports corpus chrF with character n-gram orders one through six and $\beta = 2$. Whitespace is removed within the n-gram calculation, and the score is reported on a 0–100 scale. Exact match is the fraction of full predictions equal to the reference after NFC and whitespace normalization.

Correction precision, recall, and F1 compare edit signatures relative to the original source. A token-level sequence matcher identifies non-equal source–target spans; a

predicted edit is a true positive only when its source span and replacement tokens exactly match a gold edit. If TP , FP , and FN denote matched, unnecessary, and missed edit signatures, the measures are defined in Equation (4.6).

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}. \quad (4.6)$$

The reported edit-level overcorrection rate is $FP/(TP + FP)$. A separate sentence-level rate checks whether a system changes inputs whose source already equals the reference. Span alignment makes these measures stricter than CER and can be unstable for spacing merges or complex rewrites, so I interpret them together with predictions and error examples.

4.6.3 Development Selection and Frozen Test Comparison

The 30 development examples are used for checkpoint selection. The primary selection metric is CER, with WER, chrF, exact match, correction scores, and overcorrection retained for diagnosis. A method is not selected because it performs well on one convenient secondary metric. Once M6 and every other condition have a fixed selection record, all checkpoint paths and manifest hashes are frozen.

The 29 test inputs are then evaluated with the same preprocessing and greedy decoding. M0 is calculated directly, and gold-only plus M1–M4/M6 are loaded from the frozen paths. The output includes per-row predictions, metrics, inference time, checkpoint identity, and a test-summary CSV. No generator, model parameter, training duration, or checkpoint can be changed in response to the test scores.

Because only one model seed is available and the test set is small, the final report does not treat minor decimal differences as robust rankings. A paired sentence-level bootstrap with 5,000 resamples and seed 2026 quantifies test-sample uncertainty for CER and for each system’s CER difference from M0. These intervals measure uncertainty due to the 29 observed test sentences only; they cannot substitute for missing training-seed variance. Error-category breakdowns are presented only where the number of relevant examples is sufficient; otherwise they remain qualitative.

4.6.4 External Oolel-Corrector Comparison

The published `Oolel-Corrector` is a ready informal-to-standard system rather than a synthetic generation method [5]. An unchanged evaluation on the same gold test inputs could provide useful external context, but it was not completed here. A later run would require a pinned model revision and license, an overlap audit against its training data, and a record of the prompt, decoding, output parsing, hardware, latency, and memory.

Because the corrector uses a different architecture and training history, its score

would not belong in the controlled generator ranking. If evaluated later, it would appear in a separate external-system comparison against M0 and the best frozen project checkpoint.

4.6.5 Final Evaluation State

All six trained conditions completed development selection. Their checkpoint paths and run-manifest hashes were frozen under selection fingerprint `b3fca292...69b564`, after which the 29 locked test inputs were evaluated once on 13 September 2026. The evaluation wrote one prediction file and one metric file per condition together with a common summary. No generator, training hyperparameter, or checkpoint was altered in response to these test scores.

M1–M4 and the official M6 export entered training under a documented quality-gate override because their optional 100-row human-review sheets were not complete. Schema, provenance, leakage, and reproducibility checks were nevertheless retained. Consequently, automatic transformation statistics and downstream results are valid for the saved artifacts, while claims about human-perceived naturalness remain limited. A later hierarchical M6 preview was created for inspection only; because it postdates the frozen comparison, it is excluded from all results in Chapter 5.

Retrieval, RAG, machine-translation transfer, and the external Oolel-Corrector evaluation remain future work. They require relevance or translation judgments, a pinned external model revision, and a distinct comparison protocol.

Chapter 5

Results and Analysis

This chapter presents the completed benchmark with the TNT-inspired non-autoregressive edit Transformer. I first compare the transformations found in the gold-training comments with those generated by the matched synthetic corpora. I then report checkpoint selection and the single frozen-test evaluation.

The results are mixed but informative. Synthetic pretraining changes the model’s behaviour, and four methods reduce test character error relative to leaving the input unchanged. Word error, however, remains worse than the identity baseline, strict correction precision is very low, and the ranking changes across metrics and splits. The evidence supports a comparison of training signals, with M4 strongest on the main character metrics, but not a claim that any of the resulting models is ready for deployment.

5.1 Authentic Gold-Training Transformations

5.1.1 Annotation Coverage and Language Composition

The 142 gold-training sentences contain 1,315 saved linguistic occurrences. Of these, 1,297 (98.6%) are marked reviewed and 18 (1.4%) uncertain. The dominant language relation is Wolof-to-Wolof: 900 occurrences (68.4%), of which 599 were changed during normalization. French-to-French spans account for 215 occurrences (16.3%); only 18 of them were changed, principally when the French surface form itself required spelling correction. A further 185 occurrences (14.1%) remain unknown-to-unknown, reflecting tokens for which the available dictionaries and contextual review did not justify a stronger language claim.

Protection is not marginal. The annotation marks 407 occurrences (31.0%) as protected and a further 25 as uncertain; 169 person entities were identified. All 174 occurrences assigned the PROP_N category were unchanged, consistent with the deliberate protection of names. These observations support the methodological choice to preserve names and legitimate French spans instead of applying Wolof spelling rules indiscriminately. They also show why a generator that only learns global character substitutions cannot fully represent the task.

5.1.2 Error Inventory

Table 5.1 lists the most frequent gold-training labels. Counts are occurrence-level and multi-label: a token may simultaneously be tagged, for example, as a substitution, a diacritic change, and phonetic spelling. The percentages therefore do not sum to 100%.

Table 5.1 – Principal transformation labels in the 1,315 gold-training occurrences.

Label	Count	Occ. %	Label	Count	Occ. %
Identity	689	52.4	Deletion	194	14.8
Phonetic spelling	309	23.5	Diacritic	177	13.5
Substitution	294	22.4	Spacing merge	98	7.5
Insertion	250	19.0	Repetition	47	3.6

Identity is the single largest class, covering 689 occurrences. This does not imply that half of the annotated sentences require no normalization: every retained gold sentence differs from its target. Instead, it shows that normalization is sparse within a sentence and that most surrounding material remains unchanged. Among changed units, the leading labels are phonetic spelling, substitution, insertion, deletion, and diacritic restoration. Spacing merges are also substantial and often correspond to several formal words typed as one informal unit. The resulting task combines local copying, character repair, word-boundary recovery, and selective non-intervention.

Figure 5.1 visualizes the non-identity labels. Less frequent categories, including abbreviation, French spelling, punctuation, and uncertainty, are tabulated in Appendix B.7.

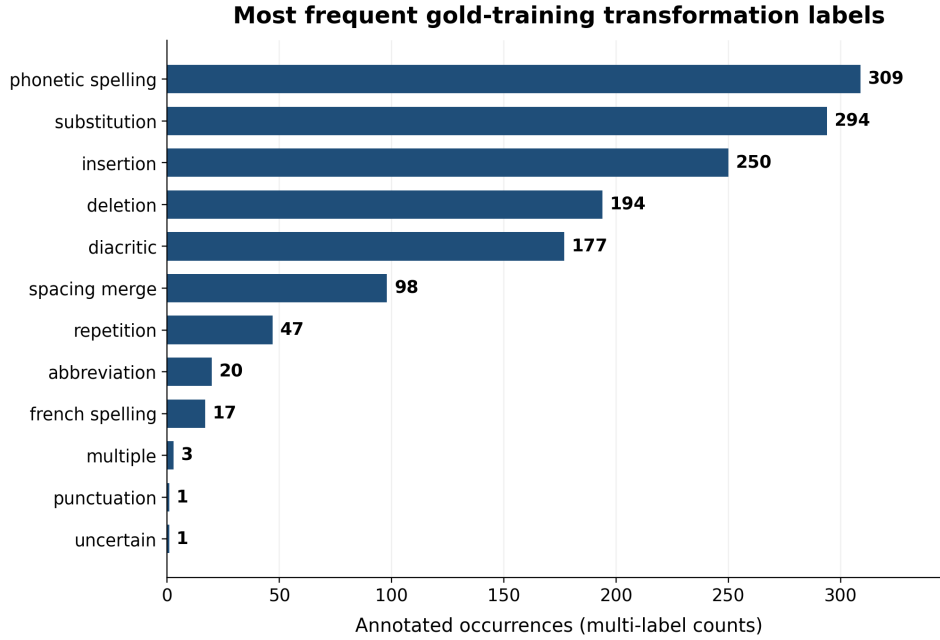


Figure 5.1 – Most frequent manually reviewed transformation labels in the gold-training data. Since labels are multi-valued, the sum exceeds the number of changed occurrences. Source: gold-training annotations.

5.1.3 Part-of-Speech Differences

Variation is not distributed uniformly across grammatical categories. Verbs are both frequent and highly variable: 207 of 268 annotated verb occurrences (77.2%) differ from their formal forms. Pronouns change in 40 of 70 cases (57.1%), nouns in 115 of 235 cases (48.9%), and determiners in 26 of 105 cases (24.8%). Proper nouns remain unchanged in all 174 observed cases, consistent with the name-protection policy. The UNKNOWN group has 149 changes among 263 occurrences (56.7%); it includes difficult lexical cases and many spacing-merge units that do not correspond to one conventional POS.

These figures are descriptive of this small corpus, not universal probabilities for Wolof. Nevertheless, they justify M6’s attempt to condition reusable transformations on POS and protection status. After excluding protected, uncertain, non-Wolof, and otherwise ineligible rows, its frozen training profile retained 865 occurrences, including 588 changed occurrences, from which it accepted 171 lexical mappings and 105 contextual character transformations. The complete POS cross-tabulation appears in Appendix B.7.

5.2 Intrinsic Comparison of Training Corpora

5.2.1 Perturbation Strength

To compare exposure without consulting development or test targets, each synthetic source was scored against its common formal target. The same calculation was applied to the 142 authentic gold-training pairs. These source-to-target metrics measure perturbation magnitude, not naturalness or semantic preservation. Table 5.2 and Figure 5.2 show the result.

Table 5.2 – Intrinsic source-to-target distance of gold train and matched synthetic corpora.

Corpus	Rows	Changed %	CER ↓	WER ↓	chrF ↑
Gold train	142	100.0	0.210	0.539	59.74
M1 rules	3,330	99.9	0.253	0.569	47.10
M2 Eflomal + rules	3,330	100.0	0.348	0.626	38.37
M3 CMDR + rules	3,330	99.9	0.300	0.596	43.44
M4 Oolel external	3,330	100.0	0.262	0.606	45.81
M6 gold-informed	3,330	93.3	0.046	0.135	88.56

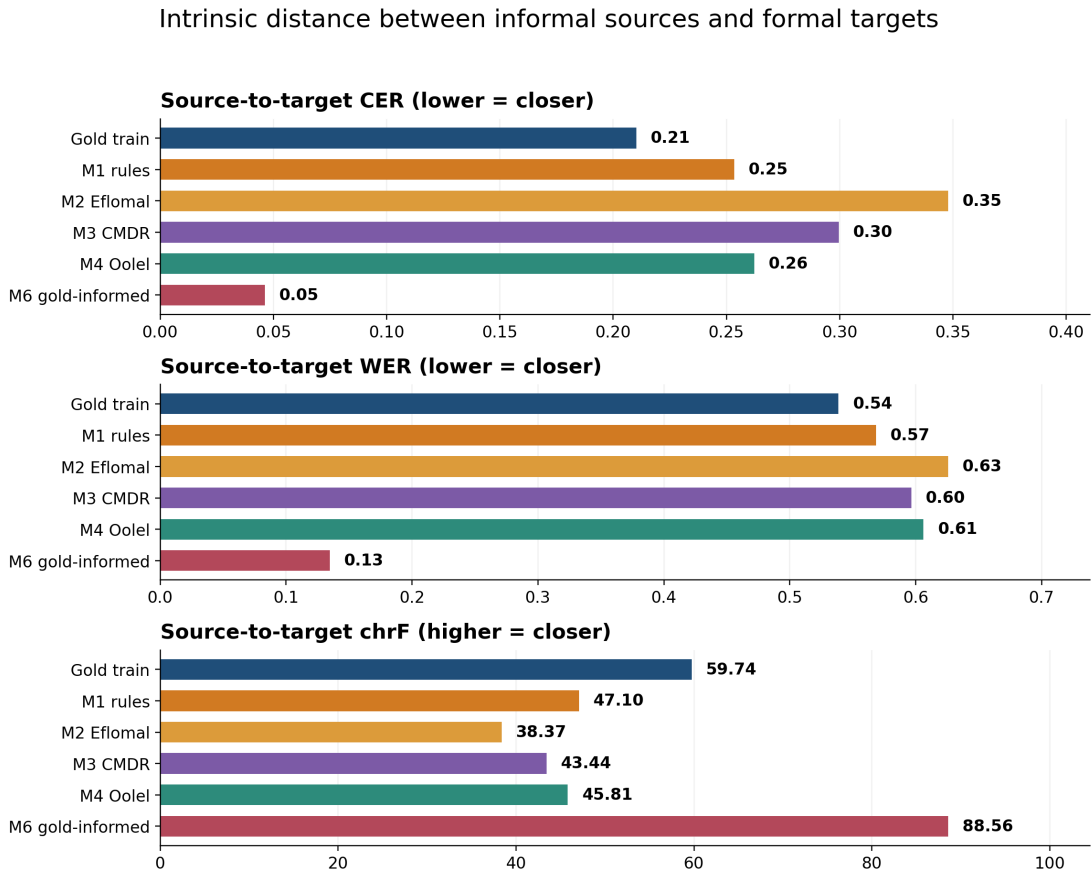


Figure 5.2 – Distance between each informal source corpus and its formal targets. Gold train is an authentic reference; all synthetic values use the matched 3,330-target samples. Source: saved benchmark outputs.

The authentic comments average 10.10 character edits and 5.40 word edits per

sentence. At the aggregate level, M1 is closest to their CER and WER, while M4 has a similar CER but a somewhat larger WER. M2 is the strongest perturbation, partly because 94.5% of its matched rows receive at least one alignment-based French substitution in addition to the shared spelling rules. M3 applies an alignment substitution to 32.9% of rows and consequently lies between M1 and M2.

M6 occupies the opposite extreme. Its matched inputs average 2.35 recorded generator edits: 0.92 lexical-mapping edits and 1.43 contextual transformation edits. Its CER is only 0.046 and 6.7% of rows remain identical to their targets. Although M6 draws its transformations from authentic training annotations and respects language and entity protection, it under-represents the overall amount of change observed in the selected gold comments. Its high intrinsic chrF of 88.56 mainly reflects conservative generation, not necessarily better synthetic quality.

5.2.2 Transformation Coverage and Mismatch

M1 averages 16.01 recorded rule applications per matched row. Several operations fire on most of the corpus: double-vowel reduction appears in 93.3% of rows, $u \rightarrow ou$ in 91.0%, lowercasing in 88.7%, and final-*e* or final-*n* transformations in more than 60%. This provides broad and consistent supervision, but it is much less sparse than the gold occurrence profile and may teach the evaluator to expect changes everywhere.

M2 averages 2.62 alignment substitutions and 13.72 spelling-rule edits per row. The near-universal code switching is difficult to reconcile directly with the gold annotation, where French-to-French material accounts for 16.3% of occurrences and is normally preserved. The units are not identical—row-level switches and occurrence-level language labels cannot be equated—but the contrast indicates a plausible distribution mismatch. M3 is gentler, averaging 0.46 alignment substitutions and 15.50 spelling edits. M4 averages 20.20 exported edits per row, but the external dataset exposes only the generic label `external_llm_generation_unspecified`; its transformation mechanism cannot be decomposed as precisely from the published wrapper.

M6 reproduces the gold label vocabulary more directly. In the matched set, insertion appears on 55.1% of rows, diacritic change on 51.2%, phonetic spelling on 46.8%, deletion on 45.0%, substitution on 39.2%, and repetition on 13.5%. Spacing merges, despite appearing 98 times in the gold annotations, occur on only 0.4% of M6 rows. This exposes an important weakness of the frozen M6 design: local transformations transfer readily, whereas multi-token boundary phenomena are difficult to generalize from token-level templates. Detailed mechanism counts are retained in Appendix B.7.

5.2.3 Edit-Operation Supervision

The TNT-inspired alignment converts these corpus differences into operation labels. Table 5.3 reports their distribution over non-padding source positions. This view is especially useful because the architecture learns the operation inventory directly.

Table 5.3 – Character-operation distribution used to train the TNT-inspired evaluator.

Corpus	Keep (%)	Delete (%)	Substitute (%)	Expand (%)
Gold train	81.06	6.37	9.32	3.26
M1 rules	77.07	9.31	11.14	2.48
M2 Eflomal + rules	69.41	10.07	17.69	2.83
M3 CMDR + rules	73.30	10.04	14.08	2.58
M4 Oolel external	76.27	9.14	11.84	2.75
M6 gold-informed	95.61	2.45	1.25	0.69

Gold train assigns 81.1% of source positions to KEEP. M1 and M4 are the closest synthetic conditions at 77.1% and 76.3%, although both still teach more deletions and substitutions than the gold sample. M2 is the most aggressive: only 69.4% of positions are copied and 17.7% are substituted. Conversely, M6 labels 95.6% of positions as KEEP. This class distribution predicts two observed tendencies: Gold-only and M6 receive a strong copying signal, whereas M1–M4 are more willing to alter unseen text. It does not by itself determine quality because an operation can have the right class but the wrong location or replacement character.

5.3 Development-Set Selection

Table 5.4 reports the selected development checkpoints. M0 is included as a diagnostic reference and was not used to select a trained checkpoint.

Table 5.4 – Development results used for checkpoint selection (30 sentences).

Condition	CER ↓	WER ↓	chrF ↑	Exact
M0 identity	0.233	0.631	56.05	0.000
Gold only	0.230	0.627	56.07	0.000
M1 + gold	0.181	0.668	55.89	0.000
M2 + gold	0.197	0.658	56.46	0.000
M3 + gold	0.192	0.668	55.28	0.000
M4 + gold	0.193	0.647	56.59	0.000
M6 + gold	0.194	0.637	56.60	0.000

The non-autoregressive evaluator passes the minimal character-level sanity check that the earlier compact model failed. Every synthetic-pretrained condition obtains lower development CER than M0, with M1 best at 0.181. The picture is less favorable

at word level: Gold-only has the lowest WER, while M6 is the strongest synthetic condition. chrF values are close to identity and M6 is marginally highest. No model produces an exactly correct development sentence.

Selected synthetic-stage epochs are 15 for M1, 10 for M2–M4, and 20 for M6. During gold fine-tuning, the selected epochs are 15 for M1 and M6, 10 for M2, 20 for M3, and 5 for M4; Gold-only also selects epoch 5. This variation confirms that a single epoch budget does not suit every training distribution, while the scheduled checkpoint rule prevents retrospective method-specific tuning. It also foreshadows ranking instability: M1 leads development CER, but this advantage does not transfer unchanged to the held-out video.

5.4 Frozen Test Results

5.4.1 Automatic Metrics

After the six checkpoint records were frozen, all conditions were evaluated once on the 29 test comments. Table 5.5 and Figure 5.3 contain the primary results.

Table 5.5 – Frozen normalization results on the held-out gold test set (29 sentences).

Condition	CER ↓	WER ↓	chrF ↑	Exact
M0 identity	0.220	0.570	58.50	0.000
Gold only	0.220	0.570	58.50	0.000
M1 + gold	0.195	0.607	56.73	0.034
M2 + gold	0.200	0.614	57.90	0.000
M3 + gold	0.210	0.629	55.09	0.000
M4 + gold	0.191	0.598	58.79	0.000
M6 + gold	0.206	<u>0.592</u>	57.98	0.000

Note: bold marks the best value for each metric; underlining marks the best WER among synthetic-pretrained conditions. Exact is sentence exact match.

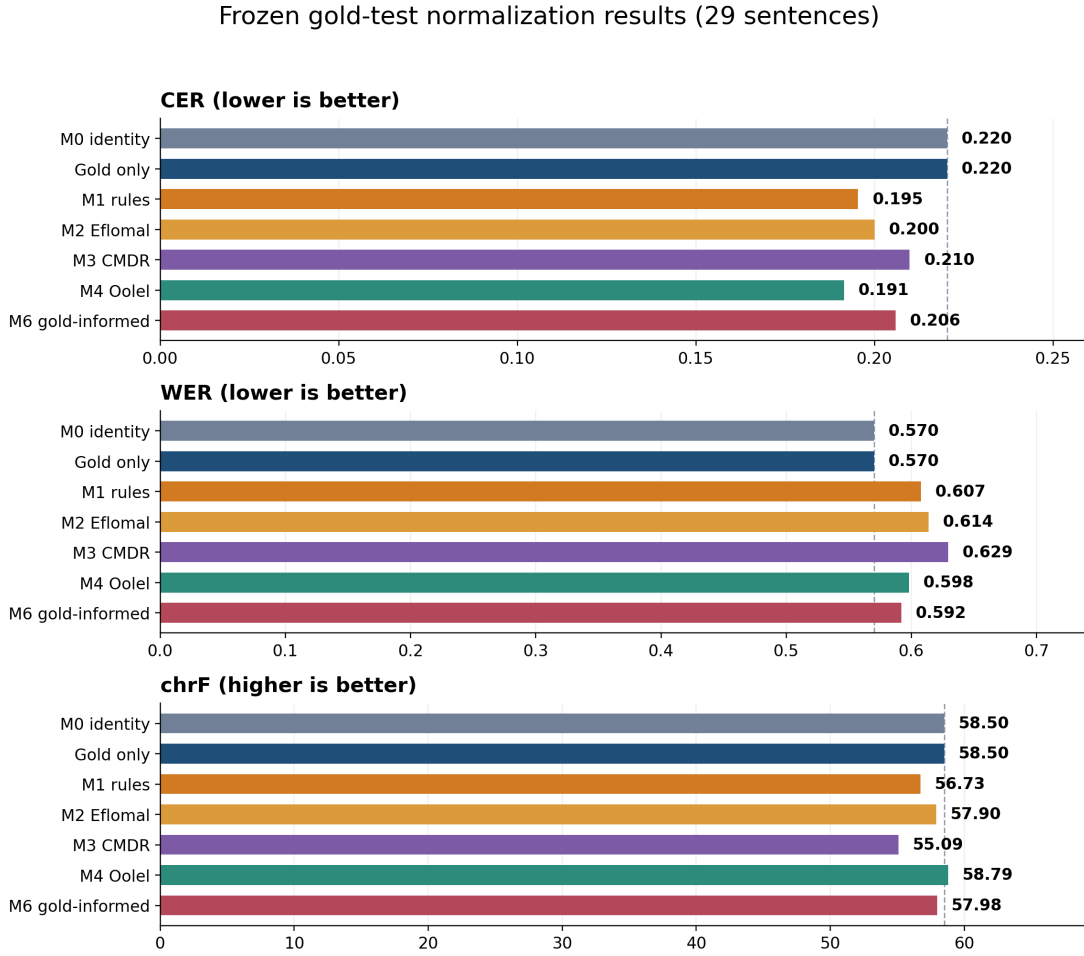


Figure 5.3 – CER, WER, and chrF on the single frozen 29-sentence test evaluation. Source: frozen-test outputs.

Gold-only returns every test input unchanged and is therefore numerically identical to M0. This is an important negative control: 142 training pairs do not overcome the dominant KEEP class sufficiently to produce generalizable corrections. Synthetic pretraining changes this behavior. All five synthetic conditions modify at least 24 of the 29 sentences, and all obtain a lower CER than identity. M4 is best at 0.191, a 13.1% relative reduction from 0.220; M1 follows at 0.195. M4 is also the only trained condition whose chrF exceeds identity, although only by 0.29 points.

The advantage does not extend uniformly to word structure. Identity and Gold-only retain the best WER of 0.570, whereas synthetic-condition WER ranges from 0.592 for M6 to 0.629 for M3. The models often correct a character-level pattern while missing a space, inflection, or lexical replacement required by the reference. M4 improves character distance on 19 test sentences, ties identity on nine, and worsens one. M1 improves 17, ties six, and worsens six; M2 improves 16, M3 15, and M6 15. Only M1 exactly normalizes one sentence, changing *mane bass rek la fan* to *man bass rek la fan*.

The strict edit-signature results remain weak. Across the test set there are 75 required token-span edits. M1 and M6 recover two each, M3 and M4 recover one each,

and M2 recovers none. M6 has the highest correction F1 at 0.030, followed by M1 at 0.028. Depending on the method, between 96.5% and 100% of proposed edit signatures are false positives under this exact-span definition. Some proposed changes still reduce CER without matching an entire gold token-span replacement, so the edit score is stricter than character distance; nevertheless, the very low precision shows that the models are not reliable correctors. The test set contains no unchanged source–reference sentence, so unchanged-sentence safety cannot be measured from this split.

5.4.2 Test-Sample Uncertainty

Table 5.6 reports 95% percentile intervals from 5,000 paired sentence-level bootstrap resamples. The difference column uses the same resampled sentences for a condition and M0.

Table 5.6 – Paired bootstrap uncertainty for test CER (5,000 resamples).

Condition	CER	95% interval	Δ vs. M0 [95% interval]
M0 identity	0.220	[0.172, 0.271]	0.000 [0.000, 0.000]
Gold only	0.220	[0.172, 0.271]	0.000 [0.000, 0.000]
M1 + gold	0.195	[0.157, 0.243]	−0.025 [−0.048, −0.004]
M2 + gold	0.200	[0.159, 0.247]	−0.020 [−0.037, −0.006]
M3 + gold	0.210	[0.166, 0.258]	−0.010 [−0.032, 0.007]
M4 + gold	0.191	[0.151, 0.237]	−0.029 [−0.045, −0.015]
M6 + gold	0.206	[0.160, 0.257]	−0.014 [−0.030, −0.001]

The paired intervals for M1, M2, M4, and M6 remain below zero, indicating a consistent CER reduction relative to identity across bootstrap resamples of these 29 sentences. M3’s interval crosses zero. This supports the claim that several synthetic conditions supply a useful character-level signal on the observed test video. It is not a population-level significance result: the intervals do not quantify initialization, stochastic optimization, annotation choices, or variation between unseen videos.

Pairwise resampling further limits the strength of a winner claim. The M4–M1 CER difference is −0.004 in M4’s favor, with a paired 95% interval spanning approximately −0.019 to 0.010; their ordering is therefore unresolved. M4’s advantage over M2, M3, and M6 does not cross zero in this resampling analysis, but these comparisons remain conditional on one seed, one test video, and the chosen evaluator. The appropriate conclusion is that M4 forms the leading group with M1 rather than that a few thousandths of CER establish a universal ranking.

5.5 Qualitative Prediction Analysis

The new architecture avoids the catastrophic free-generation failure observed in the earlier autoregressive pilot. Every final prediction is non-empty and all conditions produce 29 distinct outputs for 29 inputs. Prediction length remains close to reference length: the aggregate ratio ranges from 0.948 for M1 to 0.968 for M6. The principal failure is therefore no longer sentence replacement or looping, but an inaccurate decision about which local characters to change.

Table 5.7 illustrates the behavior on one short and one longer test example. The first shows a useful but incomplete learned rule. Several synthetic conditions normalize *amoufi* toward *amu fi*, reducing character error, but they do not restore the doubled vowel in *moom* or the required space in *amu fi*. The second example shows how broadly learned substitutions can damage names or already acceptable spans while still correcting some frequent correspondences.

Table 5.7 – Representative frozen-test predictions from the TNT-inspired evaluator.

Field	Text
Informal input A	<i>mareme mom dey forcer trop vraiment amoufi place</i>
Gold target A	<i>mareme moom dey forcer trop vraiment amu fi place</i>
Gold-only A	<i>mareme mom dey forcer trop vraiment amoufi place</i>
M1 / M4 / M6 A	<i>mareme mom dey forcer trop vraiment amufi place</i>
M2 A	<i>mareme mom dey forcer trop vraiment amëfi place</i>
Informal input B	<i>bass miii kou beugue leuffff leu boy biiii sonal nga domou diambour dji waay ish.</i>
Gold target B	<i>bass mii ku bëgg lëf la boy bi sonal nga doomu jaambur ji waay ish</i>
M4 B	<i>bass miii ku bëgëe lëffff lu boy biiii sonal nga domu diambur ji waay isx.</i>
M6 B	<i>bass miii kñu bggggg léfff lë boy biji sonal nga domu diambñur ji waay ish.</i>

Across predictions, the most frequent learned actions include deleting *o* or *e*, replacing *kh* by *x*, and converting spellings such as *eu* toward Wolof diacritics. These directions are plausible in many contexts, but the model applies them without adequate lexical or entity discrimination. Longer examples accumulate several locally plausible but jointly incorrect edits. Spacing remains particularly difficult because a missing or added boundary alters the token-level edit signature even when neighboring characters improve. This explains the combination of lower CER, worse WER, and extremely low correction F1.

The qualitative evidence also clarifies the role of the evaluator. Parallel edit prediction is sufficiently competent to expose differences between synthetic corpora, unlike the collapsed autoregressive pilot. It remains a weak normalizer because its decisions are

character-local, it has no pretrained lexical knowledge, and the Gold corpus provides limited contextual supervision. The relative downstream gains are informative for comparison, but the predictions are not reliable enough for deployment as corrected Wolof text.

5.6 Interpretation with Respect to the Research Question

5.6.1 What Can Be Concluded

The benchmark establishes four findings for the saved configuration.

1. **Synthetic pretraining is useful at character level.** All five synthetic conditions beat Gold-only on test CER, and M1, M2, M4, and M6 have paired bootstrap intervals below identity. This supports H1 for CER, but not for WER or correction F1.
2. **M4 and M1 form the leading character-level group.** M4 has the best frozen-test CER and chrF, while M1 has the best development CER and the only exact test sentence. Their pairwise CER interval overlaps zero, so the data do not separate them reliably.
3. **Transformation calibration matters.** M2’s aggressive operation distribution produces weaker transfer, while M6’s 95.6% KEEP supervision is too conservative. M4 and M1 expose the model to an edit intensity closer to Gold train. This supports H2 and H4: volume and intrinsic closeness are related to, but do not determine, downstream utility.
4. **The evaluator remains limited.** All synthetic systems worsen WER relative to identity and correction F1 remains at or below 0.030. The benchmark is sensitive enough for cautious data comparisons but does not yield a usable normalizer.

These findings answer the research question more cautiously than a simple leaderboard would. Under the fixed TNT-inspired evaluator and limited Gold supervision, synthetic pretraining improves character-level reconstruction of authentic comments, and M4 supplies the strongest overall observed signal. The evidence does not show reliable word-level normalization, nor does it establish M4 as universally superior to M1. Synthetic data therefore reduce part of the orthographic gap, but the result depends on the metric, perturbation distribution, evaluator, and small held-out sample.

5.6.2 Gold-Informed Generation and Fairness

M6’s use of gold-training transformations is best understood as supervised, train-only feature extraction for synthetic pretraining. It is not test leakage: development and test annotations are excluded, exact held-out source/pair overlap is zero, and the final test remained frozen. The comparison is fair only under that interpretation, however. M6 uses information unavailable to M1–M4 and tests whether a small authentic training sample can inform a better generator; it is not a fully data-independent baseline.

The results show both the promise and risk of this strategy. M6 captures authentic labels, POS conditioning, language protection, and entity protection, but its CER of 0.046 against formal targets and 95.6% KEEP operations indicate that it stays much closer to formal text than the authentic comments do. Downstream, M6 obtains the best synthetic-condition WER and correction F1, consistent with its restraint, but only the fourth-best CER. It therefore does not validate H3’s expected overall advantage. The 171 direct lexical mappings may be easier to reuse than the 105 contextual transformations, while spacing merges generalize poorly. A stronger version could learn hierarchical rules and calibrate sentence-level edit budgets to the gold distribution. Such a redesign would use training-derived calibration data and a new benchmark version, leaving the frozen test result unchanged.

5.6.3 Data Availability and Model Complexity

The gold-only control has only 142 training pairs. The synthetic stages expand pretraining to 3,330 matched pairs per method, but all formal targets originate from a limited parallel corpus and each generator provides only one retained realization per target. This is modest supervision for learning sentence-level normalization from scratch. The earlier word-level corrector handled shorter mappings with fewer contextual and word-boundary decisions, so its faster convergence does not imply that this sentence-level task is equally easy.

The chosen evaluator has 2.15 million trainable parameters and is computationally small relative to mT5 or mBART, but it begins from random weights. The edit formulation supplies an appropriate copying bias without a learned copy gate and eliminates autoregressive exposure bias. It still has to infer lexical context, word boundaries, code-switch preservation, and entity safety from the benchmark data. A pretrained multilingual system would bring stronger representations, but earlier mT5 trials were too slow for the available laptop workflow. The final compromise makes a controlled multi-method run possible and produces interpretable local edits, while limiting semantic and long-range modelling capacity.

Saved trainer runtimes sum to approximately 7.8 hours across the six complete conditions when their stages are added sequentially. Gold-only takes 0.20 hours. Each

synthetic-plus-gold condition takes between 1.47 and 1.54 hours, of which approximately 0.20 hours is gold fine-tuning. Similar runtimes are expected because the row counts, epochs, batch sizes, and architecture are fixed. Only one shared initialization and one training seed could be afforded. More authentic data would help estimate contextual edit decisions and would provide a stronger Gold-only control. More synthetic data may help only if additional realizations improve coverage without amplifying method-specific distribution mismatch.

5.7 Threats to Validity

5.7.1 Internal Validity

The controlled design holds targets, row counts, initialization, architecture, gold supervision, and selection rule fixed, and exact held-out input/pair leakage checks return zero. Nevertheless, method review sheets were incomplete and training used a documented override, so semantic preservation and human-perceived naturalness are not fully quantified. M4’s transformation provenance is coarser than that of local generators. M6 uses train-only gold knowledge and is therefore a different class of method. The single seed prevents estimation of optimization variance, and development selection over many epochs may overfit the 30 development examples.

5.7.2 Construct Validity

Sentence normalization is a proxy for the broader aim of robust semantic retrieval and language-model interaction. CER, WER, chrF, and exact edits evaluate reconstruction, not semantic equivalence or retrieval relevance. Intrinsic source-to-target distance measures severity rather than realism. A generator can match gold CER while producing linguistically implausible edits, or can preserve meaning while receiving a poor exact score. The disagreement between improved CER and weak WER/correction F1 demonstrates this construct gap directly.

All retained gold sentences require at least one correction. This makes the test useful for correction ability but unsuitable for measuring whether a model safely leaves already-standard sentences unchanged. A broader test could include unchanged, lightly changed, and heavily changed examples and report results by language mix and error category.

5.7.3 External Validity

The 201 gold pairs come from five videos. Training, development, and test are video-disjoint, but development and test each contain only one video. Video identity is

therefore entangled with subject matter, community vocabulary, and comment style. The frozen test has only 29 sentences, and its bootstrap intervals describe resampling of this one video rather than the wider Wolof-speaking social-media population. YouTube comments also do not cover messaging, search queries, speech transcripts, or other informal domains.

Finally, the results concern one TNT-inspired character edit Transformer and do not generalize directly to pretrained multilingual models, retrieval encoders, or machine-translation systems. The evaluator is sensitive to synthetic pretraining, but its weak exact correction performance makes the observed ranking architecture- and task-specific rather than an intrinsic ordering of M1–M4 and M6.

5.8 Chapter Summary

The gold annotations reveal a sparse normalization task: unchanged context coexists with phonetic spelling, substitutions, insertions, deletions, diacritics, and word-boundary errors, while French spans and proper names often require protection. The synthetic methods expose very different distributions. M1 and M4 produce operation frequencies closest to Gold train, M2 uses aggressive code switching and substitutions, M3 moderates that switching, and M6 transfers authentic train-only patterns but is substantially too conservative.

On the frozen test, Gold-only reproduces the identity baseline at CER 0.220. Synthetic pretraining reduces CER to between 0.191 and 0.210, with M4 best and M1 close behind. M4 also slightly exceeds identity in chrF, while M6 has the best synthetic-condition WER and correction F1. Nevertheless, every synthetic method remains worse than identity on WER and exact edit recovery is rare. The central empirical conclusion is therefore two-sided: synthetic data demonstrably teach transferable character correspondences that the small Gold set does not, but the available supervision and lightweight evaluator are insufficient for reliable sentence normalization or an unconditional generator ranking.

Chapter 6

Conclusion and Future Work

6.1 Answer to the Research Question

I began with a practical observation: systems trained mainly on standardized Wolof often fail when users write the same language with informal orthography, French code switching, omitted spaces, repeated letters, or phonetic spellings. My initial aim was to improve semantic retrieval and interaction with multilingual systems. Since no sufficiently large formal-informal parallel corpus was available, I first investigated whether different synthetic generation strategies could provide useful adaptation data.

The answer to the central research question is qualified but positive at character level. Gold-only training on 142 sentences learned no transferable correction on the frozen test: it returned every input unchanged and obtained a CER of 0.220. All five synthetic-pretrained conditions lowered CER to between 0.191 and 0.210. M4, based on the external Oolel LLM-generated corpus, obtained the best CER (0.191) and chrF (58.79). M1 followed closely with a CER of 0.195 and produced the only exactly normalized test sentence. On the observed 29-sentence sample, paired bootstrap intervals support an improvement over identity for M1, M2, M4, and M6.

That gain does not amount to successful sentence normalization. The identity baseline retains the best WER at 0.570, while the synthetic models range from 0.592 to 0.629. Strict correction F1 never exceeds 0.030, which means that most proposed token-span edits do not exactly match the human normalization. I conclude that synthetic pretraining transfers recurring character correspondences that the small Gold set cannot teach on its own. The available data and evaluator are still insufficient for reliable word boundaries, contextual corrections, and entity-safe normalization. M4 provides the strongest observed synthetic supervision for this architecture, but its small advantage over M1 is too uncertain to support a universal ranking.

6.1.1 Answers to the Operational Research Questions

RQ1 Does synthetic pretraining improve over Gold-only and identity? Yes for CER: every synthetic condition improves on Gold-only, and four have paired bootstrap intervals below identity. No for WER and reliable edit recovery: every

synthetic condition has higher WER than identity and very low correction F1. Hypothesis H1 is therefore supported only for character-level reconstruction.

RQ2 How do the five retained strategies compare under controlled conditions? M4 has the strongest frozen-test CER and chrF, M1 has the strongest development CER and the only exact test match, and M6 has the best synthetic-condition WER and correction F1. M2 and M3 provide weaker transfer despite adding alignment or distributional code-switching components. Hypothesis H2 is supported because the matched corpora produce measurably different operation distributions and downstream behavior, although the small test set prevents a definitive total ordering.

RQ3 Does the Gold-informed M6 engine provide greater utility? Not overall. Its language, POS, entity, and protection constraints make it more conservative and give it the best edit F1 among synthetic methods, but it ranks fourth on CER. M6 labels 95.6% of source positions as KEEP, compared with 81.1% in Gold train, and rarely reproduces spacing merges. H3 is not confirmed in its strong form. The result suggests that correct rule provenance is insufficient if rule coverage and sentence-level edit frequency are poorly calibrated.

RQ4 How closely do synthetic transformations match authentic phenomena? M1 and M4 are closest to Gold train in aggregate edit strength and operation balance. M2 and M3 overproduce substitutions, largely through their code-switching components. M6 captures authentic error labels and protection logic but underproduces changes. These profiles explain part of the downstream ranking while confirming H4: neither maximum perturbation nor minimum source–target distance is a sufficient quality criterion.

RQ5 What do prediction failures reveal? The model learns frequent local relations such as deleting an informal *o*, replacing *kh* with *x*, or restoring diacritic-like forms, but it applies them too broadly and often misses spacing, morphology, lexical context, and names. Longer sentences accumulate locally plausible but jointly incorrect edits. H5 is supported qualitatively, but the small number of examples per error category prevents reliable category-specific significance tests.

6.2 Summary of Contributions and Objectives

Beyond the final metric table, I built an end-to-end, auditable workflow that connects authentic data collection, manual normalization, linguistic annotation, synthetic gener-

ation, leakage-aware sampling, controlled training, frozen evaluation, and report-ready analysis.

1. A corpus of public YouTube comments was collected and processed with recorded provenance, heuristic filtering, and local-model assistance.
2. A Gold benchmark of 201 retained informal–formal pairs was manually produced from five videos and divided into video-disjoint train, development, and test sets of 142, 30, and 29 sentences.
3. The 1,315 Gold-training token occurrences were enriched with normalization labels, POS, language, named-entity, protection, review, and lookup provenance. This exposes the importance of sparse edits, French preservation, proper-name protection, and spacing merges.
4. Five retained synthetic corpora were exported: deterministic Beqi-inspired rules (M1), Eflomal alignment plus rules (M2), distributional code mixing plus rules (M3), external LLM-generated Oolel pairs (M4), and the proposed Gold-informed linguistic engine (M6). The exploratory unsupervised M5 method was documented and excluded when its mapping coverage proved insufficient.
5. A common pool of 3,330 formal targets, fixed initialization, single architecture, seed, training budget, and Gold fine-tuning protocol was enforced. Exact held-out input and pair overlap is zero for every synthetic method.
6. A TNT-inspired non-autoregressive character edit Transformer was implemented as the common evaluator. It removes autoregressive exposure bias, makes evaluation fast, records operation and character supervision, and supports independent resumption or rerunning of each method.
7. Final predictions, manifests, training curves, corpus profiles, bootstrap intervals, and qualitative diagnostics were preserved. The frozen test is protected by a fingerprint over selected checkpoints.

Table 6.1 compares these outcomes with the SMART objectives defined in Chapter 1.

Table 6.1 – Final assessment of the project objectives.

ID	Status	Final assessment
O1	Achieved	Authentic candidate comments, provenance, cleaning, and filtering workflows were implemented and documented.
O2	Achieved	The minimum target of 200 pairs and five videos was reached with 201 validated pairs and a locked video-disjoint split.
O3	Achieved	M1–M4 and M6 were implemented or integrated with traceable exports and manifests.
O4	Achieved	Common targets, row counts, initialization, architecture, seed, Gold data, and overlap audits were held fixed.
O5	Achieved	Six trained conditions completed selection and frozen-test evaluation with the common TNT-inspired evaluator.
O6	Partly achieved	Automatic, intrinsic, annotation, uncertainty, and qualitative analyses were completed; human naturalness review and downstream retrieval evaluation remain incomplete.
O7	Achieved	Code, notebooks, manifests, hashes, checkpoints, predictions, figures, and environment information form a reproducible audit trail.

The work meets its core engineering and comparative objectives. Its scientific conclusion is deliberately narrower than a production claim: synthetic data provide useful character-level supervision in this benchmark, while the observed distribution mismatches help explain why that benefit does not extend to reliable sentence normalization.

6.3 Limitations

6.3.1 Gold Data and Sampling

The principal limitation is the amount and diversity of authentic parallel data. Two hundred and one sentences are enough to create a pilot benchmark but not to estimate the full distribution of informal Wolof. Development and test each contain one video, so video identity is confounded with topic, channel community, and vocabulary. The frozen test has only 29 sentences, and all require at least one correction. It cannot measure whether a system safely preserves sentences that are already standard. Although paired bootstrap resampling represents uncertainty within this test video, it does not represent new videos or new social-media platforms.

Manual annotation is also interpretive. A single informal sentence may allow several acceptable standardizations, particularly for code-switched phrases, names, punctuation, and word segmentation. The token-level review improves transparency but does not eliminate annotator subjectivity. Some occurrences remain uncertain or unknown, and error labels are multi-valued. Independent double annotation and agreement measurement were outside the available time budget.

6.3.2 Synthetic Data Quality

The methods are controlled by target and quantity, but they do not expose identical provenance. M1–M3 and M6 record detailed local transformations, whereas M4 is an external LLM-generated resource with coarser transformation metadata. The planned 100-row human review of every synthetic corpus was not completed before training, so the report cannot provide a complete human ranking of fluency, informality, and semantic preservation. Intrinsic CER, WER, and chrF only measure how much the source differs from the formal target; they do not establish that the change resembles authentic usage or preserves meaning.

M6 occupies a special methodological category because it learns mappings and contextual rules from Gold train. This is not held-out leakage, but it gives M6 supervision unavailable to the generic baselines. Its evaluation therefore answers whether train-only Gold knowledge helps generation, not whether it is a data-independent competitor. M4 may also contain unknown overlap or distributional exposure through its original model training; exact overlap with the project held-out inputs is absent, but its full upstream training history cannot be reconstructed locally.

6.3.3 Evaluator and Metrics

The final evaluator is small, character-level, non-autoregressive, and trained from scratch. Its explicit edit representation is appropriate for sparse normalization and avoids recurrent decoding collapse, but it has limited lexical and semantic knowledge. Source-anchored local segments make long-distance reordering and complex word-boundary recovery difficult. The maximum length of 256 truncates a small number of synthetic sources and targets. The 32-character local segment bound covers all aligned training pairs but is an implementation constraint rather than a linguistic principle.

Metric disagreement is substantial. CER rewards partial orthographic progress even when a token remains incorrect; WER penalizes that token completely. Strict correction F1 requires an exact source span and replacement, so a partially correct edit counts as both a false positive and a missed Gold edit. Conversely, neither CER nor chrF guarantees semantic preservation. Sentence normalization is itself only a proxy for retrieval, translation, and LLM-query robustness. The benchmark therefore measures synthetic usefulness for one reconstruction architecture and cannot directly prove improvement on the motivating downstream applications.

6.3.4 Experimental and Computational Constraints

Only one seed and one shared initialization were used. This improves direct comparability but does not quantify training variance. Gold-dev was consulted at four

scheduled points during synthetic pretraining and again during Gold fine-tuning. No development example contributed gradients, yet repeated selection on 30 sentences can adapt decisions to that small sample. The different development and test rankings illustrate this risk.

Local laptop constraints ruled out a complete controlled comparison with mT5, mBART, or several random seeds. The final suite required approximately 7.8 sequential trainer-hours even with fast parallel reconstruction. Energy use was not instrumented, so no numerical carbon claim is made. Reproducible manifests reduce procedural uncertainty, but they do not compensate for missing model families and seeds.

6.4 Future Directions

6.4.1 Expand and Strengthen the Gold Benchmark

The highest-priority extension is a larger set of authentic pairs drawn from more channels, topics, speakers, time periods, and platforms, with the same provenance and ethical safeguards. A stronger test set would deliberately mix unchanged, lightly changed, and heavily changed sentences and include controlled strata for French code switching, entities, spacing errors, and sentence length. Review of a representative subset by a second qualified annotator would make it possible to report normalization and error-label agreement. Multiple acceptable references could also represent cases where Wolof standardization or code-switch treatment permits more than one defensible answer.

6.4.2 Improve Synthetic Generators Without Using Held-Out Data

M1–M3 could reduce universally firing rules and calibrate edit budgets to Gold-train operation frequencies. Alignment-based substitutions would benefit from stronger lexical, grammatical, and semantic filters so that French segments appear only where the bilingual context supports them. For M6, hierarchical generalization offers a path beyond direct mapping: reusable character rules could be conditioned on local context, POS, language, protection status, and entity class, while spacing merges could be represented as multi-token transformations. Calibration would remain confined to Gold train or an inner split of it; Gold-dev and Gold-test remain evaluation data rather than inputs for redesigning the generator.

Future exports could contain multiple informal realizations per formal target, explicit rejection reasons, and confidence values. Applying the same seeded human-review sample to every method would provide comparable judgments of meaning preservation,

plausibility, code-switch naturalness, and severity. These judgments would complement transformation distance and help distinguish a useful hard example from arbitrary noise.

6.4.3 Use Stronger and Multiple Evaluators

The TNT-inspired model can remain a fast, interpretable character-edit probe. A second evaluator based on multilingual pretraining, such as mT5 or mBART, would show whether the findings survive beyond a randomly initialized local architecture. Parameter-efficient fine-tuning, mixed precision, dynamic batching, and cloud or institutional GPUs could make multiple seeds feasible. A word- or subword-aware edit tagger is another promising middle ground because it could preserve non-autoregressive speed while conditioning decisions on lexical units and character representations.

Most importantly, normalization is only one view of synthetic quality. A retrieval experiment could index formal documents and compare whether informal queries retrieve the correct passages before and after normalization or contrastive adaptation. A machine-translation experiment could measure whether synthetic pretraining improves translations of informal Wolof, while an LLM robustness experiment could compare answers to formal and informal paraphrases. Agreement across these evaluators would provide stronger evidence that a generation method bridges the actual semantic-access gap rather than merely optimizing orthographic similarity.

6.4.4 Deployment, Ethics, and Sustainability

A deployed normalizer would need to expose confidence and preserve the original message rather than silently overwrite it. Names, French spans, dialectal variants, and identity-bearing spellings require special care because an automatic notion of “correction” can erase legitimate variation. Acceptable normalization policies are best defined with Wolof language specialists and users, while error monitoring can be stratified by source community and language mix to avoid optimizing only for the styles most represented in the collected videos.

Any further collection would continue to follow platform terms, data-minimization and retention policies, and the principles of the GDPR [25]. Public availability does not remove privacy concerns, particularly when usernames, entities, or sensitive content are present. Separate documentation of redistribution rights would clarify what can be shared for YouTube data, dictionaries, external models, and generated corpora. From a sustainability perspective, the fast edit evaluator is useful for screening experiments before committing to larger pretrained models; later experiments could record energy or GPU-hours and reserve expensive runs for protocols fixed in advance.

6.5 Final Perspective

The project does not end with a universal Wolof corrector, but it resolves an important methodological uncertainty. Synthetic data are not automatically valuable because they are numerous, natural-looking, or derived from authentic errors. Their usefulness depends on which transformations they teach, how often they fire, what content they preserve, and whether an evaluator can exploit them. In this study, synthetic pretraining succeeds in transferring character-level normalization patterns that the small Gold set cannot learn alone. M4 and M1 provide the strongest observed signals, while M6 demonstrates that linguistic safeguards improve restraint but require better coverage and calibration.

The resulting corpus, annotation scheme, generation engines, matched benchmark, edit evaluator, and frozen evidence form a reusable foundation for the original goal: making informal Wolof easier to retrieve, translate, and process. The next decisive step is not to optimize another decimal place on the present 29 sentences, but to expand authentic evaluation and test whether the synthetic-data gains survive stronger models, multiple seeds, and semantic downstream tasks.

Bibliography

- [1] K. Al Sharou, Z. Li, and L. Specia, “Towards a better understanding of noise in natural language processing,” in *Proceedings of RANLP*, 2021, pp. 53–62. DOI: 10.26615/978-954-452-072-4_007
- [2] A. Das and B. Gambäck, “Code-mixing in social media text: The last language identification frontier?” *Traitement Automatique des Langues*, vol. 54, no. 3, pp. 41–64, 2013.
- [3] C. M. B. Dione, “Finite-state tokenization for a deep wolof lfg grammar,” *Bergen Language and Linguistics Studies*, vol. 8, no. 1, 2017. DOI: 10.15845/bells.v8i1.1340
- [4] D. Mbaye and M. Diallo, “Beqi: Revitalize the senegalese wolof language with a robust spelling corrector,” in *Innovations and Interdisciplinary Solutions for Underserved Areas*, Preprint published in 2023, 2025, pp. 311–325. DOI: 10.1007/978-3-031-86493-3_25 arXiv: 2305.08518.
- [5] Soynade Research, *Oolel-corrector: A fine-tuned wolof spelling corrector*, Hugging Face Models, License: AGPL-3.0. Accessed: 2026-07-26, 2026. [Online]. Available: <https://huggingface.co/soynade-research/Oolel-Corrector>
- [6] D. Jin, Z. Jin, Z. Hu, O. Vechtomova, and R. Mihalcea, “Deep learning for text style transfer: A survey,” *Computational Linguistics*, vol. 48, no. 1, pp. 155–205, 2022. DOI: 10.1162/coli_a_00426
- [7] K. Dekker and R. van der Goot, “Synthetic data for english lexical normalization: How close can we get to manually annotated data?” In *Proceedings of LREC*, 2020, pp. 6300–6309.
- [8] S. Gouws, D. Hovy, and D. Metzler, “Unsupervised mining of lexical variants from noisy text,” in *Proceedings of the First Workshop on Unsupervised Learning in NLP*, 2011, pp. 82–90.
- [9] C. Li and Y. Liu, “Improving text normalization via unsupervised model and discriminative reranking,” in *Proceedings of the ACL Student Research Workshop*, 2014, pp. 86–93. DOI: 10.3115/v1/P14-3012

- [10] A. Roy, S. Ghosh, K. Ghosh, and S. Ghosh, “An unsupervised normalization algorithm for noisy text: A case study for information retrieval and stance detection,” *ACM Journal of Data and Information Quality*, vol. 13, no. 3, pp. 1–25, 2021, Article 17. DOI: [10.1145/3418036](https://doi.org/10.1145/3418036)
- [11] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2009.
- [12] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [13] C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [14] L. Xue et al., “Mt5: A massively multilingual pre-trained text-to-text transformer,” in *Proceedings of NAACL-HLT*, 2021, pp. 483–498. DOI: [10.18653/v1/2021.naacl-main.41](https://doi.org/10.18653/v1/2021.naacl-main.41)
- [15] Y. Liu et al., “Multilingual denoising pre-training for neural machine translation,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 726–742, 2020. DOI: [10.1162/tacl_a_00343](https://doi.org/10.1162/tacl_a_00343)
- [16] G. Jawahar, E. M. B. Nagoudi, M. Abdul-Mageed, and L. V. S. Lakshmanan, “Exploring text-to-text transformers for english to hinglish machine translation with synthetic code-mixing,” in *Proceedings of the Fifth Workshop on Computational Approaches to Linguistic Code-Switching*, 2021, pp. 36–46. DOI: [10.18653/v1/2021.calcs-1.6](https://doi.org/10.18653/v1/2021.calcs-1.6)
- [17] P. F. Brown, V. J. Della Pietra, S. A. Della Pietra, and R. L. Mercer, “The mathematics of statistical machine translation: Parameter estimation,” *Computational Linguistics*, vol. 19, no. 2, pp. 263–311, 1993.
- [18] R. Östling and J. Tiedemann, “Efficient word alignment with markov chain monte carlo,” *The Prague Bulletin of Mathematical Linguistics*, vol. 106, pp. 125–146, 2016. DOI: [10.1515/pralin-2016-0013](https://doi.org/10.1515/pralin-2016-0013)
- [19] C.-Y. Li and N. T. Vu, “Improving code-switching language modeling with artificially generated texts using cycle-consistent adversarial networks,” in *Proceedings of Interspeech*, 2020, pp. 1057–1061.
- [20] K. R. Chandu and A. W. Black, “Style variation as a vantage point for code-switching,” in *Proceedings of Interspeech*, 2020, pp. 4761–4765. DOI: [10.21437/Interspeech.2020-2574](https://doi.org/10.21437/Interspeech.2020-2574)

- [21] Soynade Research, *Wolof non-standard to standard parallel pairs*, Hugging Face Datasets, License: CC BY-SA 4.0. Accessed: 2026-07-26, 2026. [Online]. Available: <https://huggingface.co/datasets/soynade-research/Wolof-Non-Standard-Orthography>
- [22] Z. Zhang et al., “Style transfer as unsupervised machine translation,” *arXiv preprint arXiv:1808.07894*, 2018. arXiv: 1808.07894.
- [23] J. He, X. Wang, G. Neubig, and T. Berg-Kirkpatrick, “A probabilistic formulation of unsupervised text style transfer,” in *Proceedings of ICLR*, 2020.
- [24] F. Tan, Y. Hu, C. Hu, K. Li, and K. Yen, “TNT: Text normalization based pre-training of transformers for content moderation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 4735–4741. DOI: 10.18653/v1/2020.emnlp-main.379
- [25] European Parliament and Council of the European Union, *Regulation (eu) 2016/679 (general data protection regulation)*, Official Journal of the European Union, L 119, Especially Articles 5 and 89, 2016.
- [26] Google for Developers. “Youtube data api overview.” Official technical documentation, Accessed: Sep. 16, 2026. [Online]. Available: <https://developers.google.com/youtube/v3/getting-started>

Appendix A

Implementation and Reproducibility Reference

This appendix identifies the implementation units needed to reproduce the methodology in Chapter 4. It is not a substitute for the complete source repository, available at https://github.com/IThioye/wolof_synthetic_data_generation. Generated corpora, model checkpoints, and raw platform data are intentionally kept separate from source code because their size, provenance, and redistribution constraints differ.

A.1 Source-Code Map

Table A.1 maps the principal implementation units to their responsibilities and outputs.

Table A.1 – Principal implementation units and responsibilities.

Path	Responsibility	Main outputs
<code>src/config.py</code>	Central path registry and operational constants	Consistent data, artifact, checkpoint, and result locations
<code>src/annotation/</code>	Sentence-level and token-level Flask annotation services	Gold history, corrections, linguistic labels, and curated lookup
<code>src/pipelines/</code>	Cleaning, reconstruction, reconciliation, split creation, and integrity audits	Corrected gold data and video-disjoint splits
<code>src/generation/benchmark_baselines.py</code>	M0 identity export and M1 Beqi-inspired rules	M0/M1 pairs and manifests
<code>src/generation/alignment_benchmarks.py</code>	Eflomal and distributional code-mixing generation	M2/M3 pairs, reviews, and manifests

Path	Responsibility	Main outputs
src/generation/ oolel_baseline.py	Validation and adaptation of the external Oolel dataset	M4 accepted/rejected rows and manifest
src/generation/ youtube_variant_ mining.py	Leakage-controlled lexical-mining prototype	Excluded M5 candidates, mappings, and diagnostics
src/generation/gold_ linguistic_engine.py	Gold-train-only mapping and transformation engine	M6 pairs, profile, mappings, templates, and manifest
src/modeling/tnt_ edit_transformer.py	Character vocabulary, edit alignment, encoder model, reconstruction, and initialization	Shared initialization and model checkpoints
src/modeling/tnt_ benchmark_runner.py	Final TNT namespace and architecture protocol	Architecture record and benchmark entry points
src/modeling/ compact_benchmark_ runner.py	Reused matched sampling, training orchestration, checkpoint selection, and test freezing	Protocol, audits, selections, and frozen evaluation
src/modeling/metrics. py	Shared normalization metrics	CER, WER, chrF, exact match, correction scores, and overcorrection
notebooks/m0_ identity_baseline. ipynb to m6_gold_ linguistic_engine. ipynb	Inspectable method construction and export workflow	Method-specific datasets and diagnostic displays
notebooks/m7_tnt_ edit_gold_dev_ benchmark.ipynb	Final interactive benchmark controller	Per-condition training, Gold-development selection, and final test trigger

A.2 Fair Common-Target Selection

Listing A.1 is the central fairness control. Each generator is first reduced to at most one candidate per canonical formal target. The intersection is then sampled once with

seed 2026, and the same target surface form is imposed on all methods. Finally, every selected dataset is checked against both held-out gold partitions.

```

1         "target_key", keep="first"
2     ).reset_index(drop=True)
3
4     common_targets = set.intersection(
5         *(set(frame["target_key"]) for frame in
6         unique_by_method.values())
7     )
8     if len(common_targets) < protocol.matched_unique_rows:
9         raise ValueError(
10             f"Only {len(common_targets):,} formal targets occur
11             in every method; "
12             f"{protocol.matched_unique_rows:,} are required"
13         )
14     selected_targets = set(
15         pd.Series(sorted(common_targets), dtype=str)
16         .sample(n=protocol.matched_unique_rows, random_state=
17         protocol.seed)
18         .tolist()
19     )
20     # Use one shared surface form as well as one shared
21     canonical target. This
22     # prevents a stray case/Unicode variant in a generator
23     artifact from
24     # changing the supervision for only that method.
25     target_reference = unique_by_method["m4"].set_index("
26     target_key")["target_text"]
27
28     for method in METHODS:
29         unique = unique_by_method[method]
30         matched = unique.loc[unique["target_key"].isin(
31         selected_targets)].copy()
32         matched["target_text"] = matched["target_key"].map(
33         target_reference)
34         matched = (
35             matched.drop(columns="target_key")
36             .sort_values("target_text", kind="stable")
37             .reset_index(drop=True)
38         )
39         if len(matched) != protocol.matched_unique_rows:

```

```

32         raise RuntimeError(
33             f"Common-target selection produced {len(matched
34             ):,} rows for {method}"
35         )
36         dev_audit = require_no_held_out_leakage(matched, gold["
37         dev"])
38         test_audit = require_no_held_out_leakage(matched, gold[
39         "test"])
40         selected[method] = matched
41         audit_rows.append(
42             {
43                 "method": method,
44                 **source_counts[method],
45                 "unique_target_rows": len(unique),
46                 "selected_rows": len(matched),
47                 "common_target_pool": len(common_targets),
48                 "seed": protocol.seed,
49                 "dev_input_overlap": dev_audit.input_overlap,
50                 "dev_pair_overlap": dev_audit.pair_overlap,
51                 "test_input_overlap": test_audit.input_overlap,
52                 "test_pair_overlap": test_audit.pair_overlap,
53                 "source_over_255_chars": int(matched["
54         source_text"].str.len().gt(255).sum()),
55                 "target_over_255_chars": int(matched["
56         target_text"].str.len().gt(255).sum()),
57             }
58         )
59     )

```

Listing A.1 – Shared formal-target selection and held-out leakage checks.

A.3 Frozen-Test Guard

The test routine in Listing A.2 requires explicit confirmation, refuses to run while any checkpoint selection is missing, fingerprints all selected manifests, and rejects an attempt to overwrite an existing test comparison with different selections. This makes the separation between development and confirmation executable rather than relying only on researcher discipline.

```

1     return {
2         "benchmark": BENCHMARK_NAME,
3         "seed": protocol.seed,
4         "protocol": asdict(protocol),

```

```

5         "selections": selections,
6     }
7
8
9 def evaluate_frozen_test(
10     gold_test: pd.DataFrame,
11     protocol: CompactBenchmarkProtocol,
12     *,
13     confirm: bool = False,
14 ) -> pd.DataFrame:
15     """Evaluate all finalized conditions once; require an
16     explicit confirmation."""
17     if not confirm:
18         raise RuntimeError(
19             "Frozen test evaluation is disabled. Set confirm=
20             True only after all "
21             "seven conditions and the protocol are final."
22         )
23     missing = missing_selections()
24     if missing:
25         raise RuntimeError(f"Cannot evaluate the test set;
26         missing selections: {missing}")
27
28     TEST_DIR.mkdir(parents=True, exist_ok=True)
29     freeze_path = RESULTS_DIR / "frozen_test_manifest.json"
30     payload = _freeze_payload(protocol)
31     canonical = json.dumps(payload, ensure_ascii=False,
32                             sort_keys=True)
33     payload["selection_fingerprint"] = hashlib.sha256(canonical
34                                                         .encode("utf-8")).hexdigest()
35     payload["evaluated_at_utc"] = utc_now()
36     if freeze_path.is_file():

```

Listing A.2 – Checkpoint completeness and immutable frozen-test fingerprint.

A.4 Shared Generation Metrics

All trained conditions and the identity reference use the same metric implementation. Listing A.3 shows the corpus aggregation for CER, WER, chrF, and exact match. Correction-specific metrics are calculated in the same module by comparing edit signatures relative to the input.

```

1 def generation_metrics(predictions: Sequence[str], references:
  Sequence[str]) -> dict[str, float | int]:
2     if len(predictions) != len(references):
3         raise ValueError("Predictions and references must have
  equal length")
4     normalized_predictions = [normalize_for_evaluation(value)
  for value in predictions]
5     normalized_references = [normalize_for_evaluation(value)
  for value in references]
6     char_edits = sum(
7         levenshtein_distance(list(prediction), list(reference))
8         for prediction, reference in zip(normalized_predictions
  , normalized_references)
9     )
10    reference_characters = sum(len(reference) for reference in
  normalized_references)
11    word_edits = sum(
12        levenshtein_distance(evaluation_tokens(prediction),
  evaluation_tokens(reference))
13        for prediction, reference in zip(normalized_predictions
  , normalized_references)
14    )
15    reference_words = sum(len(evaluation_tokens(reference)) for
  reference in normalized_references)
16    exact = sum(
17        prediction == reference
18        for prediction, reference in zip(normalized_predictions
  , normalized_references)
19    )
20    return {
21        "rows": len(predictions),
22        "cer": char_edits / reference_characters if
  reference_characters else 0.0,
23        "wer": word_edits / reference_words if reference_words
  else 0.0,
24        "chrf": corpus_chrf(normalized_predictions,
  normalized_references),
25        "exact_match": exact / len(predictions) if predictions
  else 0.0,

```

Listing A.3 – Common corpus-level generation metrics.

A.5 Reproduction Sequence

The reproducible execution order is:

1. install the dependencies listed in Appendix C and retain UTF-8 encoding;
2. rebuild or verify the sentence annotations, linguistic annotations, and gold split manifest;
3. run the M0–M6 notebooks individually to reproduce their pairs and manifests, noting that M5 remains an excluded diagnostic;
4. open `m7_tnt_edit_gold_dev_benchmark.ipynb`, instantiate the fixed protocol, and run the dataset-audit and shared-initialization cells;
5. run or resume only the desired condition; completed attempts are reused unless the dataset/configuration fingerprint changes or a rerun is explicitly requested;
6. inspect development predictions and freeze one selection per trained condition without consulting gold-test targets;
7. enable the final-test cell only once all selections are final and its confirmation flag is deliberately set.

Each saved attempt contains its input configuration, run manifest, training history, live progress files, development predictions, metrics, selected model, and curve image. The aggregate `protocol.json`, `dataset_audit.csv`, and selection JSON files provide the compact audit trail for the complete comparison.

Appendix B

Data, Methods, and Annotation Reference

This appendix records the dataset and annotation structures used in the study. Principal results are interpreted in Chapter 5; the final sections below retain fuller cross-tabulations and runtime diagnostics for auditability.

B.1 Experimental Condition Legend

Table B.1 defines the method identifiers used consistently throughout the report and saved artifacts.

Table B.1 – Meaning and final role of each project method identifier.

ID	Method	Role in the final design
M0	Identity / no normalization	Direct reference baseline; the input is scored unchanged and no model is trained on identity pairs.
M1	Beqi-inspired deterministic rules	Retained synthetic condition providing transparent, broad character-level perturbations.
M2	Eflomal alignment plus M1 rules	Retained alignment-informed code-mixing condition.
M3	Distributional code-mixing plus M1 rules	Retained embedding- and alignment-scored code-mixing condition.
M4	External Oolel LLM-generated pairs	Retained external synthetic-data condition, adapted from a pinned dataset snapshot.
M5	YouTube lexical-variant mining	Excluded from the trained benchmark because its accepted mapping coverage and changed-row rate were insufficient. Retained only as a documented prototype.

ID	Method	Role in the final design
M6	Gold-train-informed linguistic engine	Retained proposed method; learns mappings and POS-conditioned transformations only from gold training annotations.
M7	Downstream normalization benchmark	Experiment layer rather than a generator: a fixed model and protocol measure the utility of each retained training corpus.

B.2 Core Dataset Inventory

Table B.2 records the size, purpose, and final status of the core data artifacts.

Table B.2 – Core data artifacts at the methodology snapshot.

Artifact	Size	Purpose and status
data/youtube/comments_data.json	396,559 rows	Raw public-comment research snapshot with source-video provenance.
data/youtube/clean_data.csv	242,174 rows	Deduplicated and mechanically cleaned comment corpus.
data/filtered_data.csv	5,498 rows	Comments retained by the heuristic/local-LLM Wolof filtering workflow.
data/annotations/gold_annotations.csv	477 history rows	Append-only sentence annotation history; the current view contains 201 kept comments.
data/annotations/token_corrections.csv	918 rows	Token decisions saved during sentence normalization.
data/annotations/token_linguistic_annotations.csv	1,315 rows	Gold-training occurrences with linguistic and transformation labels.
data/annotations/splits/	142/30/29	Locked train/development/test pairs from three/one/one disjoint videos.
train-00000-of-00001.parquet	17,777 rows	Common formal Wolof source pool, paired with French in the original resource.
m1_beqi_rules_pairs.csv	17,777 pairs	Complete M1 generation export; 17,017 rows differ from their target.

Artifact	Size	Purpose and status
m2_eflomal_rules_pairs.csv	17,777 pairs	Complete M2 export; 17,180 rows differ from their target.
m3_cmdr_rules_pairs.csv	17,777 pairs	Complete M3 export; 17,102 rows differ from their target.
m4_ooel_external_pairs.csv	3,433 pairs	Accepted external Ooel pairs after five duplicate-pair rejections.
m5_youtube_mined_pairs.csv	17,777 pairs	Excluded prototype export; only 1,821 rows differ from their target.
m6_gold_linguistic_pairs.csv	17,777 pairs	Proposed train-informed export; 14,215 rows differ from their target.
Matched M7 subset	3,330 per method	Identical sampled formal targets used by M1, M2, M3, M4, and M6.

Counts describe the versioned project snapshot, not timeless properties of the original sources. The authoritative provenance record is the manifest stored beside each generated artifact. Manifests contain UTC creation time, input and output hashes, parameter values, rejection counts, and method-specific dependencies.

B.3 Gold Split Contract

Table B.3 summarizes the locked partition sizes and the permitted use of each split.

Table B.3 – Locked gold partition used by the final experiment.

Partition	Rows	Videos	Permitted use
Train	142	3	Gold fine-tuning and M6 knowledge extraction
Development	30	1	Checkpoint selection and diagnostics only
Test	29	1	One-time frozen comparison only

The split seed is 2026. The definitive `split_manifest.json` also stores each file’s SHA-256 digest, the source-annotation digest, and the approved post-annotation revision reason. Video-disjointness prevents comments from the same selected video from appearing across partitions. M6 is additionally restricted to train-partition token annotations; the development and test formal targets may not influence mappings, thresholds, prompts, or generation rules.

B.4 Sentence-Annotation Schema

Table B.4 documents the fields retained in the append-only sentence-annotation history.

Table B.4 – Fields in the append-only sentence-level gold annotation file.

Field	Meaning
source_index	Stable identifier of the candidate comment in the filtered source data.
comment	Authentic informal comment presented to the annotator.
auto_suggestion	Automatically proposed formal version; assistance only, never the gold authority.
manual_formal_wolof	Human-approved normalized sentence used as the formal reference.
status	Current keep/discard decision. The newest history entry for an identifier defines its current state.
video_url	Provenance and grouping key used for video-disjoint splitting.
token_corrections_json	Structured occurrence-level correction decisions recorded during sentence review.

B.5 Linguistic-Annotation Schema and Inventories

The M6 annotation file is occurrence-based: repeated tokens intentionally receive distinct occurrence identifiers, source indices, positions, and timestamps. Its principal analytical fields are shown in Table B.5.

Table B.5 – Principal fields in the gold-training linguistic annotation file.

Field	Meaning
occurrence_id	Composite identifier derived from source index and token position.
informal_token, formal_token	Observed source unit and its normalized target unit; either may contain spaces for phrase-level corrections.
lemma, pos	Reusable lexical form and Universal-POS-style grammatical category.
source_language, target_language	Language of the observed and normalized units.
entity_type	Named-entity interpretation, kept distinct from POS and language.
protected	Whether the generator is prevented from altering the item.

Field	Meaning
<code>error_types_json</code>	One or more confirmed transformation/error labels; identity is exclusive.
<code>transformation_steps_json</code>	Automatically derived character edit script in the formal-to-informal direction.
<code>review_status</code>	reviewed or uncertain ; uncertainty is retained rather than guessed away.
<code>lexical_reusable,</code> <code>lookup_source</code>	Whether lexical fields may be reused and the evidence source used for prefill.
<code>notes, saved_utc</code>	Human/system audit note and save timestamp.

The POS inventory is NOUN, VERB, ADJ, ADV, PROPN, PRON, DET, ADP, AUX, CCONJ, SCONJ, PART, INTJ, NUM, PUNCT, SYM, X, and UNKNOWN. Language values are `wo`, `fr`, `mixed`, `other`, and `unknown`. Entity values are PERSON, LOCATION, ORGANIZATION, EVENT, PRODUCT, WORK, OTHER, NONE, and UNKNOWN. Protection may be `yes`, `no`, `contextual`, or `uncertain`.

B.6 Transformation and Error Labels

Mechanical character labels are proposed from the formal/informal comparison, whereas higher-level interpretations require contextual confirmation. Multiple labels are allowed when they describe distinct aspects of one transformation. For example, removing a word boundary and adding a character can yield both `spacing_merge` and `insertion`.

Table B.6 gives the operational definition shown to the annotator for every available error label.

Table B.6 – Error-label definitions displayed by the linguistic annotation interface.

Label	Operational definition
<code>identity</code>	Informal and formal units are identical; no transformation occurred in this occurrence.
<code>capitalization</code>	Only upper-/lower-case usage differs.
<code>diacritic</code>	A normalized accent or Wolof diacritic is removed, added, or changed.
<code>spacing_split</code>	The informal form introduces a word boundary absent from the formal unit.
<code>spacing_merge</code>	The informal form removes a formal word boundary.
<code>abbreviation</code>	The informal form is a substantially shortened realization of the formal unit.

Label	Operational definition
repetition	Characters or syllable-like sequences are repeated informally.
deletion	Non-space formal characters are absent from the informal form.
insertion	Non-space characters are added to the informal form.
substitution	A character or character sequence is replaced by another sequence.
phonetic_spelling	The informal spelling represents pronunciation rather than standard orthography.
french_spelling	A French item is present in a non-standard or misspelled French form.
lexical_replacement	Informal and formal units are different lexical choices, not only spelling variants.
punctuation	Punctuation is inserted, deleted, or replaced.
multiple	A complex transformation is not adequately represented by the selected specific labels.
other	A known transformation falls outside the available label inventory.
uncertain	The correct interpretation cannot be determined confidently.

The annotation application enforces several consistency rules. A **PERSON** entity is assigned **PROPN** and protected. Unambiguous Wolof and French dictionary hits can be prefilled, but cross-language or POS ambiguity remains visible for manual resolution. Automatically saved identity occurrences retain their own provenance. Spacing merges may be saved under the approved Wolof defaults while leaving POS and entity unknown. Changed occurrences are not skipped merely because the normalized form appeared previously; contextual language and transformation labels remain occurrence-specific.

B.7 Supplementary Results

The authoritative checkpoint selections and test predictions are stored under `results/m7_seq2seq/tnt_edit_gold_dev_v4/`. Reproducible report-level tables and plots are stored under `results/tnt_edit_report_analysis/`; they are rebuilt by `scripts/reporting/build_final_benchmark_analysis.py` without retraining or modifying the frozen outputs.

B.7.1 Complete Gold-Training Error Inventory

Table B.7 retains the complete multi-label inventory underlying the principal counts reported in Chapter 5.

Table B.7 – Complete multi-label error inventory for 1,315 gold-training occurrences.

Label	Occurrences	Share of occurrences (%)
Identity	689	52.40
Phonetic spelling	309	23.50
Substitution	294	22.36
Insertion	250	19.01
Deletion	194	14.75
Diacritic	177	13.46
Spacing merge	98	7.45
Repetition	47	3.57
Abbreviation	20	1.52
French spelling	17	1.29
Multiple	3	0.23
Punctuation	1	0.08
Uncertain	1	0.08

Shares use all 1,315 occurrences as the denominator. Because one occurrence may carry several labels, the rows are not mutually exclusive.

B.7.2 Gold Change Rate by POS

Table B.8 reports the number and proportion of changed occurrences in every observed part-of-speech category.

Table B.8 – Occurrence-level gold-training changes by POS.

POS	Occurrences	Changed	Change rate (%)
VERB	268	207	77.24
UNKNOWN	263	149	56.65
NOUN	235	115	48.94
PROPN	174	0	0.00
DET	105	26	24.76
PRON	70	40	57.14
ADV	53	19	35.85
INTJ	49	29	59.18

POS	Occurrences	Changed	Change rate (%)
CCONJ	46	22	47.83
ADP	21	6	28.57
ADJ	11	2	18.18
PUNCT	10	3	30.00
X	5	5	100.00
PART	4	2	50.00
NUM	1	1	100.00

Rates for categories with very few observations, especially X, PART, and NUM, are not reliable population estimates.

B.7.3 Matched-Corpus Transformation Mechanisms

Table B.9 compares the generator-specific transformation counters recorded for the matched corpora.

Table B.9 – Recorded transformation mechanisms in each matched 3,330-row corpus.

Method	Measure	Total	Mean/row	Rows (%)
M1	Rule edit count	53,318	16.01	99.94
M2	All edits	54,442	16.35	100.00
M2	Alignment substitutions	8,738	2.62	94.53
M2	Spelling-rule edits	45,704	13.72	99.91
M3	All edits	53,124	15.95	99.97
M3	Alignment substitutions	1,525	0.46	32.85
M3	Spelling-rule edits	51,599	15.50	99.91
M4	External exported edits	67,281	20.20	100.00
M6	All linguistic edits	7,825	2.35	93.27
M6	Direct lexical mappings	3,057	0.92	59.61
M6	Contextual transformations	4,768	1.43	76.73

Edit counters are generator-specific provenance fields and are not perfectly interchangeable. They complement, rather than replace, the common CER/WER/chrF comparison in Chapter 5.

B.7.4 Matching and Leakage Audit

Table B.10 reports the common-target selection counts and confirms the absence of exact held-out overlap.

Table B.10 – Dataset matching and held-out overlap audit.

Method	Loaded	Unique pairs	Targets	Selected	Dev ov.	Test ov.
M1	17,777	16,377	16,375	3,330	0	0
M2	17,777	16,544	16,375	3,330	0	0
M3	17,777	16,384	16,375	3,330	0	0
M4	3,433	3,433	3,430	3,330	0	0
M6	17,777	16,427	16,375	3,330	0	0

“Dev ov.” and “Test ov.” denote both exact input and exact pair overlap; all four audited overlap fields are zero for every method. The common target pool contains 3,430 unique targets. Between six and ten selected sources per method exceed 255 characters and are truncated to the model’s 256-position limit after special-token handling; six common targets exceed that threshold.

B.7.5 Runtime and Checkpoint Diagnostics

Table B.11 records the observed training time and selected gold-stage epoch for each condition.

Table B.11 – Recorded training runtime and selected epoch by condition.

Condition	Synthetic h	Gold h	Total h	Selected gold epoch
Gold only	—	0.20	0.20	5
M1 + gold	1.33	0.20	1.53	15
M2 + gold	1.28	0.20	1.47	10
M3 + gold	1.34	0.20	1.54	20
M4 + gold	1.34	0.20	1.54	5
M6 + gold	1.31	0.20	1.50	15

The selected synthetic epochs are 15 for M1, 10 for M2–M4, and 20 for M6. Runtimes are those reported by the training library on the recorded laptop runs. They include optimization, scheduled Gold-development reconstruction, logging, and checkpoint saving; they are operational measurements rather than controlled hardware benchmarks.

Appendix C

Technical Environment

C.1 Recorded Final-Benchmark Environment

Table C.1 reports the environment recorded by the final TNT-inspired benchmark manifests. Version ranges in `requirements.txt` are intentionally broader; the exact run manifest is the authority for a completed experiment.

Table C.1 – Principal environment recorded for the final benchmark.

Component	Recorded value
Host operating system	Microsoft Windows; Linux/WSL used only to rebuild Eflomal artifacts
Python	3.11.0
PyTorch	2.8.0 with CUDA 12.9 build
Transformers	5.5.4
Datasets	2.20.0
Accelerator	NVIDIA GeForce RTX 5060 Laptop GPU
Reported video memory	8,151 MiB
Numeric configuration	Float32 model parameters; TF32 enabled; FP16 and BF16 disabled
Optimizer	Fused PyTorch AdamW with weight decay 0.01
Scheduler	Constant learning rate with 5% warm-up
Experiment seed	2026 for sampling, initialization, and training controls

Strict deterministic CUDA kernels are disabled for throughput, so a repeated run with the same seed need not be bitwise identical across devices or software builds. The records do not reliably capture the exact CPU model, system RAM, total wall-clock time, or energy use for every condition, and I therefore make no retrospective numerical claims for them.

C.2 Environment Separation

Table C.2 distinguishes the execution environments required by the principal project components.

Table C.2 – Execution environments used by project components.

Environment	Components	Reason
Windows/Python	Annotation interfaces, generators, TNT-inspired edit Transformer, metrics, and reports	Primary development and model-training environment.
Linux/WSL	Eflomal compilation and alignment notebook	Native compilation is substantially simpler; the resulting lexicon is portable to Windows.
Local Ollama service	Gemma comment-filtering stage	Local inference supports resumable candidate filtering without sending the comment corpus to a paid cloud service.
Browser	Flask annotation interfaces on local ports 5000 and 5001	Human sentence and linguistic review; data are saved to local CSV files.

C.3 Declared Python Dependencies

Listing C.1 reproduces the project’s install specification. Some transitive libraries, including NumPy and the plotting backend, are resolved by the package installer and are recorded indirectly in the runtime environment rather than pinned in this file.

```

1 youtube-comment-downloader
2 yt-dlp
3 langdetect
4 pandas
5 lingua-language-detector
6 polyglot
7 streamlit
8 Flask>=3.1,<4
9 nltk>=3.9,<4
10 datasets
11 pyarrow
12 rapidfuzz
13 torch>=2.4,<3
14 transformers>=4.46,<6
15 accelerate>=1,<2
16 sentencepiece>=0.2,<1
17 peft>=0.18,<1
18 matplotlib>=3.8,<4

```

Listing C.1 – Project dependency specification (requirements.txt).

The standard Windows installation command is:

```
1 python -m pip install -r requirements.txt
```

Listing C.2 – Install the declared Python dependencies.

C.4 Operational Entry Points

```
1 # Sentence-level gold annotation (http://127.0.0.1:5000)
2 python -m src.annotation.flask_annotation_app
3
4 # Gold-train linguistic annotation (http://127.0.0.1:5001)
5 python -m src.annotation.flask_linguistic_annotation_app
6
7 # Rebuild the multilingual annotation lookup
8 python -m src.pipelines.prepare_multilingual_lookup_seed --
   overwrite
9
10 # Run automated workflow and integrity tests
11 python -m pytest
```

Listing C.3 – Principal local application and audit entry points.

The final benchmark is controlled from the M7 TNT edit notebook listed in Appendix A. Its Boolean switches permit a single method to be trained or resumed without rerunning completed methods. The final-test switch was enabled only after every development selection existed and the separate confirmation flag was set; the resulting selection fingerprint and frozen predictions are now part of the submitted evidence.

C.5 Artifact Layout

```
1 PFE/
2 |-- data/
3 |   |-- annotations/      # gold history, lookups, and locked
   splits
4 |   |-- external/         # pinned external dataset snapshots
5 |   |-- synthetic/        # M0--M6 pair exports and manifests
6 |   '-- youtube/         # raw and cleaned comment snapshots
7 |-- artifacts/           # lexicons, embeddings, mappings,
   profiles
8 |-- checkpoints/
```

```
9 |   '-- m7_seq2seq/           # immutable versioned training
   attempts
10 |-- notebooks/             # inspectable pipeline controllers
11 |-- results/
12 |   '-- m7_seq2seq/         # audits, selections, plots, frozen
   test
13 |-- src/                   # reusable annotation, generation,
   modeling code
14 |-- tests/                 # data and experiment integrity
   checks
15 '-- requirements.txt
```

Listing C.4 – Logical project artifact layout.

Large raw comments and trained checkpoints are handled separately from the lightweight public source repository according to their licenses and privacy constraints. A complete submission archive consists of the report, source commit, dependency file, dataset manifests, split manifest, selection records, frozen-test manifest, and the tables and figures used in Chapters 5 and 6.

Appendix D

Declaration of Generative Artificial-Intelligence Use

In accordance with the aivancity PFE guidelines, this appendix explains how I used generative-AI tools during the project. They supported research, implementation, and writing, but I remained responsible for the research question, annotation decisions, experimental protocol, interpretation, and final claims.

D.1 Tools and Purposes

Table D.1 distinguishes the tasks assisted by each tool from the decisions and checks I carried out myself.

Table D.1 – Declared uses of generative artificial intelligence.

Tool or system	Use in the project	My control and verification
OpenAI Codex	Interactive assistance with code and notebook implementation, refactoring, debugging, experiment orchestration, result extraction, LaTeX drafting, structural revision, and language editing.	I set the objectives, changed the project direction when the evidence required it, selected the methods and architectures to retain, ran the notebooks and training, inspected predictions and artifacts, questioned proposed interpretations, and accepted or rejected changes. I checked the reported numbers against saved experiment outputs.

Tool or system	Use in the project	My control and verification
Google Scholar’s AI-assisted discovery features	Identification and initial triage of potentially relevant publications.	I read the underlying papers and supplied the relevant sources for analysis. Citations in the report refer to the original publications rather than to generated summaries.
Local Gemma model through Ollama	Candidate filtering of cleaned YouTube comments to reduce the amount of material presented for manual annotation.	I treated the model output only as a prioritisation aid and made the keep, discard, correction, and uncertainty decisions used to construct the gold benchmark.
External Oolel re-source	The M4 benchmark condition uses a published non-standard Wolof dataset whose documentation identifies its informal side as language-model-generated.	I imported the resource at a pinned revision, validated it mechanically, kept it separate from the proposed method, and evaluated it under the same matched-target protocol as the other synthetic sources.

D.2 Extent and Limits of Assistance

Codex produced substantial implementation drafts under my iterative direction. These drafts covered annotation interfaces, generation and modeling modules, notebook controllers, validation checks, reporting utilities, and revisions to the written report. I remained responsible for understanding the resulting code, running it in the recorded environments, reviewing failures and predictions, deciding when to modify or abandon an approach, and ensuring that the final methodology matched the intended experiment. In particular, I directed the shift from semantic retrieval to synthetic-data evaluation, the move from large pretrained sequence-to-sequence models to compact and copy-aware experiments, and the final adoption of the TNT-inspired edit evaluator. These decisions followed the feasibility and model behavior I observed during the project.

I did not accept generative-model output as ground truth for the manually normalized gold sentences or occurrence-level linguistic annotations. Automatic suggestions and lookup-based prefills served as review aids, and I confirmed or corrected every retained annotation. The benchmark measurements were likewise taken from versioned training histories, predictions, manifests, and analysis files produced by the implemented pipeline

rather than generated by an AI assistant.

I applied the same source checking to AI-assisted prose as to manually drafted text. I compared method descriptions with the implementation and run artifacts, checked literature claims against the cited publications, and limited the conclusions to the completed evidence. I assume full responsibility for the accuracy, originality, and interpretation of this report.